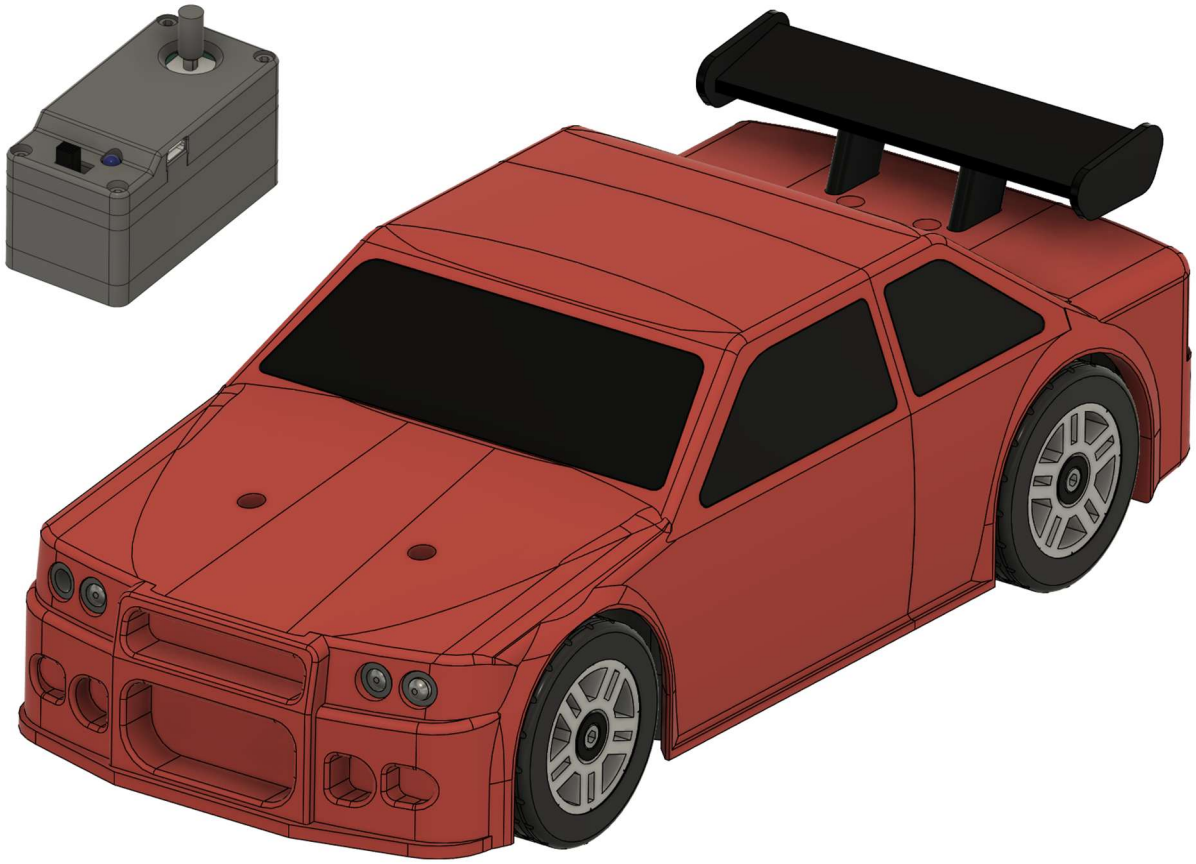
EasyRC – open-source RC platform



Manual

Version 1.2; Date: 09.01.2026

Contents

1. Remarks and specifications

1.1 Safety information

EasyRC is an open-source platform for wirelessly controlled model vehicles, which are assembled from 3D-printed components. This manual must be read fully and understood prior to the assembly to recognize possible hazards and avoid injuries or damage.

Generally, remote-controlled vehicles are not permitted on public roads. Pilots of RC vehicles are at risk of losing control over the vehicle in case of interference or if the maximum transmission distance is exceeded. EasyRC mitigates this risk by incorporating a safety routine that turns off the vehicle's motor if no signals from the remote control are received for at least 1 s (the remote control transmits 20 data packets per second, meaning a single lost data packet cannot trigger this safety routine). However, this safety routine remains ineffective if the vehicle receives interfering signals that are falsely interpreted as commands from the remote control. To this end, the vehicle checks the individual hardware address (MAC address) of the remote control that is sent with every data packet. The data is only further processed if this address matches the internally saved hardware address of the paired remote control. Therefore, it is essential to program every pair of remote control and vehicle with a unique set of hardware addresses if several EasyRC models are to be used simultaneously, meaning that one remote control will only control one vehicle. The wireless connection operates at a frequency of 2.4 GHz, which is also used for common WiFi signals. Hence, connection issues may occur in buildings with many wireless networks. If connection issues are frequent, EasyRC's radio channel can be changed.

The EasyRC platform utilizes batteries with a system voltage of up to approximately 12 V DC. This voltage is generally considered harmless to humans; there is no risk of dangerous electric shocks. Furthermore, the vehicle's circuit board is protected from high currents by a fuse. Still, batteries can be damaged by improper use (e.g., overcharging, short circuits, physical damage), which may result in incineration and the release of harmful substances. Therefore, the employed lithium-ion batteries must only be charged with suitable chargers. The maximum cell voltage (charging end voltage) of a single cell equals 4.2 V. This voltage must not be exceeded! The batteries must not be used further if they are physically damaged or have been overcharged. Short circuits must be avoided. Additionally, deep discharging (<2.5 V) irreversibly damages a cell. The circuit boards of the EasyRC platform are protected against reverse polarity. However, since the vehicle requires three cells in series, the user has to carefully check for correct polarity when installing the single cells to avoid damaging the batteries. Batteries and electronic components must not be discarded as residual waste, but must be disposed of or recycled in an environmentally compatible manner based on local available options.

1.2 EasyRC specifications

1.2.1 Remote control

Size: 80x40x41 mm^3 (LxWxH)
Weight: approx. 125 g (including battery)
Microcontroller: ESP32-C3
Wireless communication: 2.4 GHz (ESP-NOW protocol)
Joystick: Alps 10k (Playstation 4 model)
Power supply: 1x 18650 Li-Ion battery (3,7 V)
Power consumption: approx. 0.5 W
Notes:

- Battery connector is reverse polarity protected
- Low battery indicator (<3,4 V): LED starts blinking
- Self-calibrating joystick

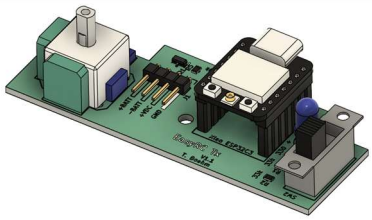
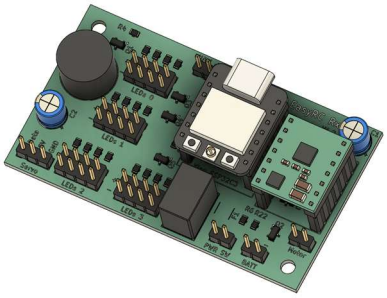
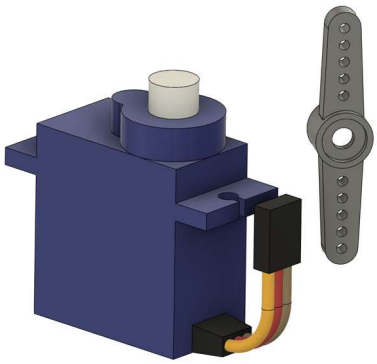
1.2.2 Vehicle

Size: 282x125x95 mm^3 (LxWxH)
Scale: 1:16
Weight: approx. 1 kg (including batteries)
Microcontroller: ESP32-C3
Wireless communication: 2.4 GHz (ESP-NOW protocol)
Motor driver: Pololu DRV8876 (QFN)
Drive motor: JGA25-370 Brushed DC motor with gearbox (12 V, 1000 U/min)
Speed: max. 10 km/h
Steering servo: MG90S metal-gear micro servo
Power supply: 3x 18650 Li-Ion battery (11.1 V)
Power consumption: approx. 0,8 W (idle); approx. 3-6 W (driving)
Notes:

- Battery connector is reverse polarity protected
- Low battery warning (<10,2 V): Motor disabled, buzzer beeps
- Connection loss warning (> 1s): Motor disabled, buzzer beeps
- No user input warning (no inputs for >1 min): Buzzer beeps
- Self-resetting fuse in case of too high currents (>1.5 A)

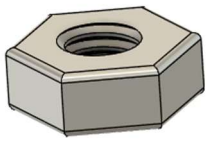
2. Part lists

2.1 Electronic components

Figure	Designation	Description
	1x EasyRC Tx circuit board	Circuit board with components for the remote control. Includes the joystick as well as the microcontroller that establishes the wireless connection. Component list in chapter 3.4.
	1x EasyRC Rx circuit board	Circuit board with components for the car. Includes the microcontroller for controlling the car and the motor driver for the drive motor. Component list in chapter 3.4.
	1x RC servo motor, type MG90S, including servo horn Color coding of the connection cable: Yellow: Data Red: +5 V Brown: GND	Motor with integrated gearbox and electronics that adjusts the output shaft within an angle interval of approx. 180° based on its input signal. The servo horn is used as a connector to the component that shall be moved and is screwed onto the servo's output shaft. Important: Verify correct polarity! Reverse polarity destroys the servo motor!
	1x Geared DC motor JGA25-370 (1000 U/min) The cable can be color-coded: Red: +12 V Black: GND	Motor with integrated 1:10 gearbox. The motor reaches 1,000 rpm at 12 V. The motor shaft is D-shaped to allow stable power transmission. The motor's rotation direction depends on the orientation of the connector.

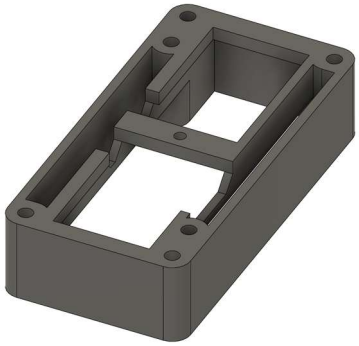
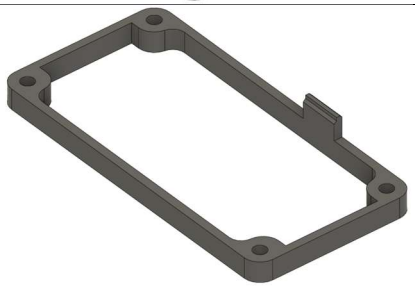
	8x wired 5-mm-LED The cable should be color-coded: Red: +5 V* (long pin) Black: GND (short pin) *Only valid for EasyRC: The series resistors that are required for the LEDs are integrated in the EasyRC circuit board.	Light-emitting diodes in different colors (2x white, 4x yellow, and 2x red). Type: Quadrios 2111O182 (white) Quadrios 2111O171 (yellow) Quadrios 2111O172 (red) Important: Use sufficiently long wires when soldering wires to the LEDs (30-40 cm) and verify correct polarity. LEDs do not work and can be damaged when incorrectly connected!
	4x 18650 Li Ion battery Voltage window: 4.2 V (fully charged) 3.3 V (discharged)	Typ: Lishen LR 1865SS 3.1 Ah Lithium-ion battery for operating EasyRC. The batteries must be charged with a dedicated charger. EasyRC monitors battery voltage and warns the user when the batteries are drained.
	Battery holder for single cells Required sizes: 1x 3 18650 cells 1x 1 18650 cells	Type: Battery holders with loose wires Important: Always confirm correct polarity before inserting cells! The batteries and connected components may be damaged by reverse polarity.
	1x On/off switch	Type: Marquardt 01801.1148-02 Switches the vehicle on and off.

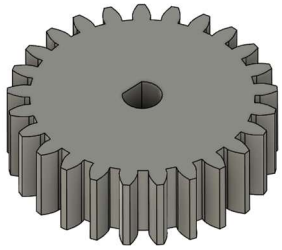
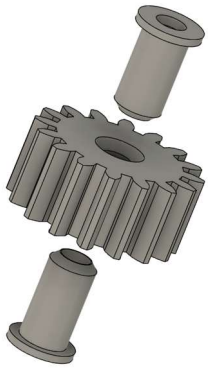
2.2 Metal parts

Figure	Designation	Description
	M3 cylinder head screw Required sizes: 29x M3x8 mm 4x M3x12 mm 7x M3x16 mm 6x M3x20 mm 2x M3x22 mm 17x M3x30 mm 2x M3x35 mm 2x M3x40 mm	Screw with metric 3 mm threading. A 2.5 mm hex key drives the head.
	64x M3 hex nut	Nut for screws with metric 3 mm threading.
	M3 countersunk screw Required size: 3x M3x10 mm	Screw with metric 3 mm threading. A 2.0 mm hex key drives the head.
	M2.5 screw Required size: 1x M2.5x6 mm	Screw with metric 2.5 mm threading. Required to connect the servo horn to the servo motor.
	Ball bearings Required types: 4x 605 (5x14x5 mm) 4x 608 (8x22x7 mm) 2x 6802 (15x24x5 mm)	The sizes of the bearings are described by the inner diameter x outer diameter x height.

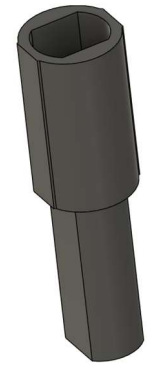
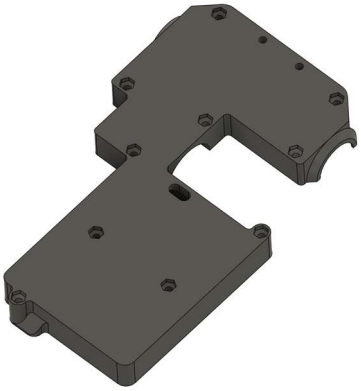
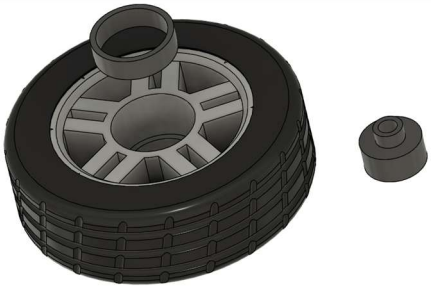
2.3 Printed parts

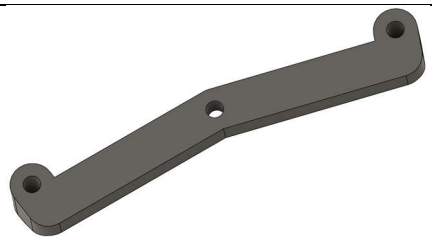
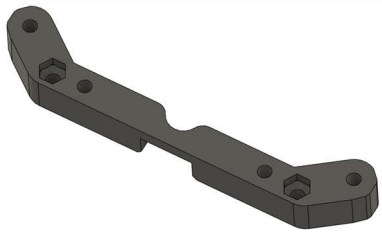
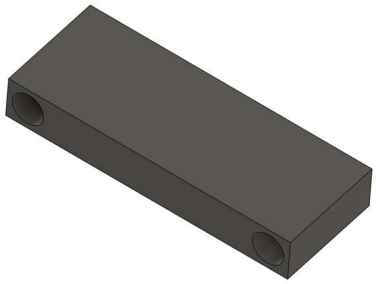
Orientations given in the hints column always refer to the part's orientation with respect to the complete model, not the orientation shown in the figure column.

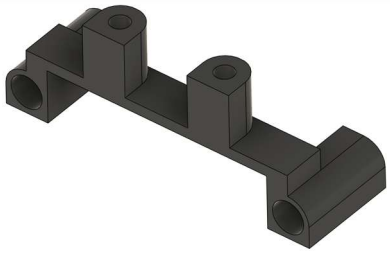
Figure	Designation, chapter	Hints for printing
	1x Remote control lower intermediate frame Chapter 4.1.1	Upper side toward the print bed; support is recommended.
	1x Remote control top Chapter 4.1.2	Bottom toward the print bed; support is required (organic support recommended).
	1x Remote control upper intermediate frame Chapter 4.1.2	Bottom toward the print bed; support is not needed.
	1x Remote control bottom Chapter 4.1.2	Upper side toward the print bed; support is not needed.

	1x Remote control joystick extension Chapter 4.1.3	Upper side toward the print bed; support is not needed.
	1x Remote control battery cover Chapter 4.1.3	Bottom toward the print bed; support is not needed.
	1x Motor gear (25 teeth) Chapter 4.2.1	Gear with flat side toward the print bed (should be printed as a single part).
	1x Intermediate gear (16 teeth) with 2x bearing mount. Chapter 4.2.2	Gear with flat side toward the print bed (should be printed as a single part). Bearing mount with the outer side toward the print bed. No support needed.
	1x Differential gear (24 teeth) with spacer ring. Chapter 4.2.3	Gear with flat side toward the print bed (should be printed as a single part).

	2x Crown gear (15 teeth) Chapter 4.2.3	Gear is required twice. Print with the teeth facing upward. Support is mandatory!
	2x Crown gear (10 teeth) with 1x spacer Chapter 4.2.3	Gear is required twice. Print with the teeth facing upward. No support needed.
	2x Differential case Chapter 4.2.3	The differential case is required twice. Print with the outside facing upward. Support is mandatory (organic support is recommended).
	2x Rear rim with tire and spacer Chapter 4.2.4	Print rim with the outside facing toward the print bed. Support is required. Print the spacer with the inside facing toward the print bed. Print the tire from very soft TPU if possible.

	2x Rear axle (in two versions with 2.5 mm length difference) Chapter 4.2.4	Axles should be printed from TPU (hard TPU is recommended). Print axles with the flattened side toward the print bed. No support needed.
	1x Chassis bottom Chapter 4.2.5	Bottom side toward the print bed. Support is needed.
	1x Chassis top Chapter 4.2.5	Upper side toward the print bed. No support needed.
	2x Front rim with tire, locking ring, and spacer Chapter 4.3.1	Print rim with the outside facing toward the print bed. Print spacer with the outside facing toward the print bed. Support is required for both components. Print the tire very soft TPU if possible.
	2x Front wheel mount (mirrored versions for left and right) with spacer and steering linkage connector Chapter 4.3.1, 4.3.2	Print front wheel mount with the bottom toward the print bed. Steering linkage connector with the upper side toward the print bed. Spacer with the inside toward the print bed. No support needed.

	1x Steering linkage Chapter 4.3.2	Upper side facing toward the print bed. No support needed.
	1x Chassis securing bracket Chapter 4.3.3	Upper side facing toward the print bed. No support needed.
	1x Battery lid Chapter 4.4.2	Bottom facing toward the print bed. No support needed.
	1x Rear bumper Chapter 4.4.3	Upper side facing toward the print bed. No support needed.
	1x Front bumper Chapter 4.4.3	Bottom facing toward the print bed. No support needed.

	1x Servo horn attachment Chapter 4.4.5	Upper side toward the print bed. No support needed. Print from TPU. The attachment is available in a large and a small version. The version that fits the servo horn has to be printed (see 4.4.5).
	1x Rear body Chapter 4.5.1	The side that connects to the front body has to face the print bed. Organic support is required. Activate the brim to improve adhesion.
	1x Rear wing holder Chapter 4.5.1	Inner side (the side that is oriented toward the vehicle's front) facing toward the print bed. No support needed.
	1x Rear wing Chapter 4.5.1	Rear side toward the print bed. Support required. Activate the brim to improve adhesion.
	1x Front body Chapter 4.5.2	The side that connects to the rear body has to face the print bed. Organic support is required. Activate the brim to improve adhesion.

	Windows (1x front, 1x left front, 1x left rear, 1x right front, 1x right rear, 1x rear) Chapter 4.5.2	Print windows with the outside toward the print bed. No support needed.
	1x Window holder Chapter 4.5.2	Upper side facing toward the print bed. Support is recommended.
	1x Front light clamp with 2x LED insert Chapter 4.5.3	Print the front light clamp with the upper side toward the print bed. Support is required. Print LED inserts with their inner sides (facing the light clamp) toward the print bed. The LED insert is required twice.
	2x Rear light clamp and exhaust with LED insert (LED insert in mirrored versions for left and right) Chapter 4.5.4	Print the rear light clamp and LED inserts with their inner sides (the sides facing the vehicle's front) toward the print bed. No support needed. The clamp is needed twice.

3. Electronics

3.1 Introduction

EasyRC utilizes the ESP32 microcontroller for the remote control, which detects user inputs via the joystick and sends them wirelessly to the vehicle. The vehicle incorporates another ESP32 microcontroller to receive these data packets and convert them into commands for the steering and drive motors. A small developer board, the Xiao ESP32-C3 from SEEEDStudio is used because this platform includes all required components to program the microcontroller (via USB-C), to provide 3.3 V power to the microcontroller from higher input voltages, and because it provides easily accessible pins to connect to peripherals. The ESP32 is ideally suited for such a system because it is an inexpensive microcontroller that offers all required functions without having to use additional chips: It not only includes digital inputs and outputs (to read simple switches and output on/off commands) but also analog inputs to read sensors with non-discrete measurement signals and convert them into digital data. To this end, it incorporates analog-digital converters that convert variable measurement signals, such as a joystick axis, into processable numbers for the microcontroller at high resolution.

Furthermore, it offers PWM-compatible outputs, allowing for equally fine control of the steering servo and the drive motor. PWM is the abbreviation for pulse width modulation and is a means to transmit a finely differentiated signal via a digital pin that can only output 0 and 1 by considering the time component (for how long per output time interval is the output set to 0 and 1? ("PWM duty cycle")). Here, the ESP32 can adjust the output time interval, also referred to as PWM frequency, over a very broad range. It can apply different frequencies to different output pins simultaneously. Lastly, the ESP32 features a module for wireless communication. It transmits at 2.4 GHz and, thus, can communicate with standard WiFi networks. In this case, this option is not used, but the proprietary ESP-NOW communication protocol of the ESP32 is employed. This protocol enables establishing a connection between two ESP32 microcontrollers, regardless of the presence or absence of WLAN, allowing the remote control to send its data packets directly to the vehicle. Furthermore, the Xiao ESP32-C3 developer board comes with an external antenna. This antenna must be installed to achieve the required radio range for a remote-controlled vehicle.

Thus, the ESP32 includes all base functions to be used as a radio and control unit for the EasyRC platform. Nonetheless, circuit boards are needed that contain additional components to allow the remote control and vehicle to be used. For example, the LED lighting in the car uses transistors as switches because the LEDs require more current than the microcontroller can provide directly. It switches a separate power supply for the LEDs using the transistor. The drive motor requires a separate motor driver that accomplishes a similar task to the transistor for the LEDs, but it must do even more than that. It allows adjusting the drive direction and speed of a brushed DC motor by using only two pins (one sets the direction and the other sets the speed). Furthermore, some smaller additions, such as resistors used as voltage dividers to allow the ESP32 to read battery voltages, or a transistor-based reverse polarity protection that prevents the boards from harm in case of an incorrectly connected battery, are included.

3.2 Circuit board manufacturing and assembly

The production files for the printed circuit boards are available on GitHub (https://github.com/TRD-B/EasyRC). They can be ordered from PCB manufacturers such as JLCPCB, PCBWay, or Aisler and then assembled with the required components. Here, lead-free solder shall be used (health protection!). The components are mostly SMD parts, which means devices that are placed on top of the circuit board and then soldered onto it. This process is a bit trickier than for THT components, whose pins are pushed through holes in the board and soldered on their bottom side, because the SMD parts are not kept in place mechanically during soldering. On the other hand, SMD components are smaller than comparable THT components, which allows circuit boards to be kept more compact. Further, only SMD part sizes large enough to allow hand soldering without special tools are used (smallest size: 0805 (2 mm x 1.25 mm)).

As a general rule of thumb for assembling a PCB, start with the smallest SMD component. Then, increasingly larger components are soldered to the board, and the THT parts are added only in the end. This way, no already soldered component can act as an obstacle in subsequent steps. For comfortable soldering, a "helping hand" - a holder with flexible arms - is recommended. The SMD parts can be soldered most easily by holding them with a pair of tweezers with delicate tips and then soldering their first contact pad. As soon as they are fixed at one point, the tweezers are no longer needed to keep them in place, and the remaining contact pads can be soldered. Importantly, when soldering the remaining contact pads, the first contact must not be melted again, as it would then become loose again instantly. The required SMD components can be easily oriented on the board. Resistors and ceramic capacitors have only two contacts and are non-polarized, meaning they do not require a specific mounting direction. The transistors have three contacts, but due to their geometry, they cannot be oriented incorrectly (when the three pins are in touch with the pads on the circuit board, the transistor is oriented correctly).

On the other hand, the THT components require more attention regarding the corrected mounting orientation. The piezo buzzer is marked with "+" at one side, and the electrolytic capacitors are also polarized components, which means they have to be placed with the anode (short pin) to the positive side and the cathode (long pin; white broad stripe on the case with a black "-") to the negative side. The same holds true for LEDs, whose pins are also polarized (anode short, cathode long). When soldering the pin headers required to connect external components to the board, a female header strip can be used as an aid to keep the pin header vertical during the soldering process. Alternatively, fine tweezers can also be used. Finally, the large components (joystick module of the remote control, DC-DC converter, piezo buzzer, and electrolytic capacitors on the vehicle's board) should be soldered.

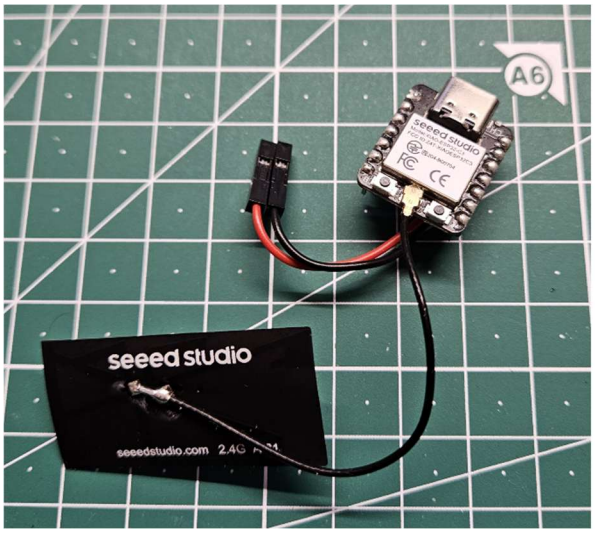
After assembling the PCBs, the contacts must be checked. Each connection needs to be mechanically stable (pushing them gently with a finger must not detach the component), and there must not be any solder bridges between different contacts (solder that forms an unwanted connection between adjacent pads on the board). Only after confirming correct soldering by visual inspection should the ESP32-C3 developer boards and the motor driver be pushed into their headers on the boards and connected to power. When adding these smaller boards to the EasyRC PCBs, the correct orientation must be verified, as incorrect (180° rotated) mounting can result in malfunction and damage to the components.

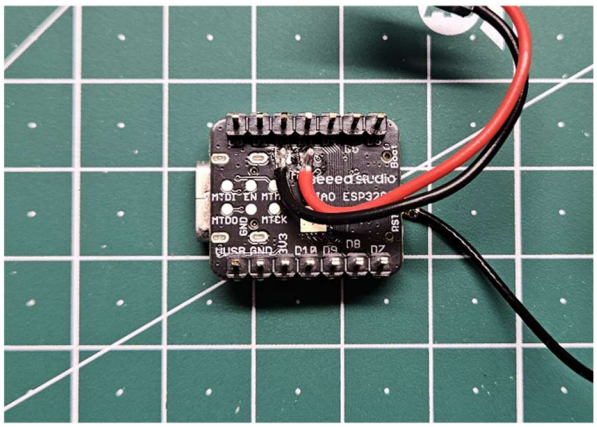
3.3 Connecting electronic components

The connection between external electric components (see Chapter 2.1) and the circuit boards should be established with 0.25 mm^2 diameter copper wire. This diameter is more than sufficient for, e.g., the LEDs, but it is also suitable for the drive motor and the on/off switch of the vehicle. Hence, only one wire size is required. It is recommended to use single-stranded copper wire with two different isolation colors, namely red and black. Black is commonly used to connect the negative side, and red is typically used to connect the positive side, which helps to connect polarized components without problems.

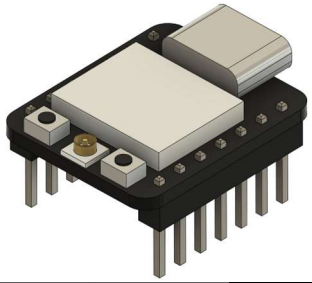
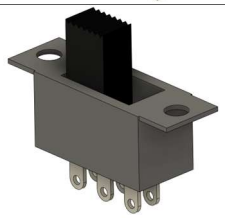
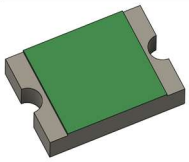
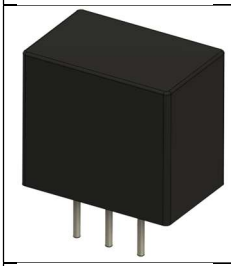
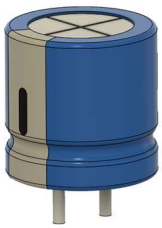
The electronic components are soldered to the copper wires and receive crimp connectors (female 2.54 mm DuPont connectors) at the end of the wires, which allows to (un)plug them easily. When crimping connectors to the wires, care must be taken to ensure the connection is mechanically stable: Even when pulling the wire, it must not loosen and slip out of the crimp pin. Depending on the crimping tool, the wire and the isolation are either crimped in one action or two separate steps are needed for both processes. If both are crimped at the same time, it may happen that the isolation is not crimped with sufficient force, meaning the connection has to be finalized with pliers to ensure the pin can be pushed into the pin housing without resistance.

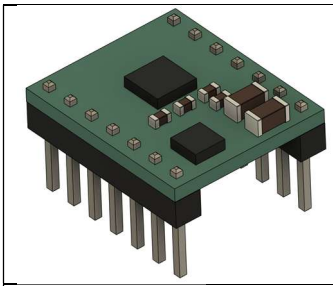
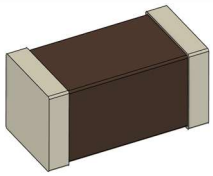
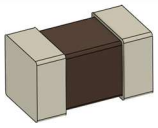
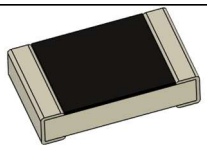
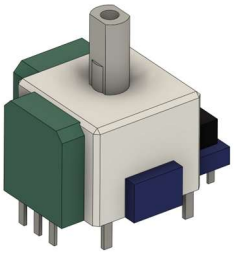
The images below show the Xiao ESP32-C3 as it is required for the remote control. The antenna needs to be connected to the vehicle's microcontroller, too, but only the microcontroller for the remote control requires soldering wires to the power supply pads on the board's bottom. For the remote control, the wires between the ESP32-C3 and the board, as well as between the battery holder and the board, must be kept short so that the case can be closed easily (see also Chapter 4.1.1). For the car, the cables should be kept a bit too long rather than too short, as the wires can be sorted more easily. Especially for the LEDs mounted in the vehicle's body, long enough wires are needed to allow for the attachment and removal of the body without inadvertently unplugging the LEDs. The motor and on/off switch of the car do not require color-coded wires because the connection orientation of the switch does not matter, and the rotation direction of the motor, which depends on its polarization, can be easily corrected in the remote control's code (Chapter 5.2).

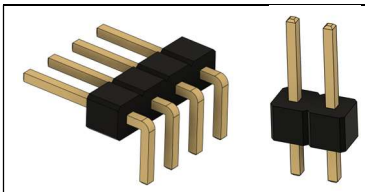
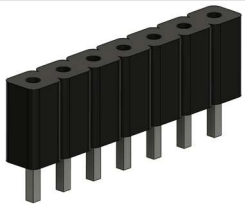




3.4 Component list

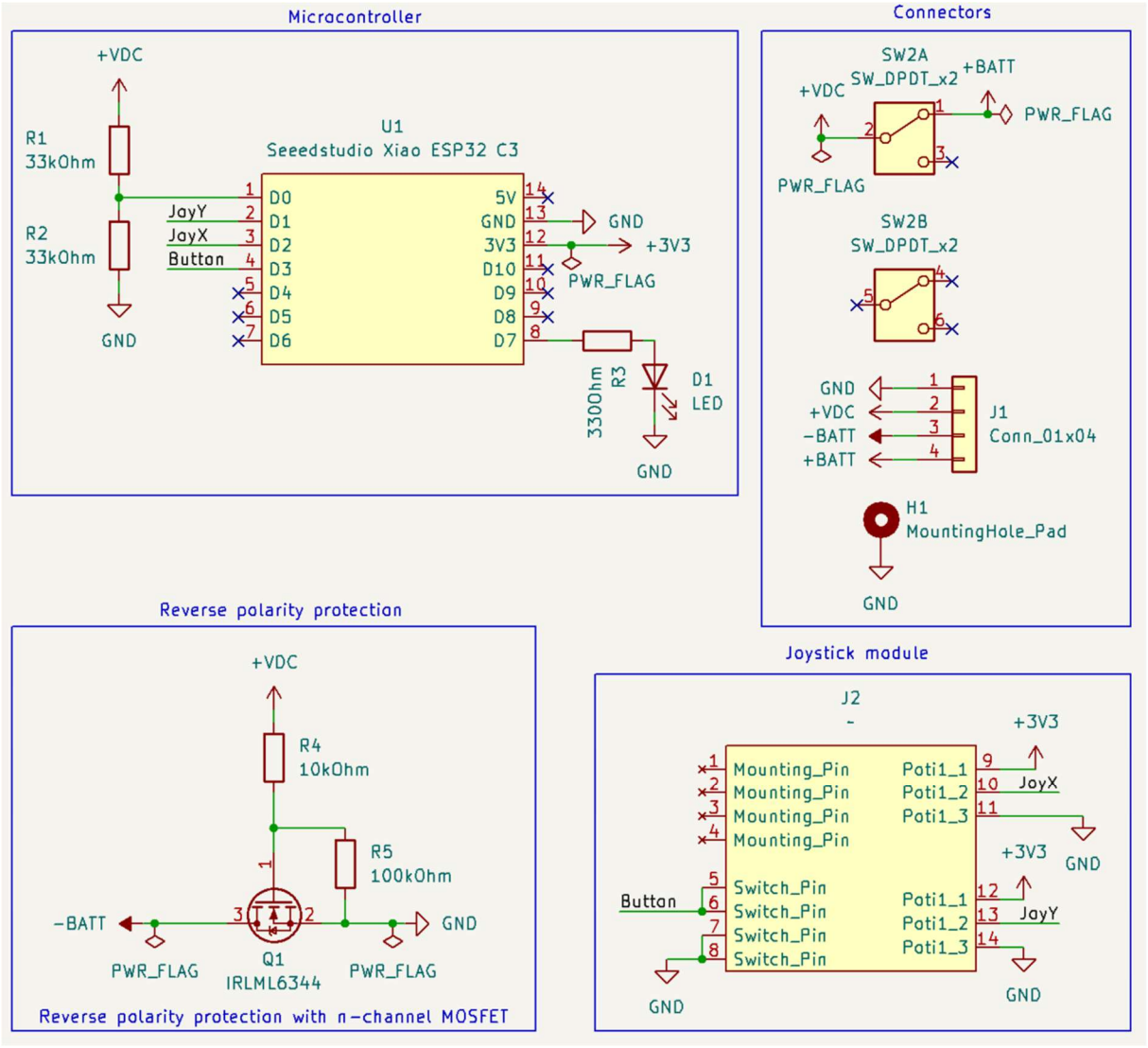
Figure	Designation	Description
	2x Microcontroller development board	Type: SEEED Studio Xiao ESP32-C3
	1x Miniature slide switch	6 pins, type Wentronic S 166. Slider height: 6-10 mm
	1x Self-resetting PTC fuse, 1.5 A	Form factor: SMD 1812 Type: ESKA FSMD1812 (FSMD150-24R)
	1x DC-DC converter, 5 V 1 A	3-pin form factor, pin compatible with LMxx linear regulators Type: Traco TSR1-2450E
	2x electrolytic capacitor, 25 V 100 µF	6.3 mm diameter, 2.5 mm pitch. Type: Panasonic EEUFM1E101
	1x Piezo buzzer	12 mm diameter, 6.5 mm pitch Type: HITPOINT PB1220PE05Q
	1x 5mm wired LED, green	Type: Quadrios 2111O169

	1x motor driver breakout board	Type: Pololu DRV8876QFN
	6x MOSFET n-channel, 5 A 30 V	Form factor: SMD SOT-23 Type: Infineon IRLML 6344
	1x Zener diode, 8.2 V	Form factor: SMD SOT-23 Type: CDIL BZX84C8,2
	2x ceramic capacitor, 10 V 22 µF	Form factor: SMD 1206 Type: Murata GRM31CR71A226KE15L
	1x ceramic capacitor, 50 V 100 nF	Form factor: SMD 0805 Type: Walsin 0805B104K500CT
	Resistor, 125 mW Required resistance values: 1x 330 Ohm 16x 470 Ohm 4x 1 kOhm 1x 10 kOhm 3x 22 kOhm 2x 33 kOhm 2x 100 kOhm	Form factor: SMD 0805 Type: Walsin WR08X
	Joystick module (compatible with PS4 controller)	Type: ALPS 10kOhm

	Pin header, 2.54 mm pitch Required sizes: 1x 1x4 (angled) 4x 1x2 (straight) 1x 1x3 (straight) 4x 2x4 (straight)	Type: W+P 946-14-004-00 (1x4 angled) MPE 087-1-002-0-S-XS0-1260 (1x20, can be cut) MPE 087-2-040-0-S-XS0-1260 (2x20, can be cut)
	1x Short-circuit bridge, 2.54 mm pitch	Type: Jumper 2.54 mm black
	Socket strip, 2.54 mm pitch. Required sizes: 4x 1x7 (8.5 mm height) 2x 1x7 (5 mm height)	Type: MPE 094-1-007-0-NFX-YS0 (8.5 mm height) Frei BL 1X10G 2,54 (5 mm height, has to be cut to a size of 1x7) The remote control requires shorter sockets to maintain the overall height of the board at a smaller size. Therefore, the pins of the microcontroller board for the remote control need to be shortened by 2-3 mm to fit into the shorter sockets.

3.5 Overview of remote control electronics

3.5.1 Circuit diagram



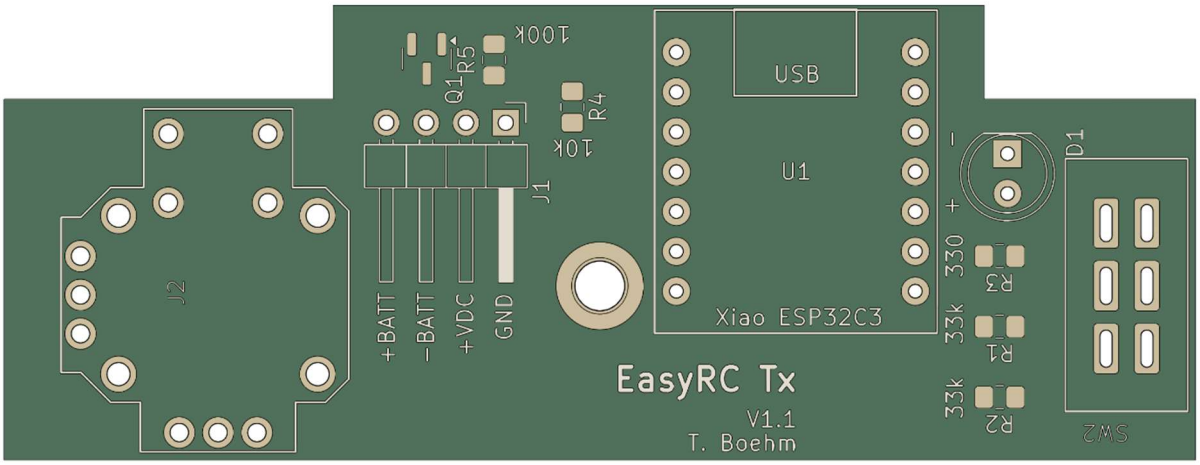
Net assignments:

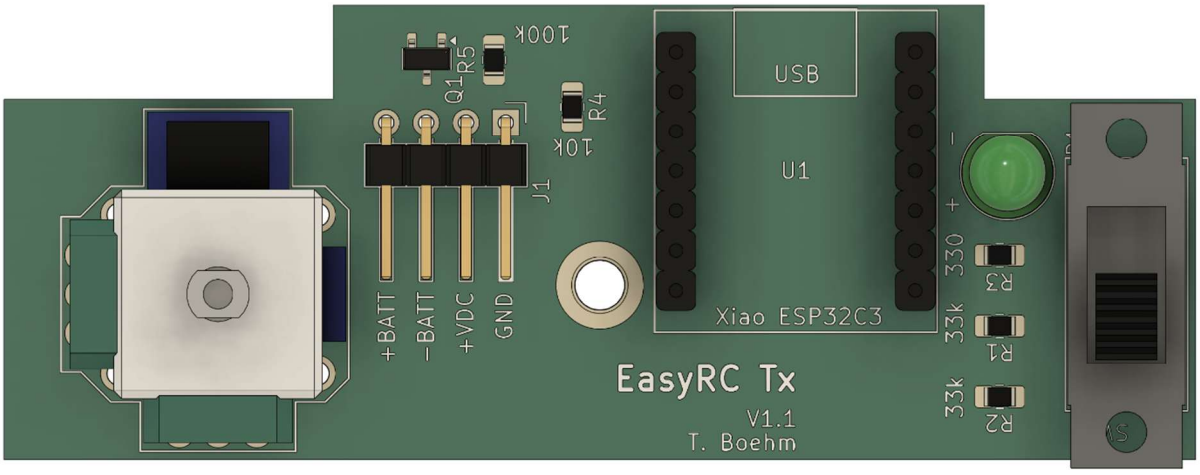
+BATT: battery anode (1S LiIon)
+VDC: battery anode after on/off switch
+3V3: regulated 3.3 V (DC-DC converter integrated in Xiao ESP32-C3)
-BATT: battery cathode
GND: battery cathode after reverse polarity protection

Explanations of the circuit snippets:

- Microcontroller: Encompasses the inputs and outputs of the Xiao ESP32-C3. The battery voltage is reduced via a voltage divider at pin D0, allowing for a safe measurement using the microcontroller's analog input. Pins D1 to D3 are connected to the joystick module. As only the pins D0 to D2 are capable of analog measurements, D1 and D2 are used for measuring the two axes of the joystick. D3 is used as a digital input for the push button. The LED at pin D7 is used with a series resistor to prevent burning the LED or damaging the microcontroller by drawing too much current (the current draw should not exceed 10 to 15 mA). The available functions of the pins of this microcontroller can be studied in detail here:
  https://wiki.seeedstudio.com/XIAO_ESP32C3_Getting_Started/#
- Connectors: Shows the different inputs and outputs of the circuit board. The miniature slide switch has two galvanically isolated switches. Only one is required to switch the power supply to the board on and off. The 4-pin connector is used to connect the board to the battery and to the power supply pads on the bottom side of the Xiao ESP32-C3. These power supply pads enable powering the microcontroller from a single Li-ion battery, as the development board includes a voltage divider that generates a stable 3.3 V from the battery voltage.
- Reverse polarity protection: Reverse polarity protection is enabled by an "inversely "connected n-channel MOSFET between the cathode of the battery and the circuit board. This layout results in the board being powered from the battery through the transistor's body diode when the battery is correctly connected. As soon as this happens, a positive voltage is also applied to the transistor's gate, meaning it is turned on, and the voltage loss through the body diode diminishes. If the battery is connected incorrectly, the body diode of the transistor blocks current, and a negative voltage appears at the transistor's gate (+VDC equals the negative battery voltage), preventing any current from flowing between the source and drain. Hence, this transistor acts as an ideal diode.
- Joystick module: The two potentiometers of the joystick are supplied with 3.3 V from the ESP32-C3, allowing their analog outputs to generate a measurable voltage between the logic level (3.3 V) and common ground (0 V). The push button is connected to common ground and the microcontroller, which allows it to be read as 0 or 1 using an internal pull-up resistor (depending on whether the button is pressed or not).

3.5.2 Printed circuit board



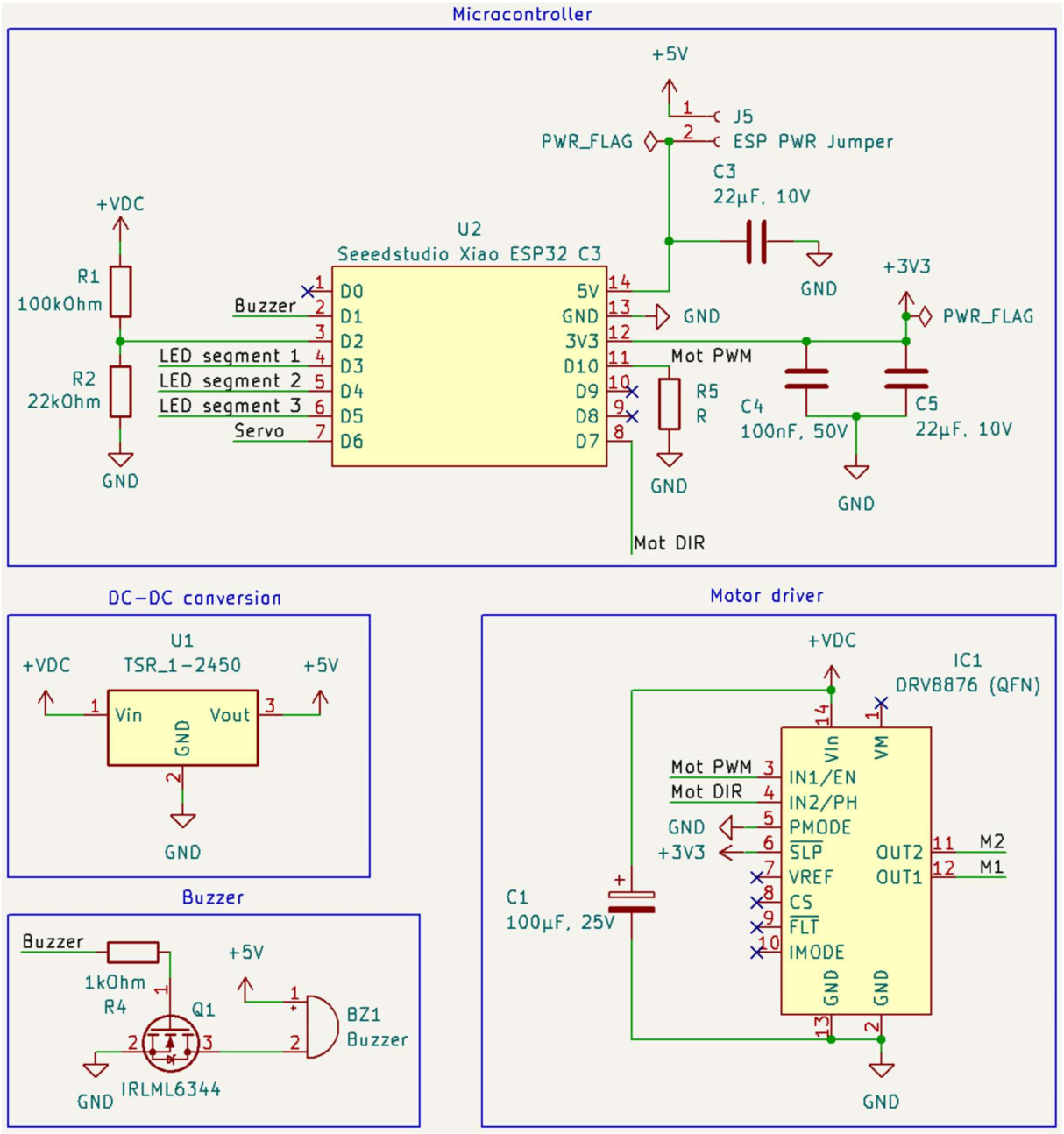


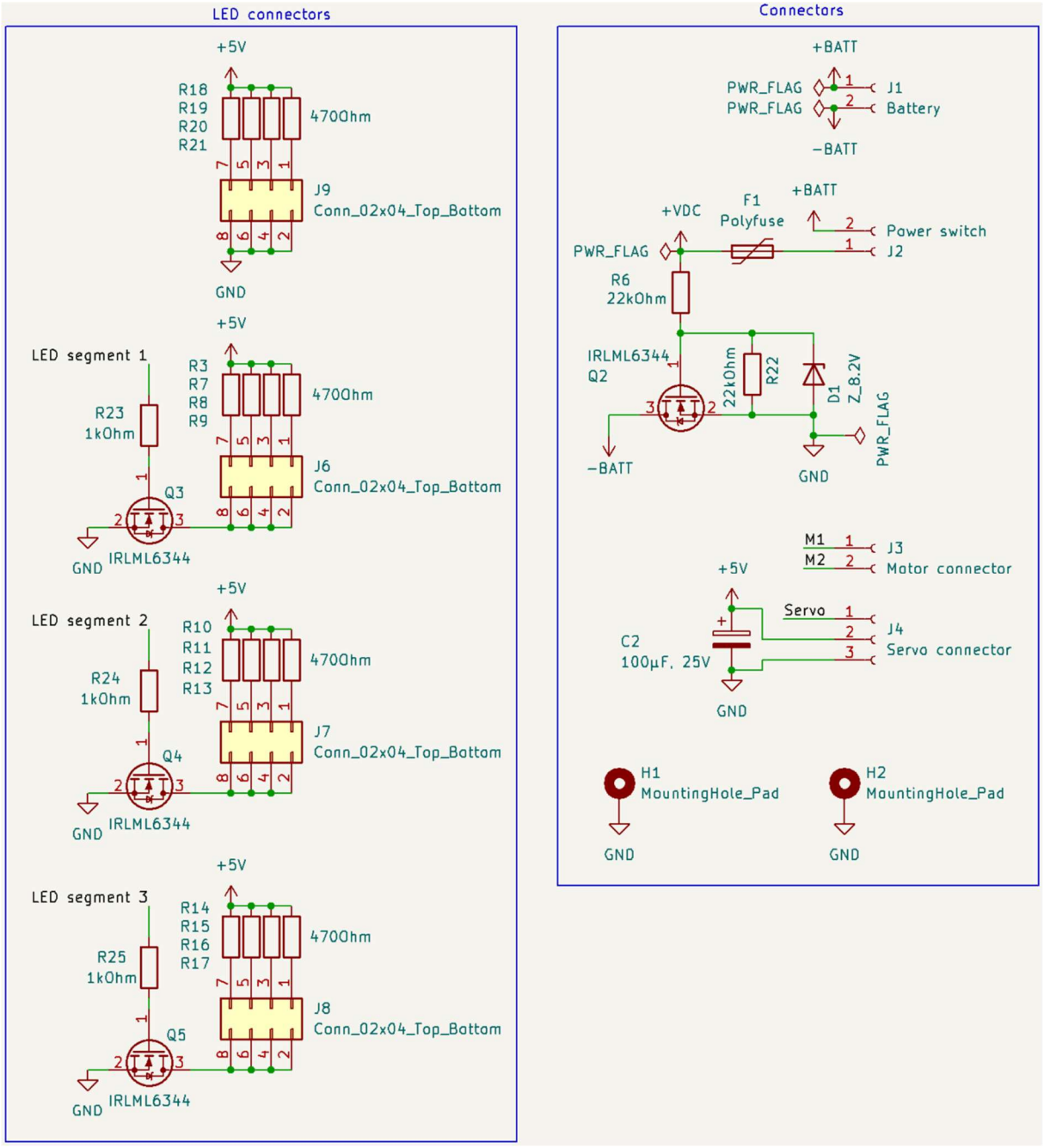
Designation	Component
D1	5 mm LED, green
J1	1x4 2.54 mm angled pin header
J2	Alps 10k joystic module
Q1	SOT-23 MOSFET IRLML 6344
R1	0805 resistor 33 kOhm
R2	0805 resistor 33 kOhm
R3	0805 resistor 330 Ohm
R4	0805 resistor 10 kOhm
R5	0805 resistor 100 kOhm
SW2	Wentronic S 166 6 pin miniature switch
U1	2x 1x7 2.54 mm socket strip (5 mm height)
Remark: The annotation of board version 1.1 still needs to be improved.	

A photograph of the PCB with the mounted microcontroller and connected battery holder is depicted in Chapter 4.1.1.

3.6 Overview of vehicle electronics

3.6.1 Circuit diagram





Net assignments:

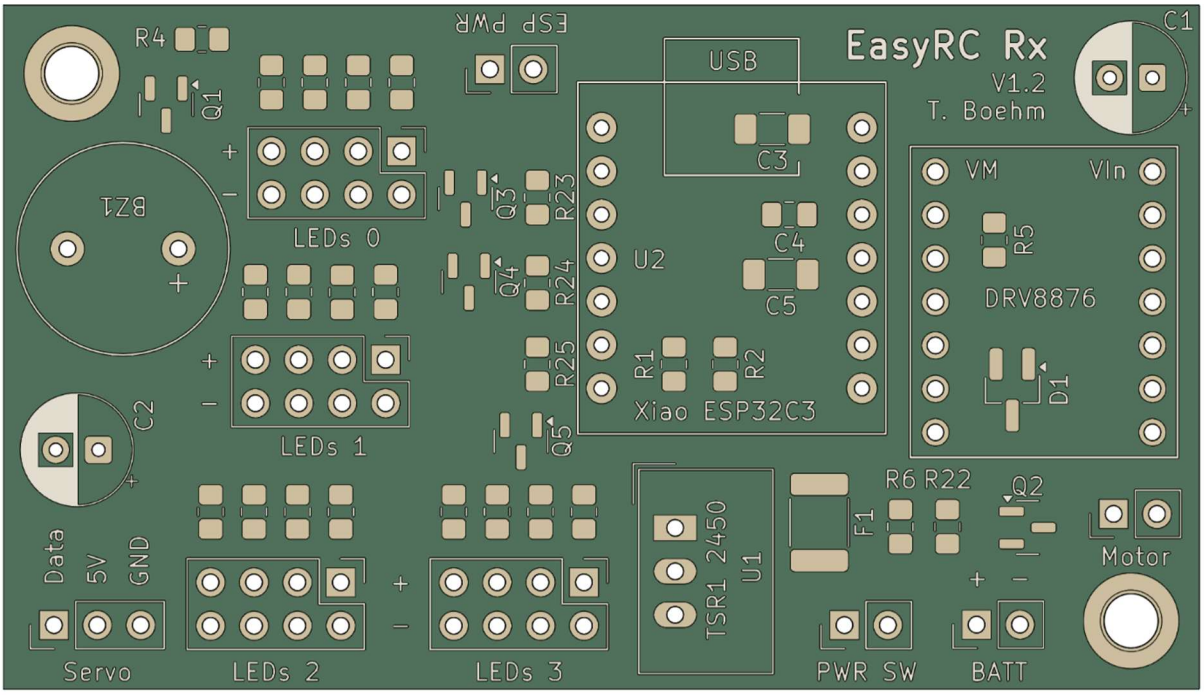
+BATT: battery anode (3S LiIon)
+VDC: battery anode after on/off switch
+5V: regulated 5 V (DC-DC converter)
+3V3: regulated 3.3 V (DC-DC converter integrated in Xiao ESP32-C3)
-BATT: battery cathode
GND: battery cathode after reverse polarity protection

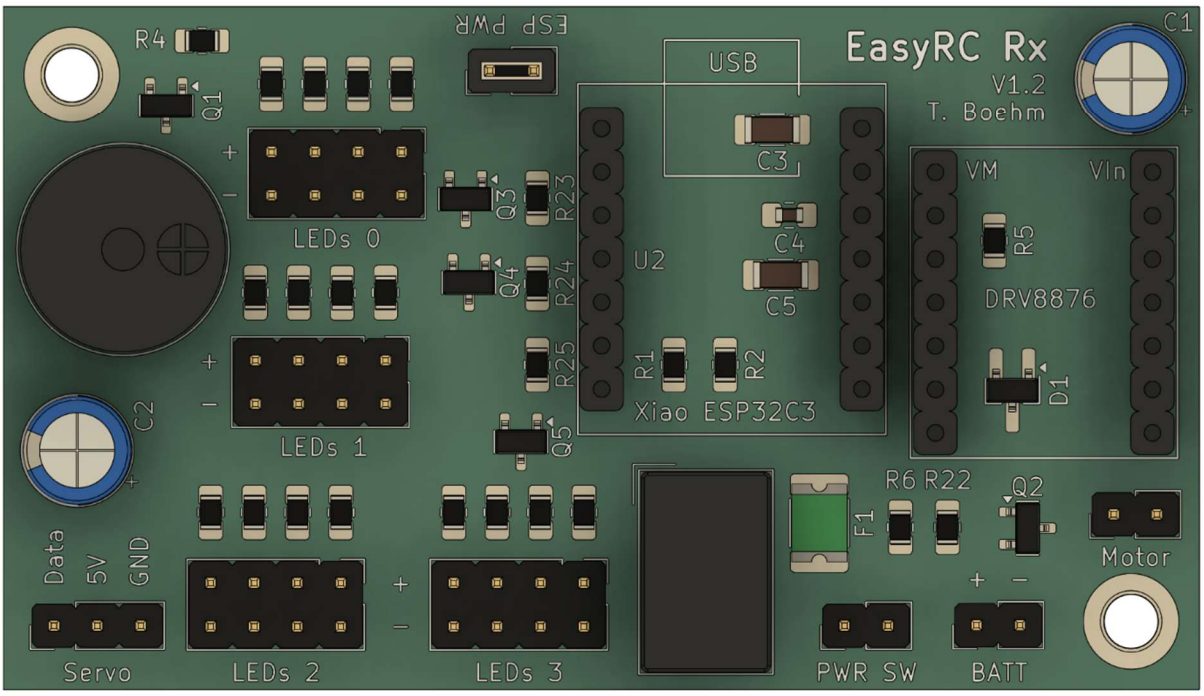
Explanations of the circuit snippets:

- Microcontroller: Encompasses the inputs and outputs of the Xiao ESP32-C3. The battery voltage is reduced by a voltage divider at pin D2 to allow a safe measurement using the microcontroller's analog input. The pins D1, D3 to D7, and D10 are used as outputs. D1, D6, and D10 are used as PWM outputs, while the other pins are used as simple digital outputs. Pin D10 (PWM control of the drive motor) is connected to a resistor R5, which is an optional pulldown resistor. Uncontrolled motor movements during the startup of the microcontroller (while pin D10 is still floating) can be prevented by adding a resistor with a resistance between 10 kOhm and 100 kOhm here. The 5 V input of the microcontroller is interrupted by a 2-pin header, which allows interrupting the connection between the 5 V net of the board and the microcontroller upon removing a jumper. This action prevents current from flowing to the board when the USB port of the microcontroller is in use, as the board could overload the USB output of a computer. Capacitors are connected near the microcontroller's 5 V input and 3.3 V output to minimize voltage fluctuations and ensure stable operation.
- DC-DC conversion: Depicts the step-down converter that generates a stable 5 V from the 3S Li-ion battery to power the microcontroller, steering servo, LEDs, and piezo buzzer.
- Motor driver: The Pololu motor driver breakout board receives data from the microcontroller via two pins (speed and rotation direction). The other pins of the driver are directly connected to logic voltage, common ground, or are unconnected. A capacitor is added near the motor driver to maintain a stable voltage during rapid load changes of the drive motor. The motor is connected to the output pins M1 and M2 of the driver. Information on the pinout and control of this motor driver can be found here: https://www.pololu.com/product/4037
- Buzzer: The piezo buzzer generates noises with specific frequencies when an alternating voltage is applied. To this end, a PWM output of the microcontroller is employed, which means the voltage at the buzzer alternates between 0 V and 5 V. The frequency is given by the PWM frequency of the microcontroller, which thereby defines the tone pitch of the piezo buzzer. The buzzer is specified for use with a 5 V power supply and requires more current than the microcontroller can provide. Therefore, a transistor is used as a switch, which is turned on and off by the PWM output of the microcontroller.
- LED connectors: The LED connectors on the board are divided into four groups, each with four connectors. LED segment 0 cannot be controlled by the microcontroller but receives power permanently. This segment is designed for the headlights. Segments 1 to 3 are controlled by the microcontroller using a transistor (similar to the piezo buzzer) and serve as indicator and brake lights. Every LED output includes a separate series resistor to limit the current flowing through each LED. A resistance of 470 Ohm is recommended, but this value can be adjusted to control the LED brightness (do not exceed the specified maximum current of the LED and consider the specified maximum heat dissipation of the series resistors).
- Connectors: Shows the connectors on the board as well as its reverse polarity protection. The positive battery connector is connected to the on/off switch, which then supplies power to the board. Here, a fuse is added that trips if the current exceeds a certain limit. Furthermore, the same reverse polarity protection concept used in the remote control's board is employed. Here, a voltage divider at the gate of the transistor is necessary, as it is specified for a maximum gate voltage of 12 V, whereas the board is supplied with up to 12.6 V when the batteries are fully charged. The voltage divider cuts this voltage in half to

remain in a safe range. Furthermore, a Zener diode is added as an additional safety measure to protect the transistor in case of inductive voltage peaks. A capacitor is connected near the steering servo connector to minimize voltage drops caused by load changes in the steering motor.

3.6.2 Printed circuit board





Designation	Component
BATT	1x2 2.54 mm pin header
BZ1	Piezo buzzer (12 mm, 6.5 mm pitch)
C1	Electrolytic capacitor, 6.3 mm, 2.5 mm pitch, 100 µF, 25 V
C2	Electrolytic capacitor, 6.3 mm, 2.5 mm pitch, 100 µF, 25 V
C3	1206 ceramic capacitor, 22 µF, 10 V
C4	0805 ceramic capacitor, 100 nF, 50 V
C5	1206 ceramic capacitor, 22 µF, 10 V
D1	SOT-23 Zener diode 8.2 V
DRV8876	2x 1x7 2.54 mm socket strip (8.5 mm height)
ESP PWR	1x2 2.54 mm pin header with short circuit bridge
F1	1812 PTC-Sicherung 1.5 A
LEDs 0	2x4 2.54 mm pin header with 4x 0805 resistor 470 Ohm
LEDs 1	2x4 2.54 mm pin header with 4x 0805 resistor 470 Ohm
LEDs 2	2x4 2.54 mm pin header with 4x 0805 resistor 470 Ohm
LEDs 3	2x4 2.54 mm pin header with 4x 0805 resistor 470 Ohm
Motor	1x2 2.54 mm pin header
PWR SW	1x2 2.54 mm pin header
Q1	SOT-23 MOSFET IRLML 6344
Q2	SOT-23 MOSFET IRLML 6344
Q3	SOT-23 MOSFET IRLML 6344
Q4	SOT-23 MOSFET IRLML 6344
Q5	SOT-23 MOSFET IRLML 6344
R1	0805 resistor 100 kOhm
R2	0805 resistor 22 kOhm
R4	0805 resistor 1 kOhm
R5	(optional) 0805 resistor 10-100 kOhm
R6	0805 resistor 22 kOhm
R22	0805 resistor 22 kOhm
R23	0805 resistor 1 kOhm
R24	0805 resistor 1 kOhm
R25	0805 resistor 1 kOhm
Servo	1x3 2.54 mm pin header
U1	DC-DC converter Traco TSR1-2450E
U2	2x 1x7 2.54 mm socket strip (8.5 mm height)
Remark: The annotation of board version 1.2 still needs to be improved. Resistors R3 and R7 to R21 (LED series resistors) are not labelled on the board due to space constraints.	

The overview of the connectors is provided in Chapter 4.4.4. The correct orientation of the Xiao ESP32-C3 and the motor driver has to be ensured when adding them to the board. For the microcontroller, the orientation of the USB port is indicated on the PCB. For the motor driver, the positions of the two pins, VM and VIn, are labeled on the board.

4. Assembly guide for the remote control and vehicle

Assembling the vehicle and the remote control requires the components from the part lists. As tools, Allen keys with 2 mm and 2.5 mm with a ball head are necessary. Further, small flat pliers and tweezers are needed.

PETG is recommended as a material for 3D printing the components because it is easy to print, sufficiently mechanically sturdy, and UV-resistant. Alternatively, PLA or another common filament can be used that offers mechanical stability and stiffness. Some parts have to be printed from flexible TPU. These parts are indicated in Chapter 2.3 and carry "TPU" in their name. A print bed with a length of at least 220 mm is required to print all parts. Recommendations for orienting the parts on the print bed and for support are provided in the hints in the printed parts list. A 0.4 mm nozzle and a layer height of 0.2 mm are recommended for all parts. An infill rate of 15 % is sufficient.

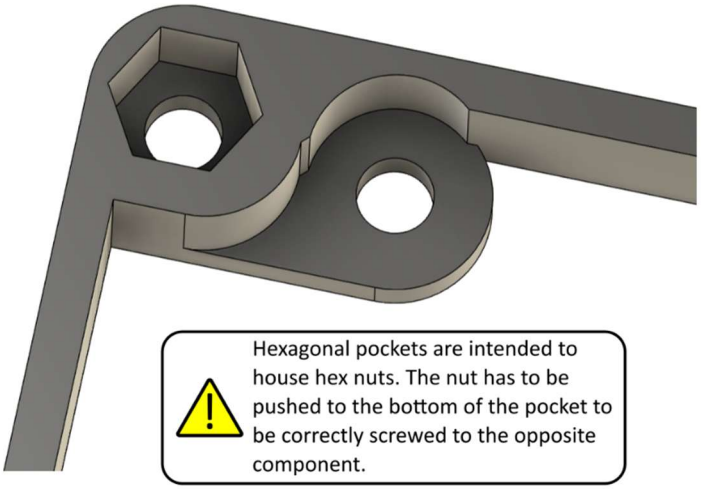
It should be noted that the printed parts do not offer perfect tolerances. This limitation is due to the type of manufacturing: 3D printing creates imperfect tolerances along specific directions (for example, a layer height of 0.2 mm results in steps of this interval along the Z-direction, meaning edges that are not horizontal or vertical can only be approximated). Further, 3D printing requires support structures for certain geometries to stabilize the part during printing (printing in the air is impossible, but contact with the build plate is required to fabricate structures stably). These support structures partially adhere to the parts, resulting in a poor surface finish. Therefore, printed parts must be fully cleaned of support remnants to ensure a correct fit.

Sometimes, also artifacts occur during 3D printing, like stringing (thin plastic fibers remain between or within parts) or geometric distortions ("elephant foot" or warping: the part warps directly above the print bed due to changes in temperature or air movement during printing). EasyRC components were developed with these issues in mind to minimize these effects as much as possible or to reduce their influence on the function as much as possible. Nonetheless, it may happen that a part does not fit perfectly into its position, rotating elements may rub against their housing, or pockets for screws and nuts may be too tight to allow components to be secured easily.

Therefore, it is essential to be aware of these potential errors during the assembly process. Before securing components with screws, check if a screw can be easily pushed into its recess without significant force. If that is not the case, the hole should be processed with a longer screw by pushing it up and down to remove excess filament, thereby using the screw's threading as a file. Rotating elements should always be tested for ease of movement prior to securing them in place. If a grinding noise occurs, check for plastic strings or imperfections in the surface finish that could cause the friction. Remove excess material carefully with a slotted screwdriver or sandpaper. The advantage of 3D printed parts is their low cost and fast manufacturing (for small batch manufacturing). If a component is defective, it can be reprinted within a few hours, allowing for a quick replacement of broken parts.

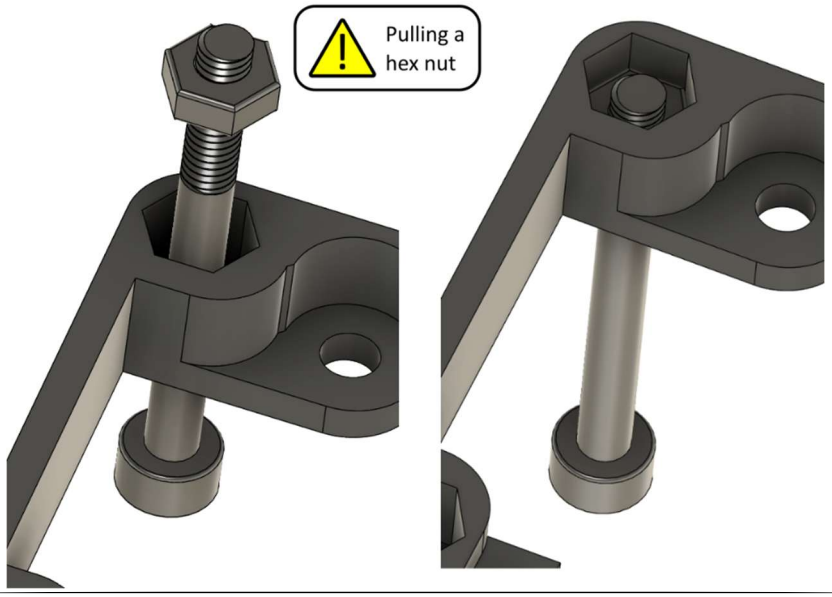
EasyRC components are assembled using metric screws and hex nuts. Hex nuts are used as thread inserts because delicate printed threads are unstable and cannot be reused multiple times before they are ground down or pulled out of the part. Alternatively, parts could be fused with cyanoacrylate adhesive (superglue), but this process does not allow them to be disassembled without destroying them. However, superglue or hot glue is a fast means to repair broken components.

Securing components with screws requires placing the hex nuts in their pockets correctly. Placing them by hand is not always possible, depending on the position of the nut within a part and also on the tolerances of the parts. To this end, the nut can be screwed onto a screw and then placed in its recess, using the screw to push it into position at the correct angle. Alternatively, the nut can be placed loosely above or within its pocket, and a screw can be approached from the opposing side to pull the nut into its recess. These two processes are depicted in the following figure.



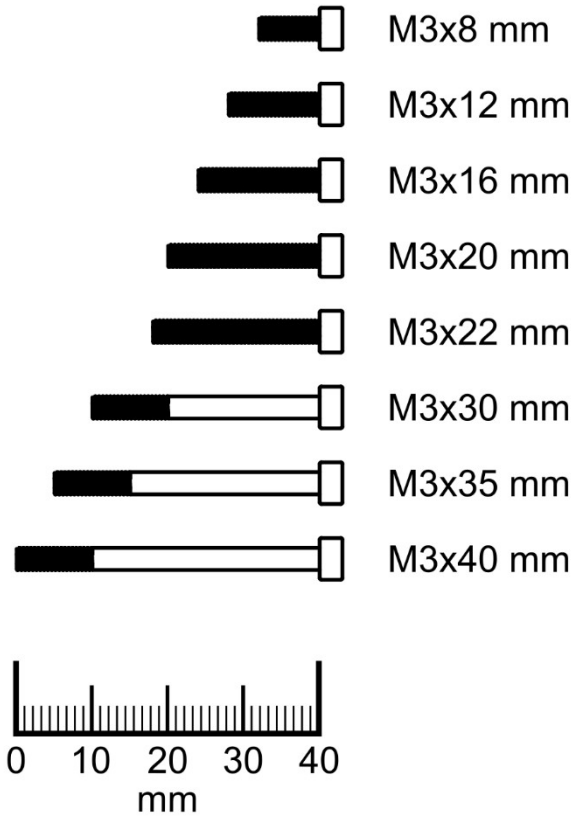
Hexagonal pockets are intended to house hex nuts. The nut has to be pushed to the bottom of the pocket to be correctly screwed to the opposite component.

Pushing a hex nut



Pulling a hex nut

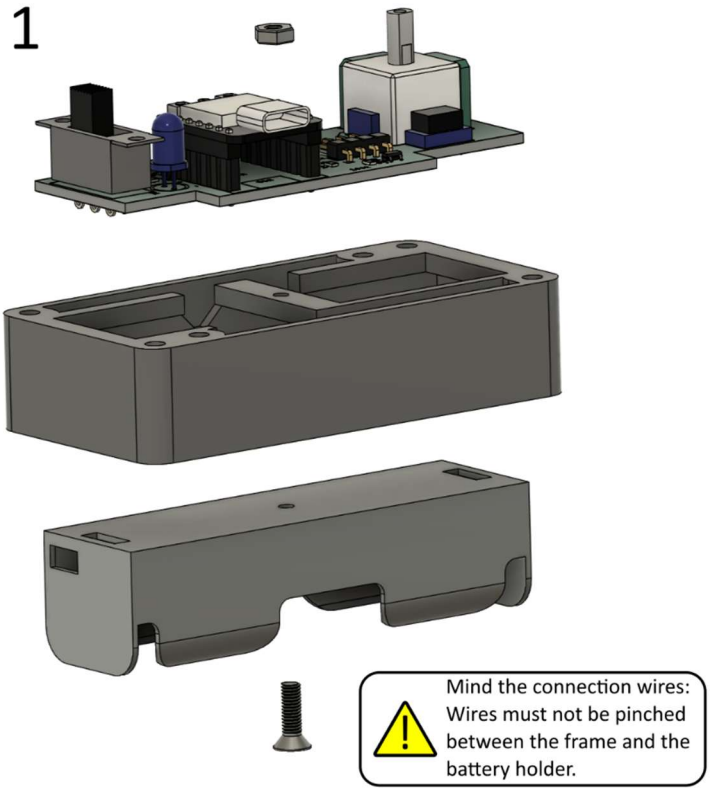
Here is a guide to help you easily find the correct screw lengths. If this page is printed precisely on A4 paper, the screw lengths match exactly those shown in the size chart.

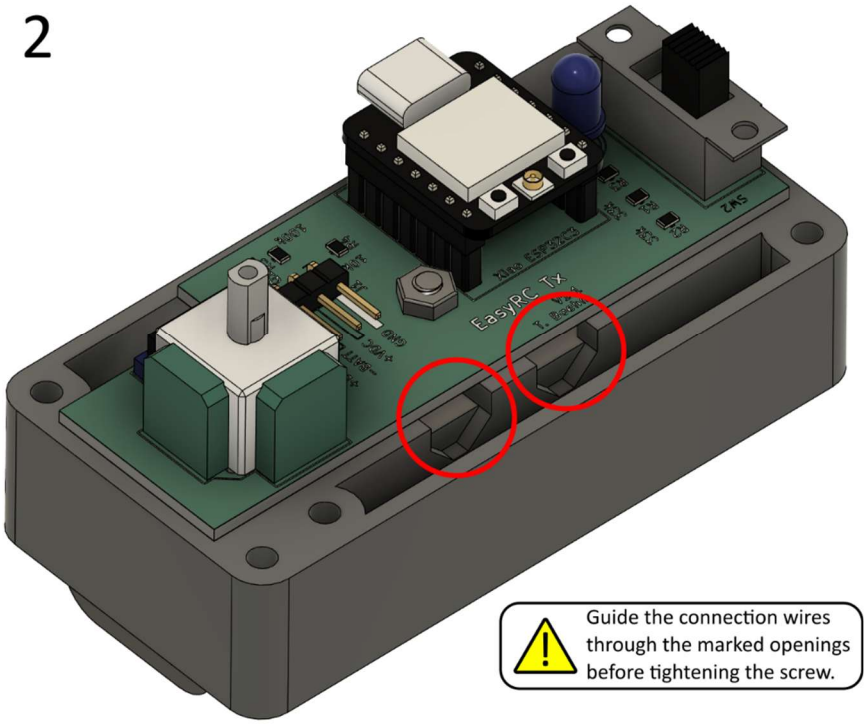


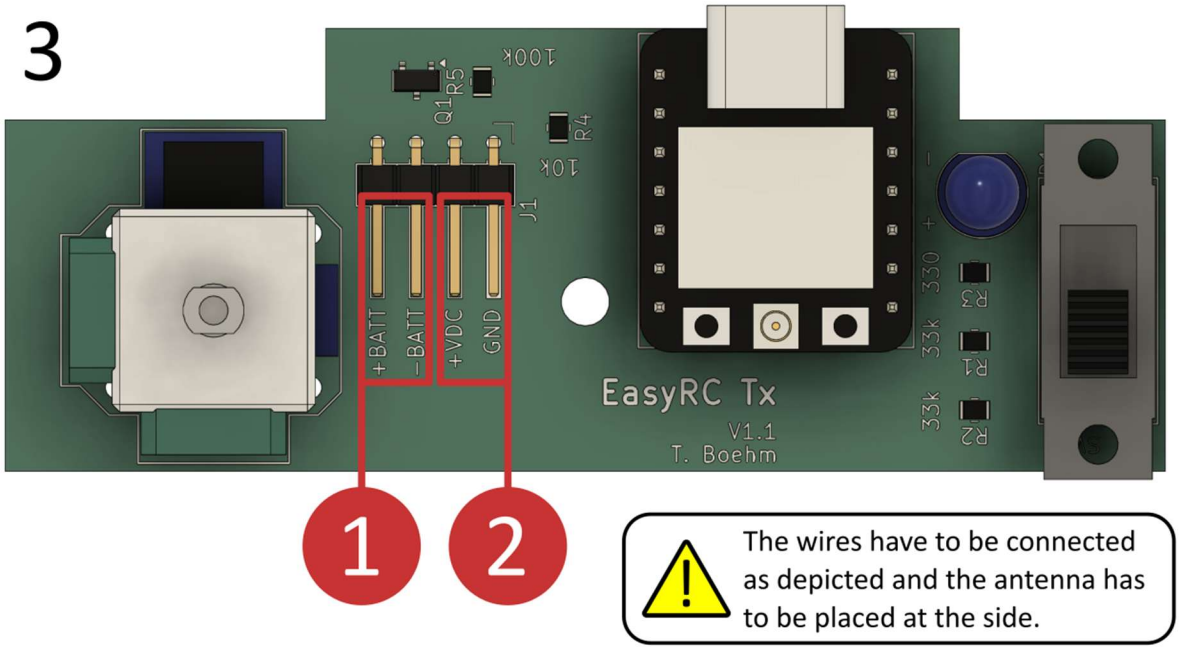
4.1 Remote control

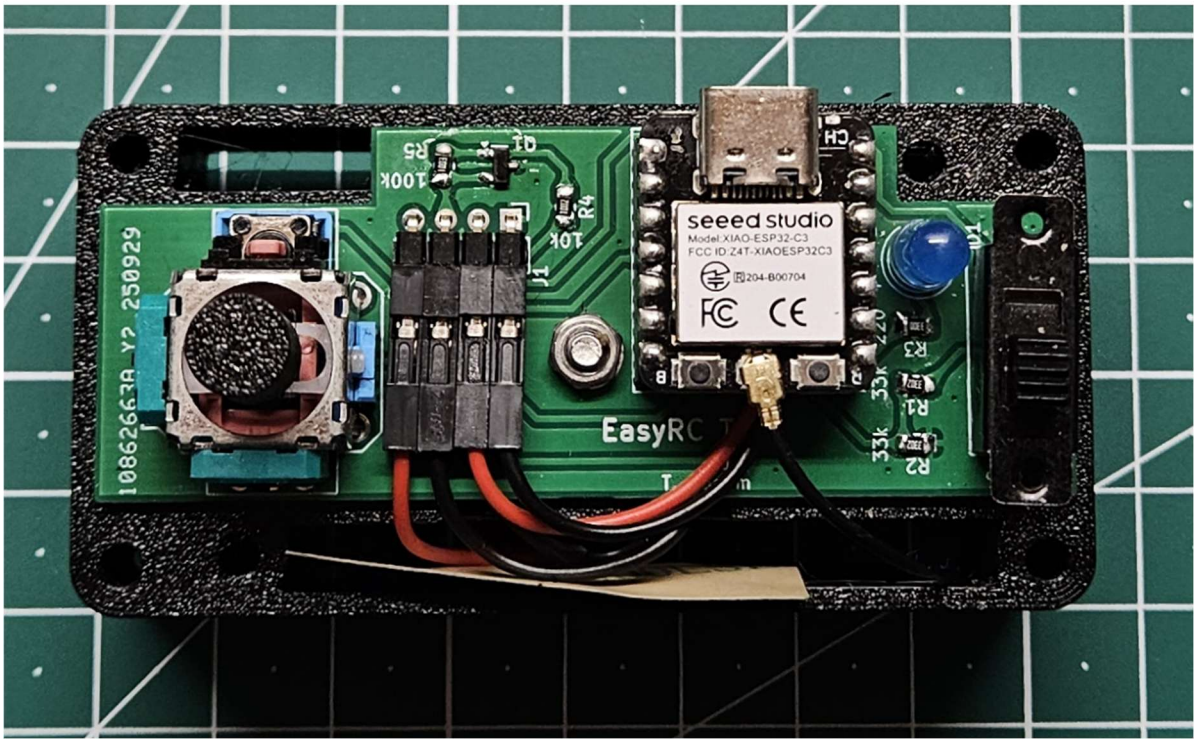
4.1.1 Installation of PCB and battery holder

Required parts: Remote control lower intermediate frame, EasyRC Tx circuit board, battery holder 1x 18650 LiIon cell, M3x10 mm countersunk screw, M3 hex nut







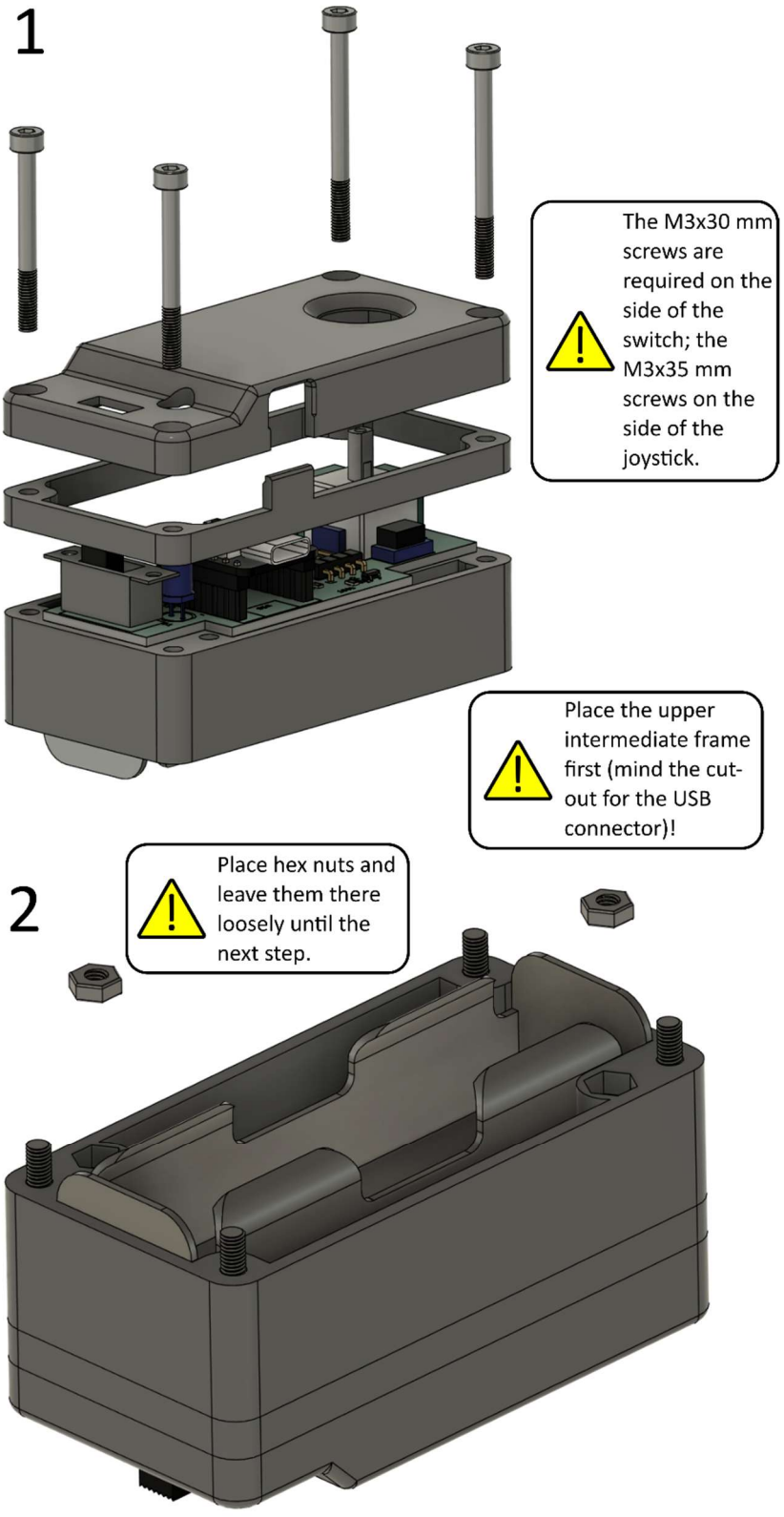


Connections: The battery holder must be connected to (1); the red wire needs to be connected to +BATT and the black wire to -BATT. The wires from the microcontroller must be connected to (2); the red wire needs to be plugged into +VDC, and the black wire into GND.

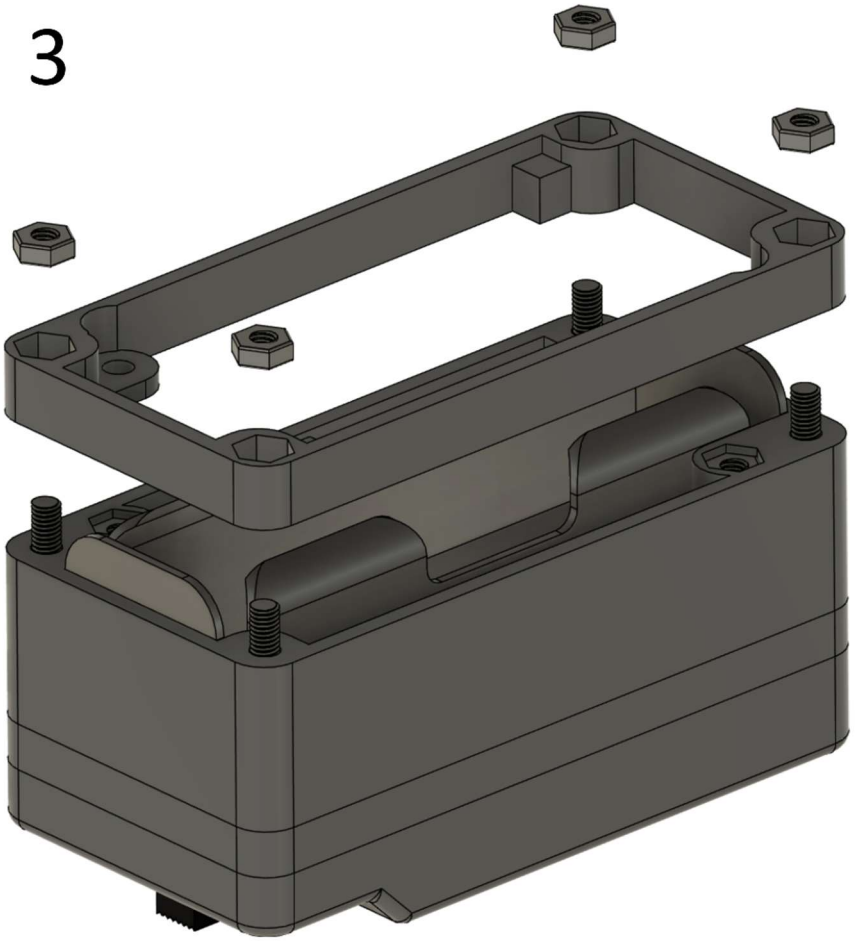
Attention: The battery connector (1) is reverse polarity protected; however, the microcontroller is not. If (2) is connected incorrectly, the microcontroller is instantly destroyed as soon as the remote control is switched on!

4.1.2 Fastening top and bottom

Required parts: Remote control (prepared), Remote control top, Remote control upper intermediate frame, Remote control bottom, M3x30 mm screw (2x), M3x35 mm screw (2x), M3 hex nut (6x)

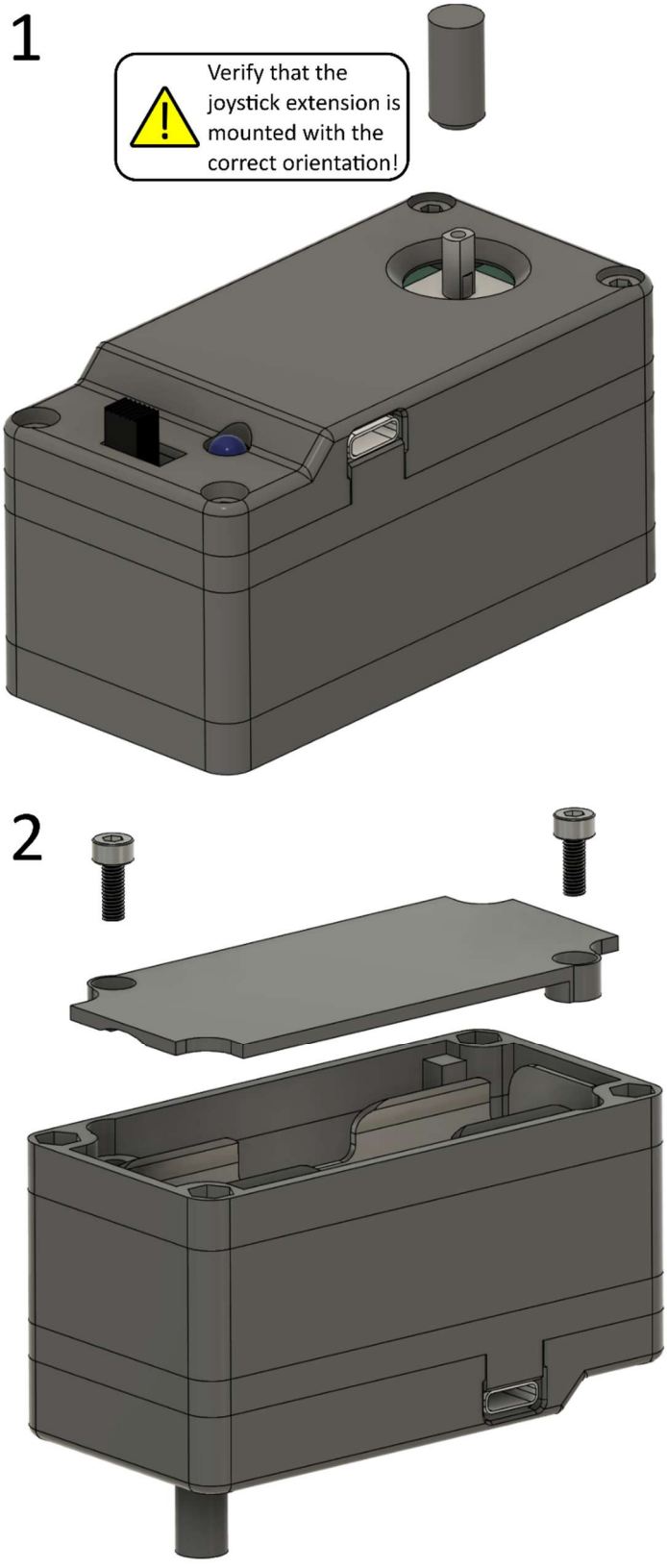


3

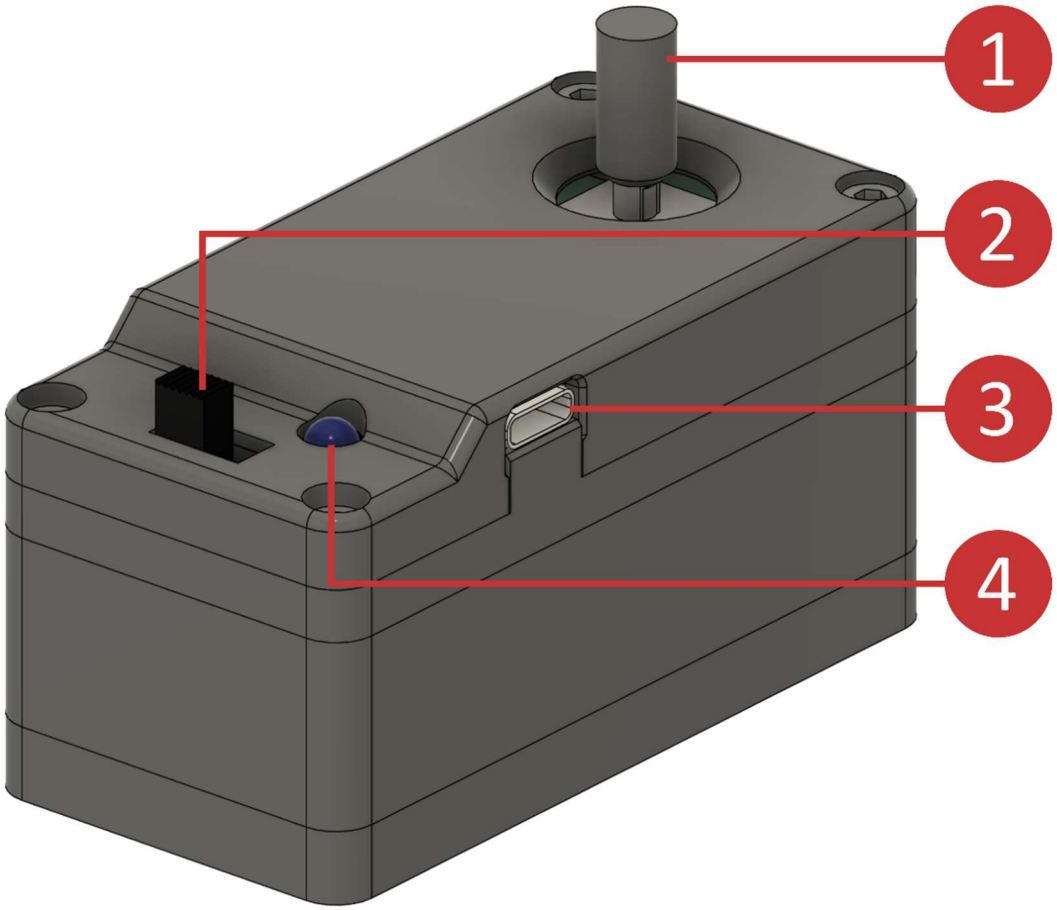


4.1.3 Completion remote control

Required parts: Remote control (prepared), Remote control joystick extension, Remote control battery cover, M3x8 mm Schraube (2x)



4.1.4 Overview remote control

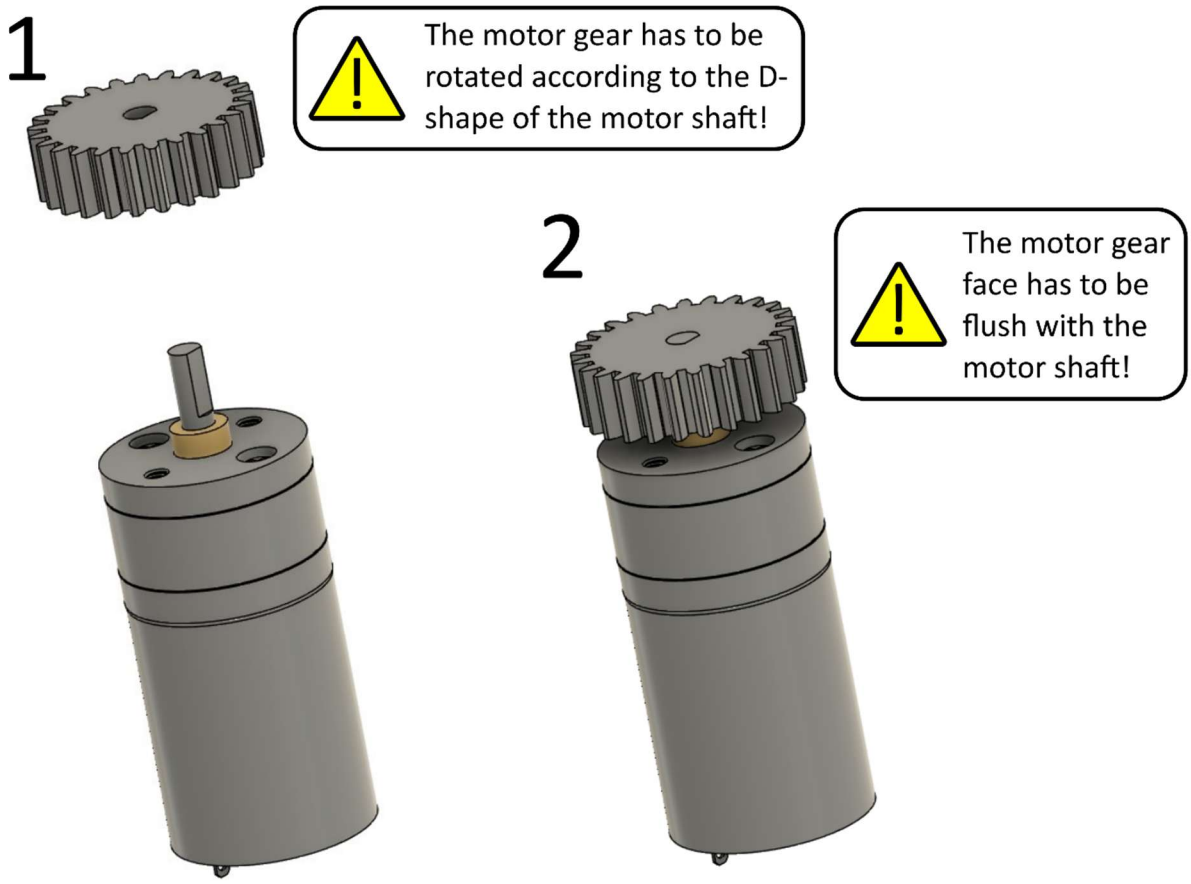


Number	Designation	Explanation
1	Joystick	Joystick used to control the vehicle (forward/reverse/left/right). Pushing the joystick activates the vehicle's horn.
2	On/off switch	Left position: off; right position: on
3	USB connector	USB connector type C
4	Status LED	The LED is permanently on while the remote control is switched on. It blinks twice per second if the battery is low.

4.2 Drive train and wheels

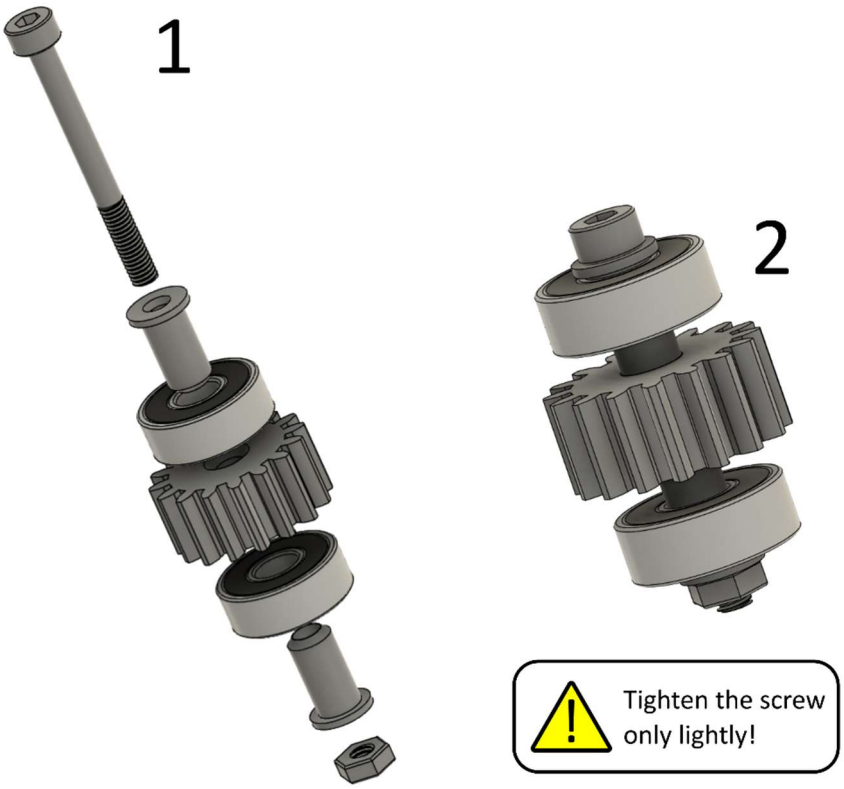
4.2.1 Motor preparation

Required parts: geared DC motor, 25 teeth gear



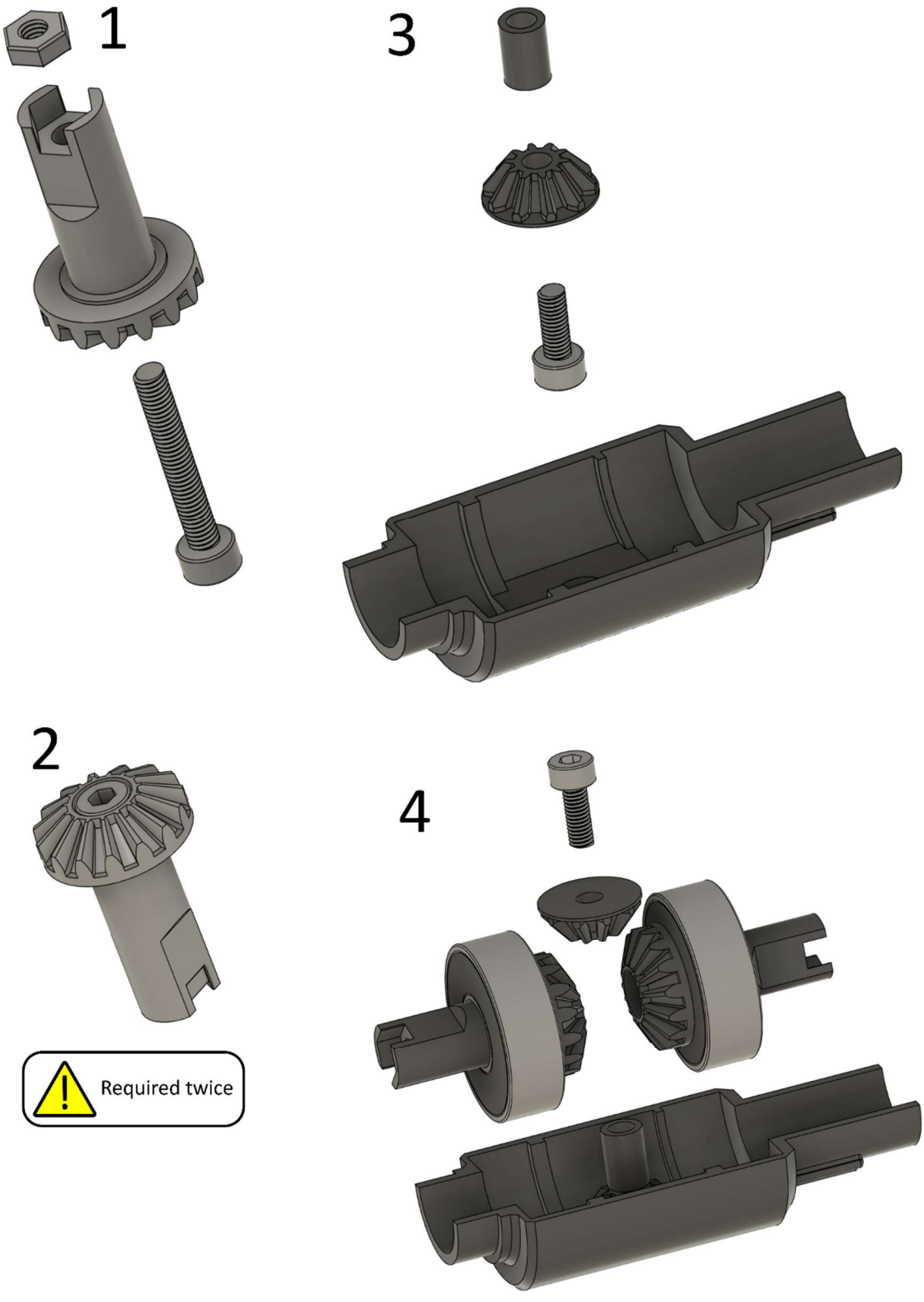
4.2.2 Intermediate gear

Required parts: 16 teeth gear, 16 teeth gear bearing mount (2x), ball bearing 605 (5x14x5 mm, 2x), M3x30 mm screw, M3 nex nut



4.2.3 Differential

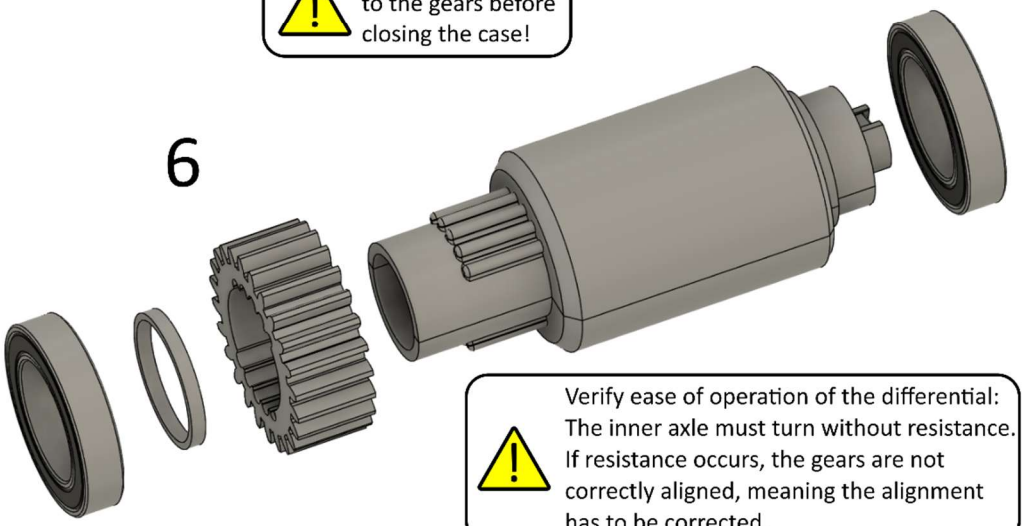
Required parts: Differential case (2x), 10 teeth crown gear (2x), differential spacer, 15 teeth crown gear (2x), 24 teeth gear, differential spacer ring, ball bearing 608 (8x22x7 mm, 2x), ball bearing 6802 (15x24x5 mm, 2x), M3x8 mm screw (2x), M3x20 mm screw (2x), M3 hex nut (2x)



5

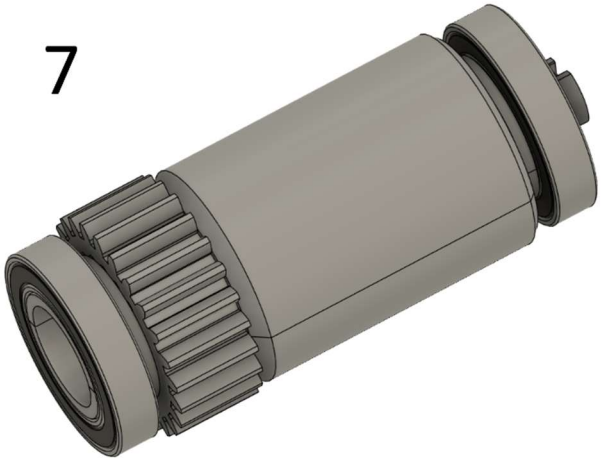
Apply some grease to the gears before closing the case!

6



Verify ease of operation of the differential: The inner axle must turn without resistance. If resistance occurs, the gears are not correctly aligned, meaning the alignment has to be corrected.

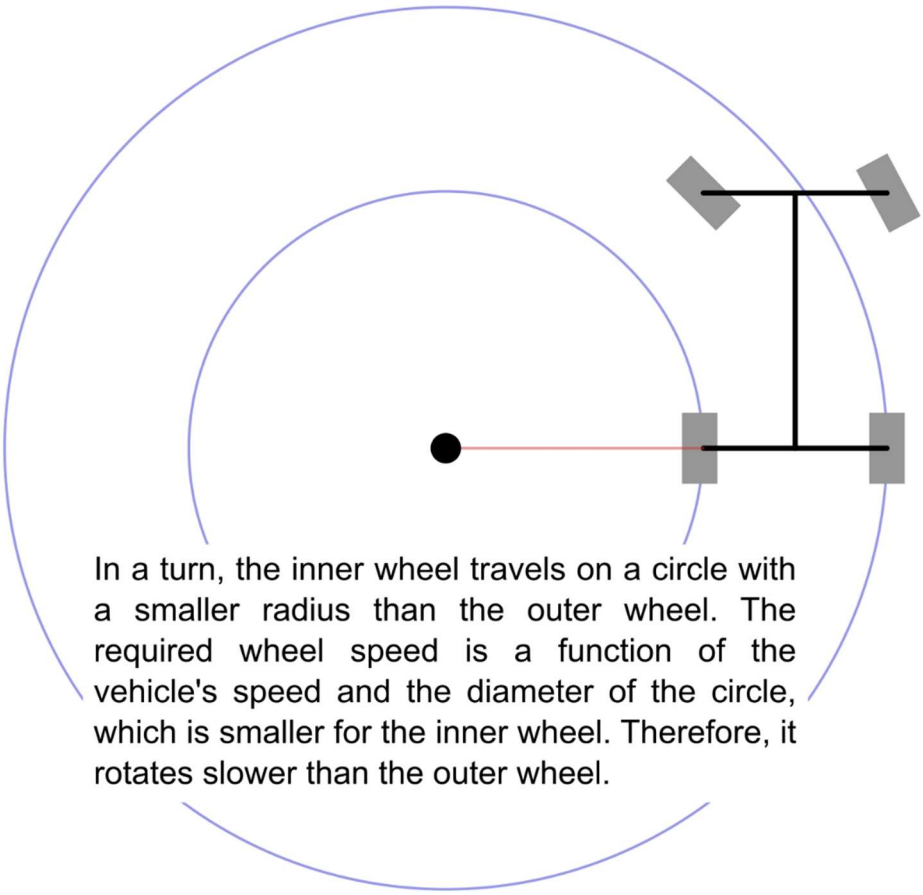
7



Did you know?

A differential is installed in every car. It is necessary to enable different wheel speeds at a driven axle, which is crucial for cornering. In a turn, the inner wheel must rotate more slowly than the outer wheel to maintain traction on both wheels. Vehicles with front-wheel drive or rear-wheel drive only need one differential between the left and right driven wheels. Cars with all-wheel drive even require three differentials (in the front and rear between the left and right wheels, as well as a center differential between the front and rear axles).

The disadvantage of a differential becomes apparent when one wheel loses traction. In this case, the wheel can spin freely while the other wheel remains stationary – the car is stuck. This issue does not apply to on-road cars, as this problem virtually never occurs. However, it is a common problem for off-road vehicles. Then, special differentials that are self-locking are employed, or manually operated differential locks are available. They turn off the differential function so that both wheels rotate at the same speed. Notably, differential locks must only be engaged in off-road use to prevent additional tire wear, poorer traction during cornering, and unnecessarily high strain on the power train when driving on-road.



In a turn, the inner wheel travels on a circle with a smaller radius than the outer wheel. The required wheel speed is a function of the vehicle's speed and the diameter of the circle, which is smaller for the inner wheel. Therefore, it rotates slower than the outer wheel.

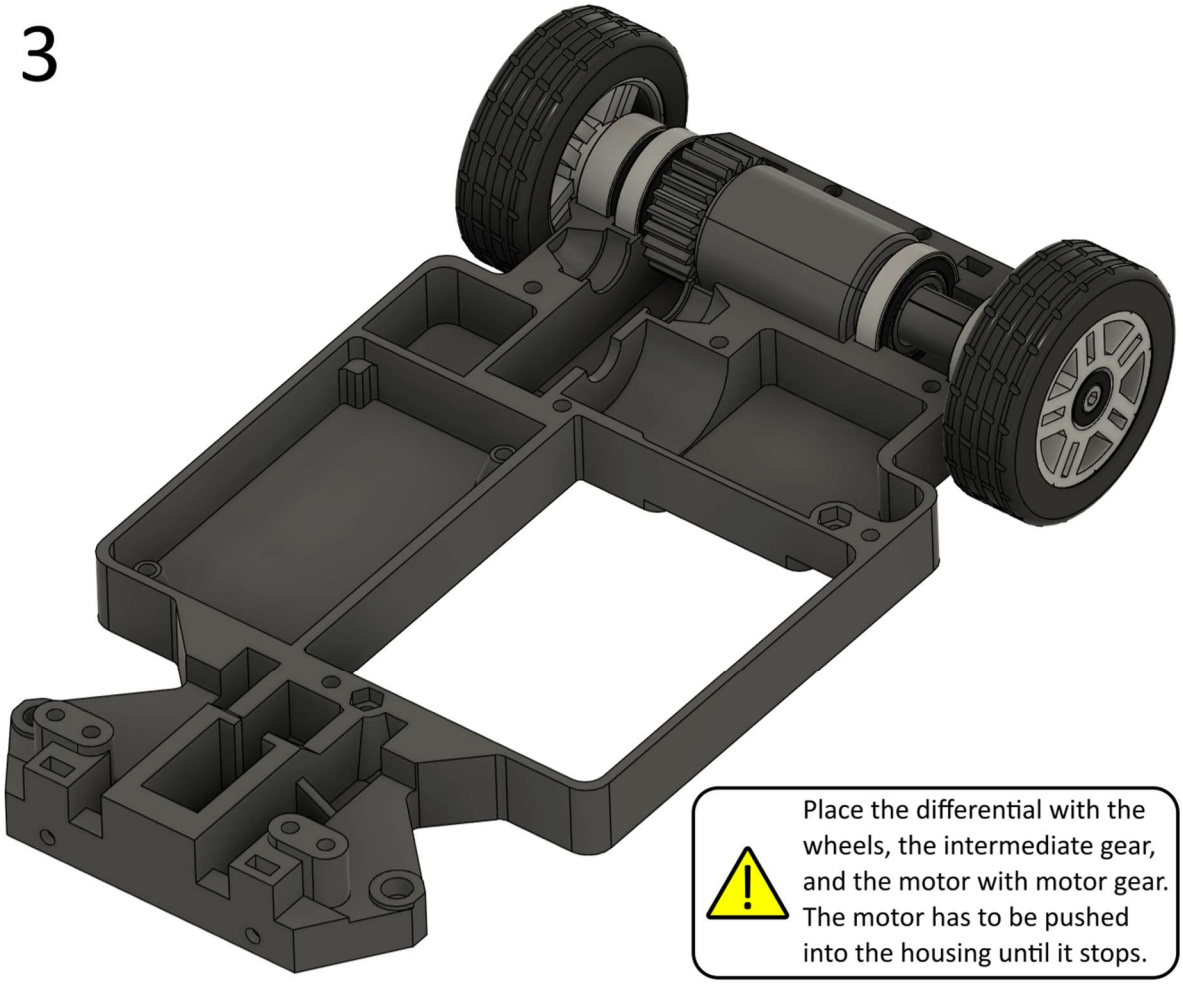
4.2.4 Rear wheels

Required parts: Rear rim (2x), Tire (2x), Rear rim spacer (2x), Left rear axle, Right rear axle, ball bearing 608 (8x22x7 mm, 2x), M3x30 mm screw (2x), M3 hex nut (2x)

4.2.5 Connection of the drive train and chassis

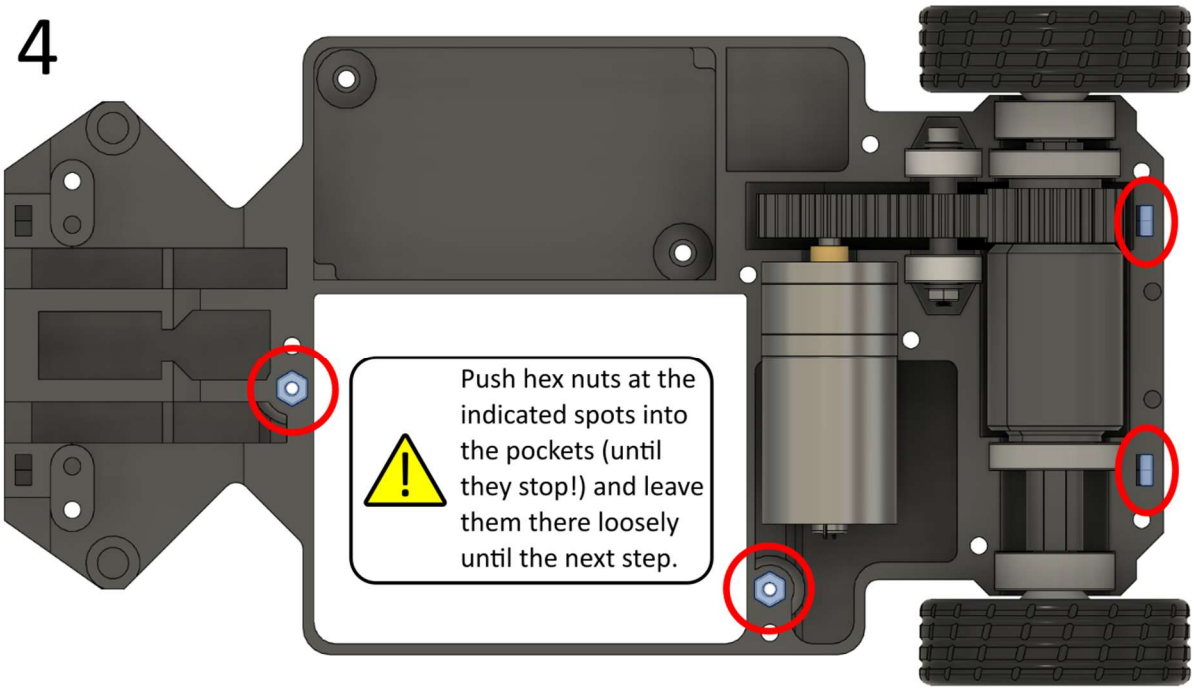
Required parts: Rear wheels, Differential, Intermediate gear, Motor with gear (all already prepared), Chassis bottom, Chassis top, M3x30 mm screw (8x), M3 hex nut (14x)

3



Place the differential with the wheels, the intermediate gear, and the motor with motor gear. The motor has to be pushed into the housing until it stops.

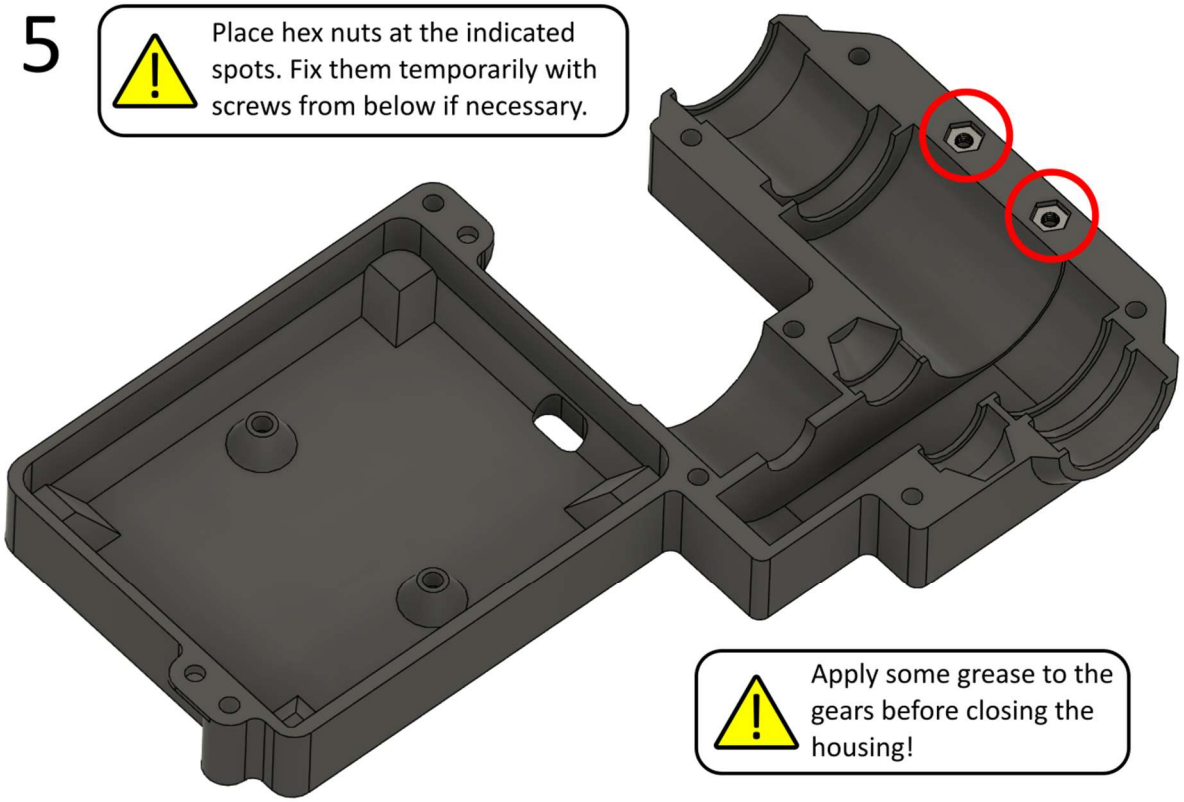
4



Push hex nuts at the indicated spots into the pockets (until they stop!) and leave them there loosely until the next step.

5

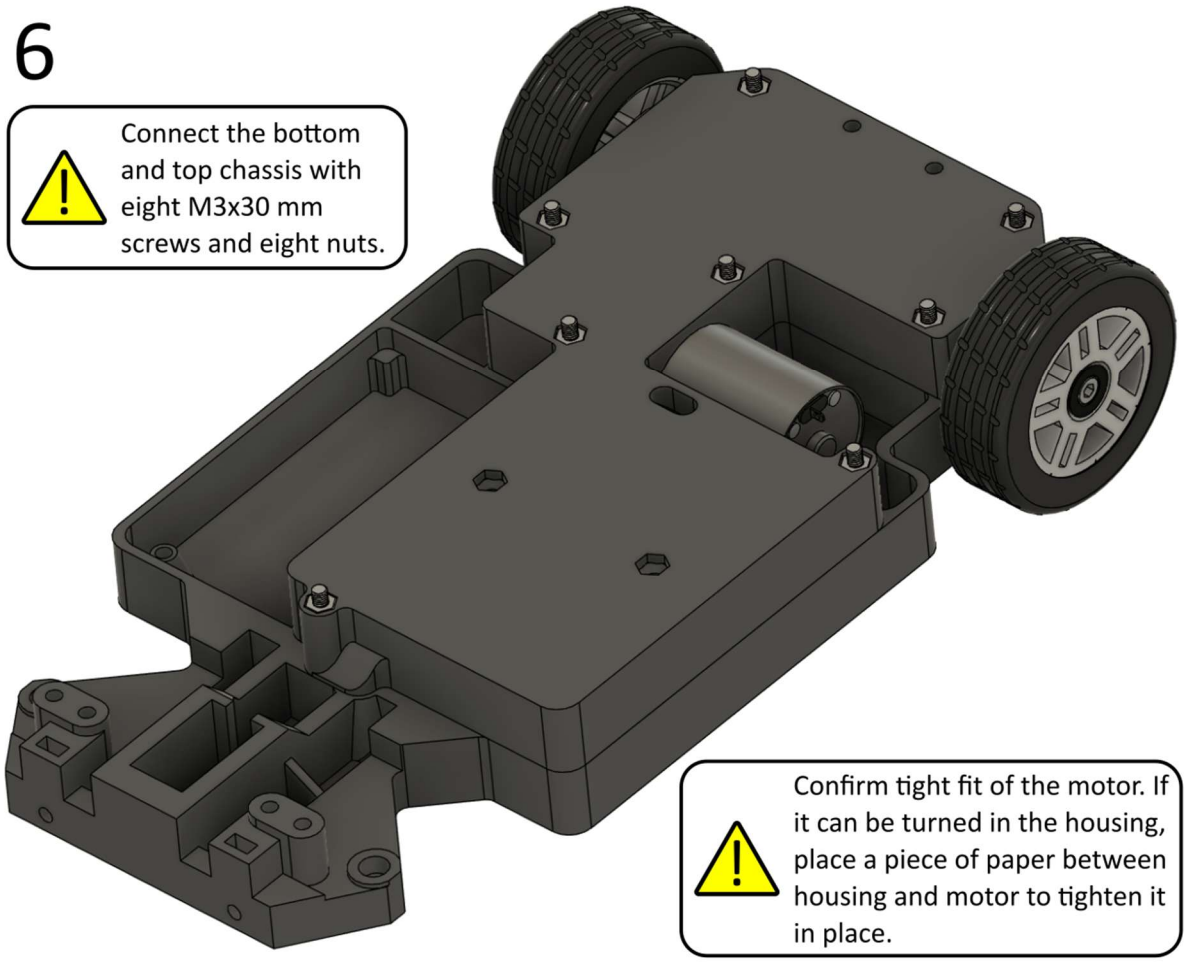
Place hex nuts at the indicated spots. Fix them temporarily with screws from below if necessary.



Apply some grease to the gears before closing the housing!

6

Connect the bottom and top chassis with eight M3x30 mm screws and eight nuts.



Confirm tight fit of the motor. If it can be turned in the housing, place a piece of paper between housing and motor to tighten it in place.

Did you know?

Gearboxes are ubiquitous: Not only cars, but also every power drill, mechanical clocks, and even salad spinners include gearboxes. A gearbox reduces or increases the speed of a mechanical drive. In return, the torque (force) becomes more or less because the power of a mechanical drive is the product of torque x speed. Ideal gearboxes have very small friction losses, meaning their efficiency is typically >90%. Hence, the output of a motor with a 1:10 transmission spins ten times slower but is almost ten times stronger. The transmission is a result of the ratio between the tooth count of the gear pair.

The EasyRC vehicle utilizes a motor with an integrated gearbox because the desired 1:10 transmission ratio can only be achieved with printed parts using a multistage gearbox, which requires a significant amount of space and increases the risk of failure. The printed gearbox in the vehicle (25:24 transmission, almost 1:1) is only used to create sufficient space for the differential between the motor and the driven axle. The tooth count of the intermediate gear (idler gear) does not play a role: The transmission ratio is only dependent on the input and output shaft.

4.3 Front wheels and steering

4.3.1 Preparation of front wheels

Required parts: Front rim (2x), Tire (2x), Front rim locking ring (2x), Front rim spacer (2x), Front wheel mount (2x, mirrored versions), Front wheel mount spacer (2x), ball bearing 605 (5x14x5 mm, 2x), M3x16 mm screw (2x), M3 hex nut (2x)

4.3.2 Steering linkage

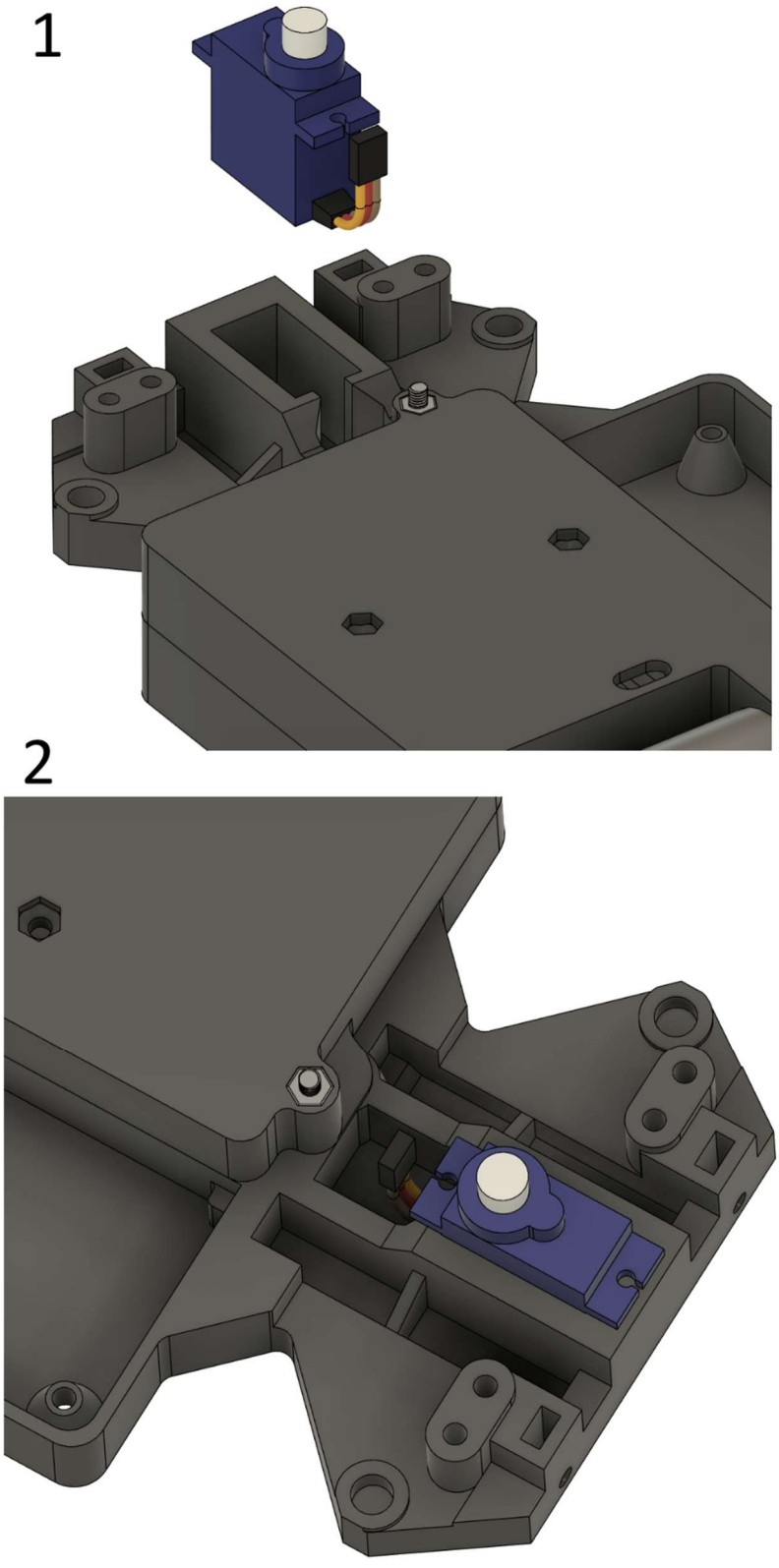
Required parts: Front wheels (prepared), Steering linkage, Steering linkage connector (2x), M3x16 mm screw (2x), M3x20 mm screw (4x), M3 hex nut (4x)

1

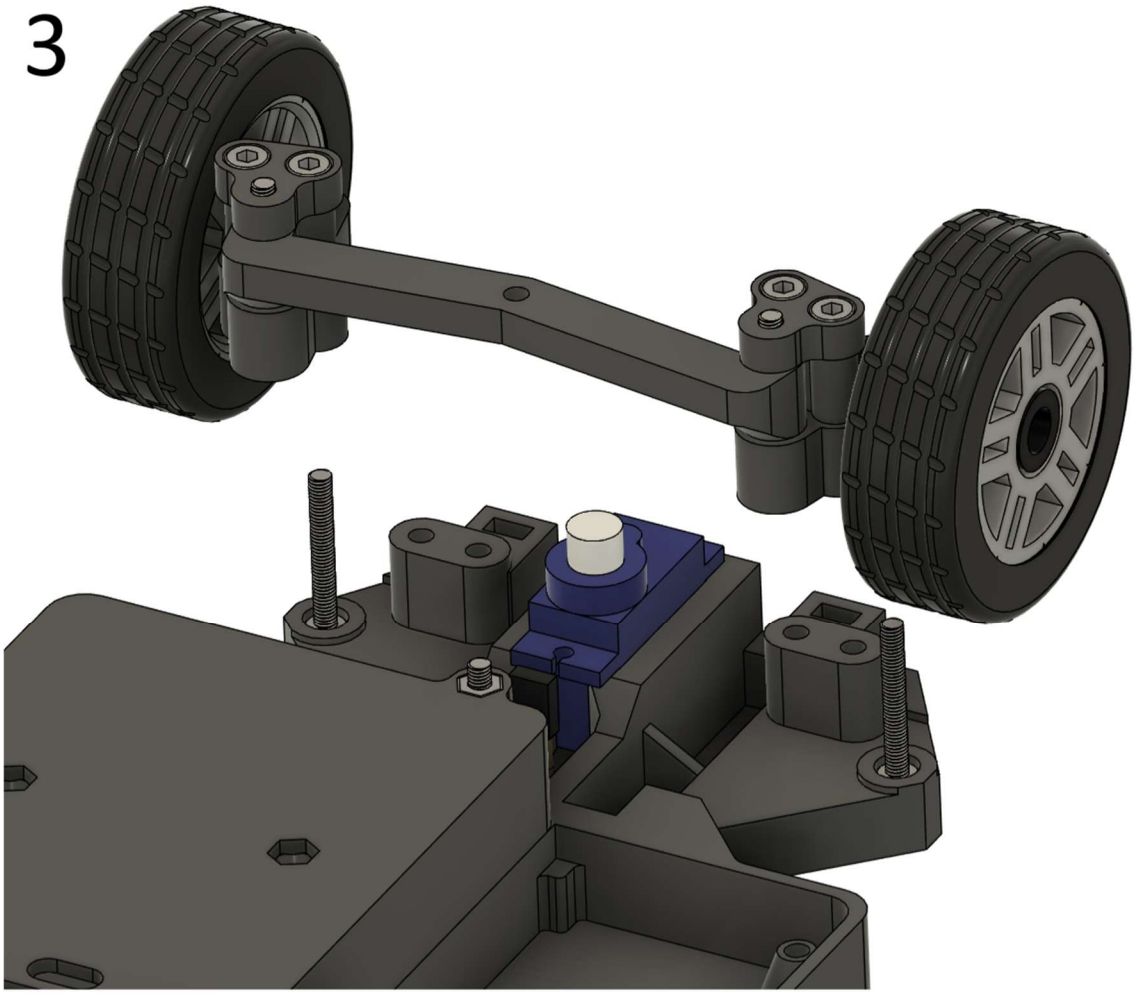
2

4.3.3 Connection of the front wheels and chassis

Required parts: Chassis (prepared), Front wheels with steering linkage (prepared), Chassis securing bracket, MG90S servo motor, M3x22 mm screw (2x), M3x30 mm screw (2x), M3 hex nut (4x)

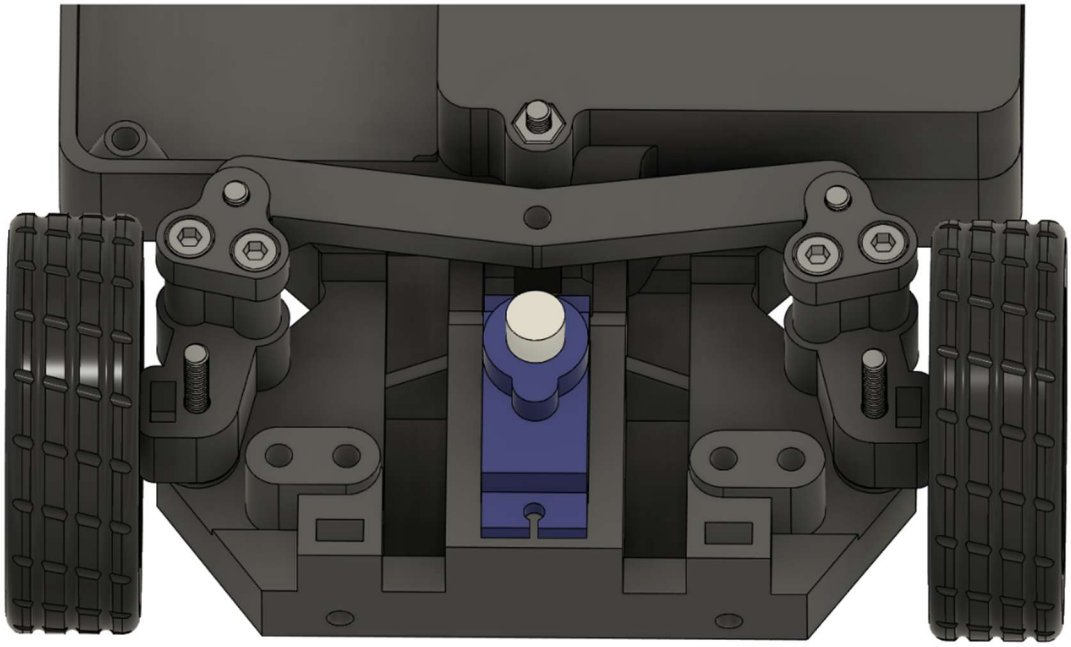


3



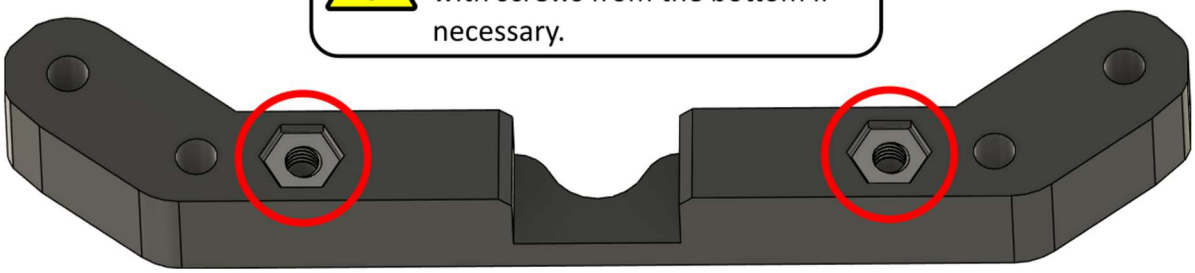
4

Place prepared front wheels with steering linkage onto two M3x22 screws. The components only stay loosely in place: Keep them in place until the next step.

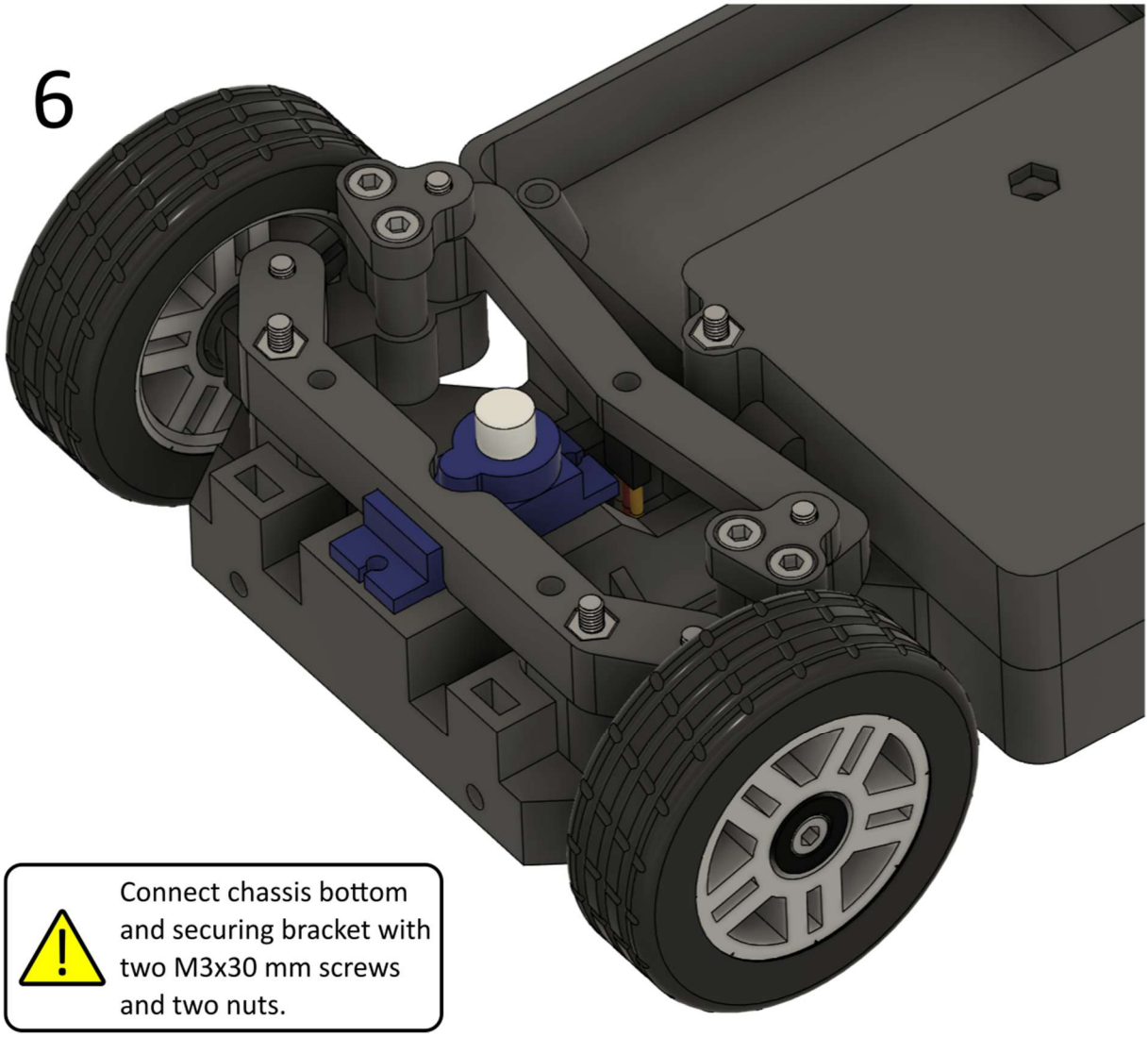


5

Place hex nuts in the indicated pockets. Secure them temporarily with screws from the bottom if necessary.



6



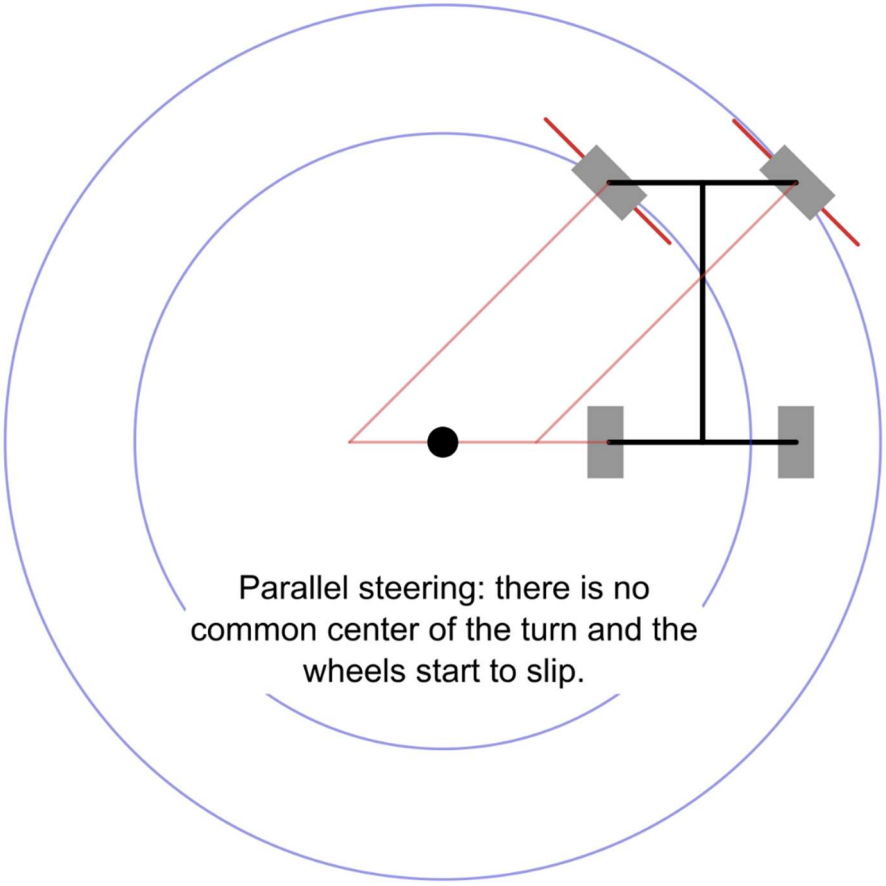
Connect chassis bottom and securing bracket with two M3x30 mm screws and two nuts.

Did you know?

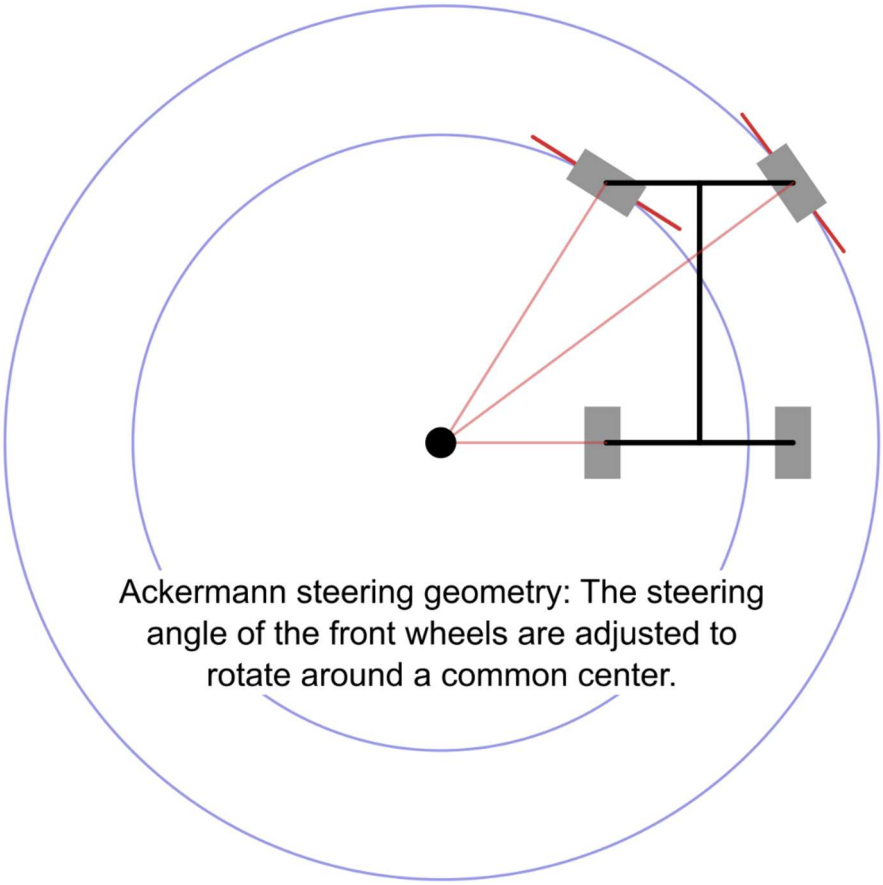
The inner wheel moves a little further than the outer wheel when the steering wheel is rotated during a turn: The steering angle of the inner wheel is larger than that of the outer wheel. This arrangement is referred to as Ackermann steering geometry and supports vehicle traction in tight turns. Since only the front wheels are actively steered in cars, a car has to travel on a circle whose center is an extension of the rear axle to prevent the wheels from slipping. Front wheel steering is also the reason why parking in narrow parking lots is easier in reverse than when driving forward: The turning point of the car is always located at the rear axle.

If the front wheels use the exact same steering angle in this constellation, their angles do not meet at this turning point. Consequently, they cannot move in a stable circle but are pulled sideways relative to their direction of travel. The Ackermann steering geometry approaches both front wheels to this ideal turning point, allowing the car to corner as stably as possible. This concept was originally developed for horse-drawn carriages and later adapted for cars. However, cars often do not employ full Ackermann steering geometry because this setup predominantly increases traction at low speeds. Fast cornering results in various other effects, such as shifting the car's mass to the outer wheels and body roll toward the outer side of the turn, meaning that other steering geometries work better.

F1 cars represent the extreme case: they even use anti-Ackermann steering geometry, meaning the outer wheel has a larger steering angle than the inner wheel to maintain a high tire temperature due to the additional friction (higher slip angle) of the inner wheel. High-speed applications require high tire temperatures to ensure optimal traction. The disadvantages of this configuration are elevated tire wear and poorer turning characteristics at slow speeds, which are not significant concerns for these racing cars.



Parallel steering: there is no common center of the turn and the wheels start to slip.



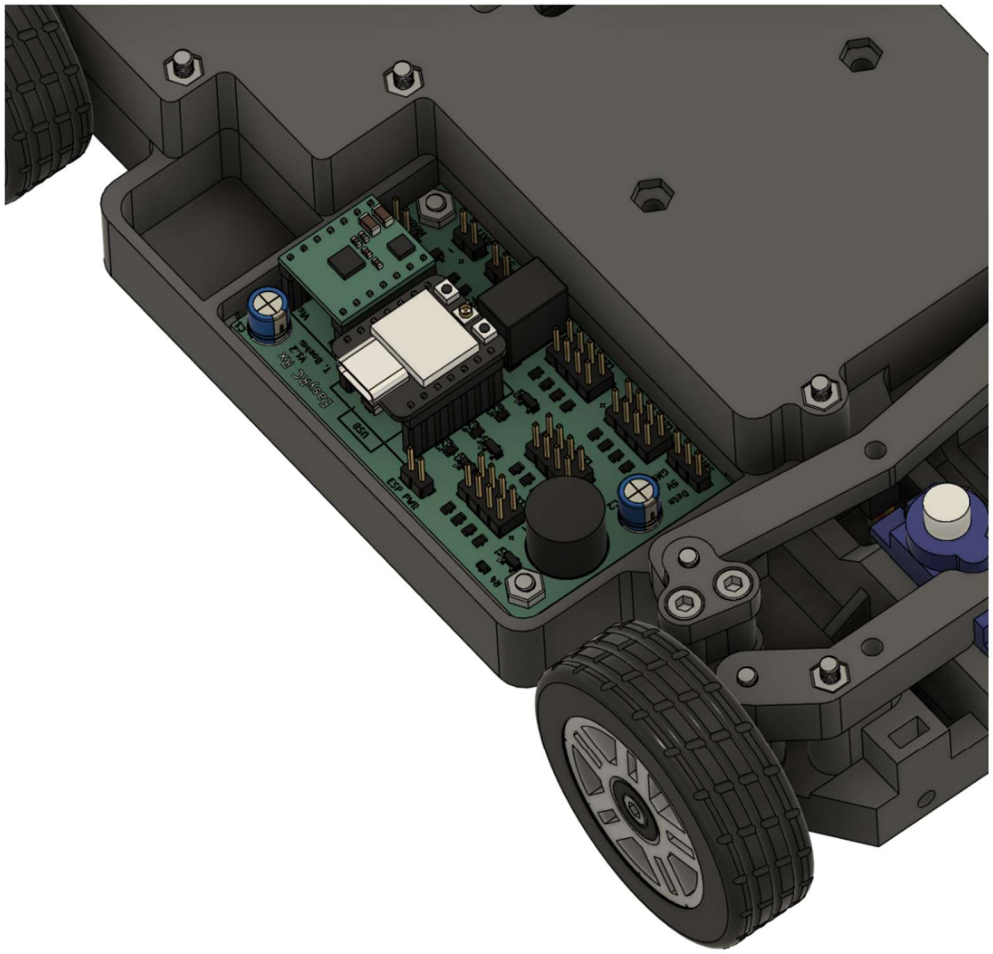
Ackermann steering geometry: The steering angle of the front wheels are adjusted to rotate around a common center.

4.4 Finalization of the chassis

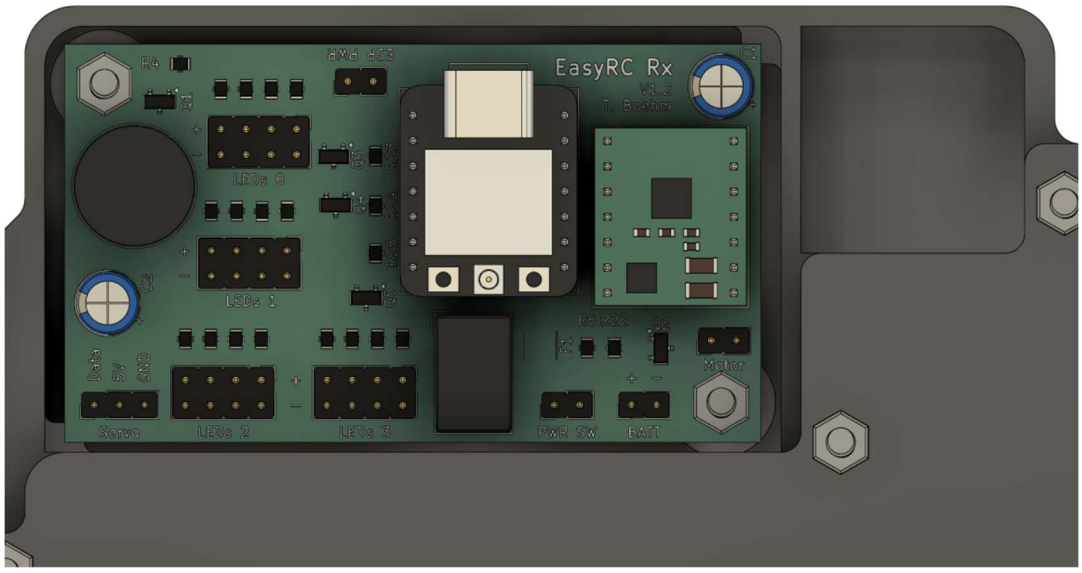
4.4.1 Printed circuit board

Required parts: Chassis (prepared), EasyRC Rx circuit board, M3x12 mm screw (2x), M3 hex nut (2x)

1



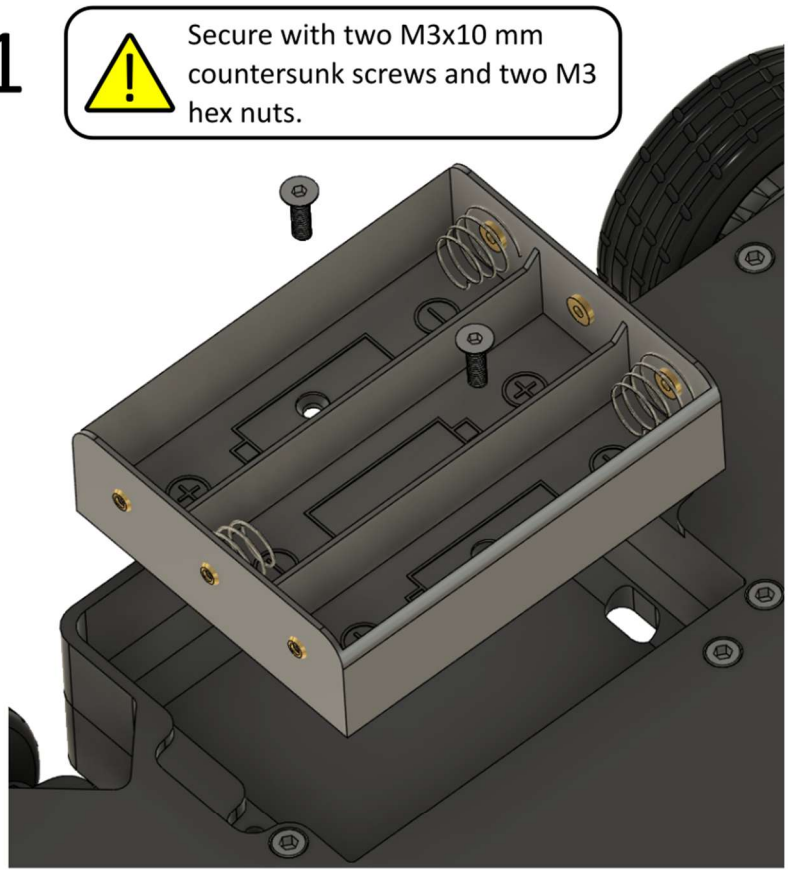
2



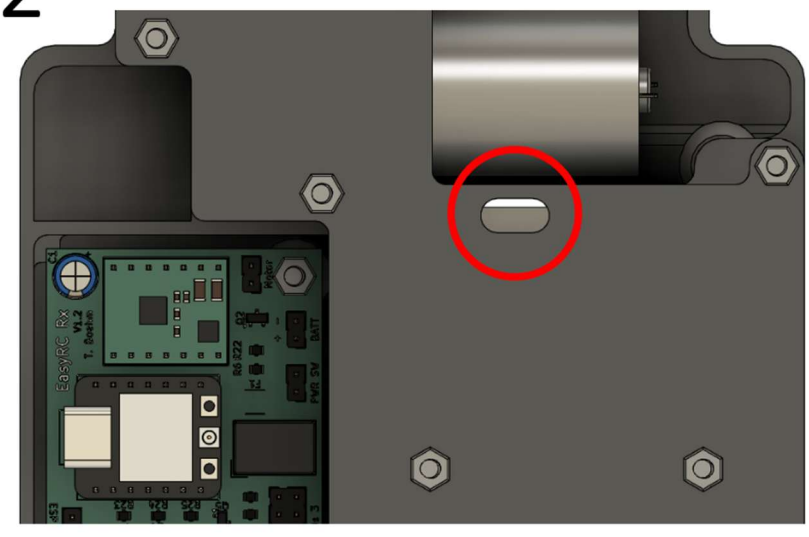
4.4.2 Battery compartment

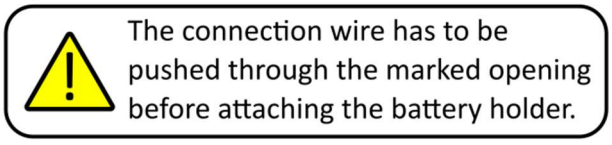
Required parts: Chassis (prepared), Battery lid, Battery holder 3x 18650 LiIon cell, M3x12 mm screw (2x), M3x10 mm countersunk screw (2x), M3 hex nut (2x)

1



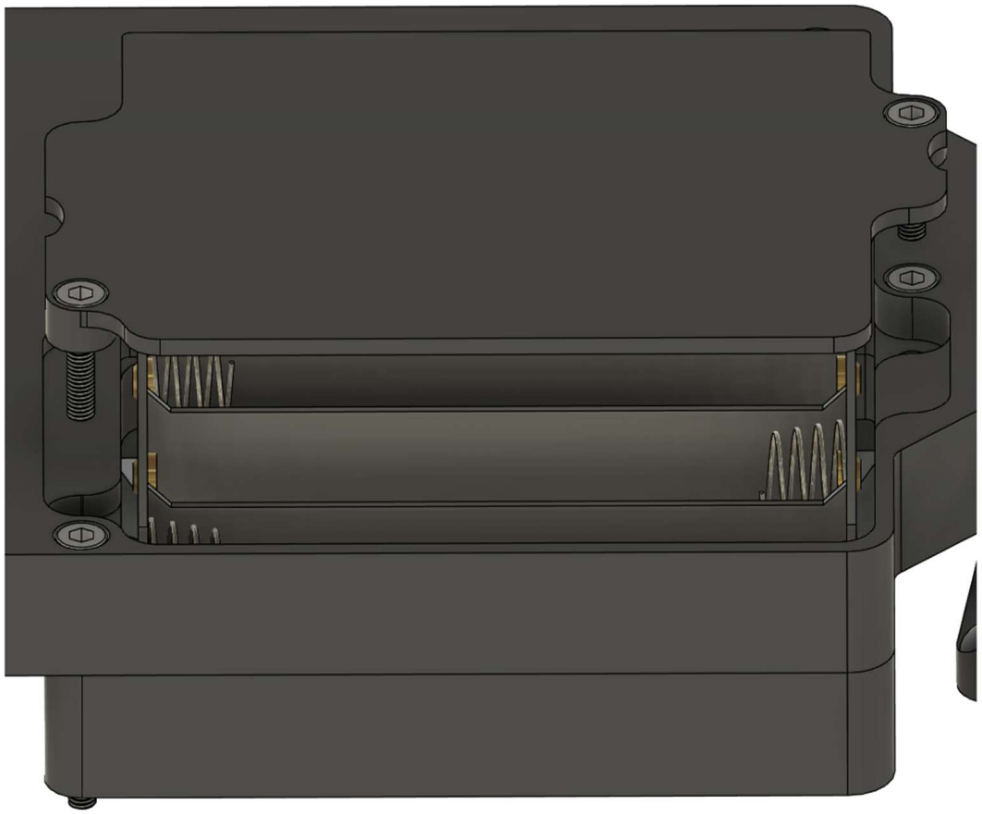
2





3

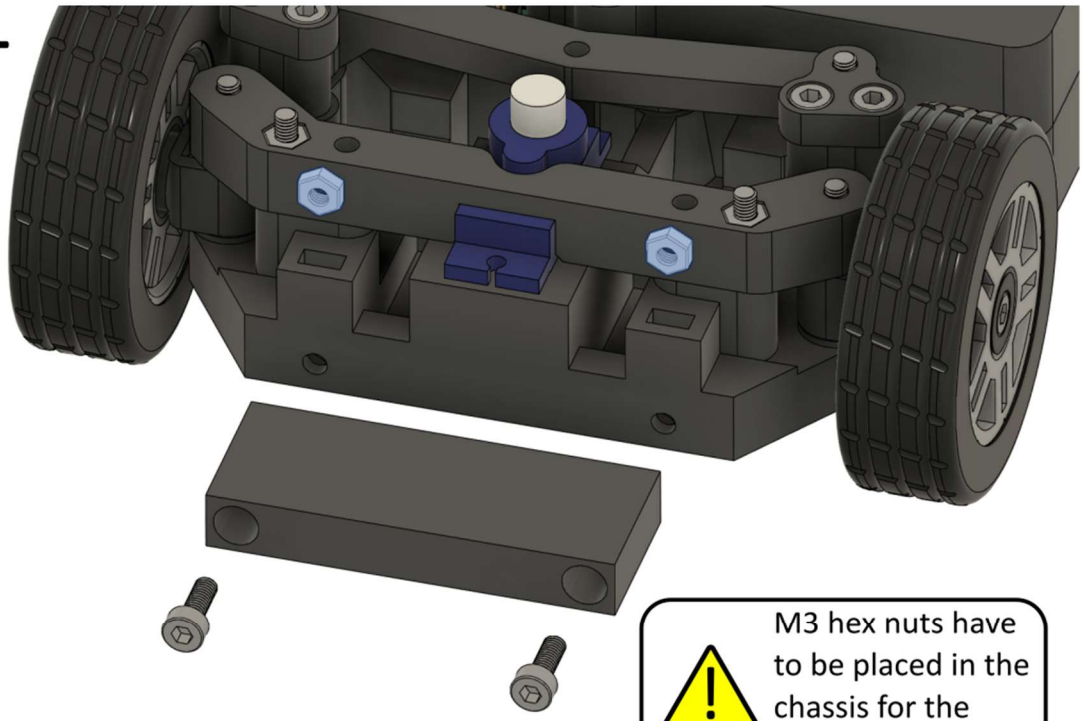
The battery lid has to be fixed with two M3x12 mm screws.



4.4.3 Bumpers

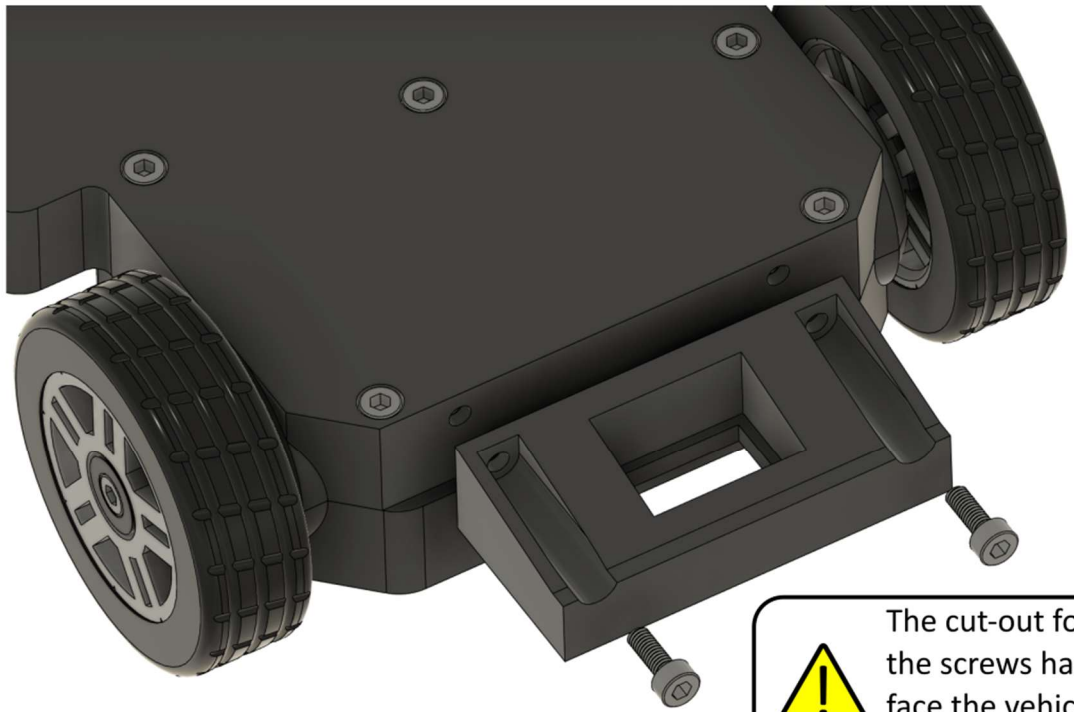
Required parts: Chassis (prepared), Front bumper, Rear bumper, on/off switch, M3x8 mm screw (4x), M3 hex nut (2x)

1



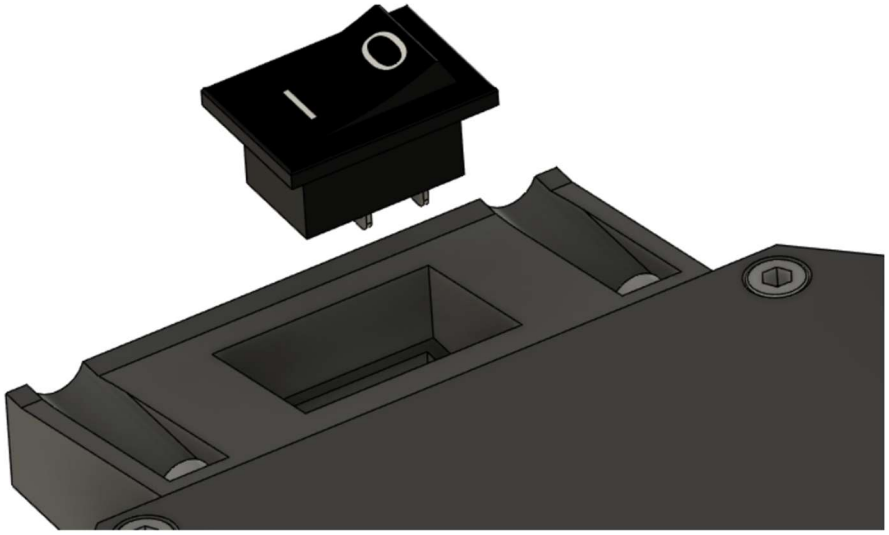
M3 hex nuts have to be placed in the chassis for the front bumper

2



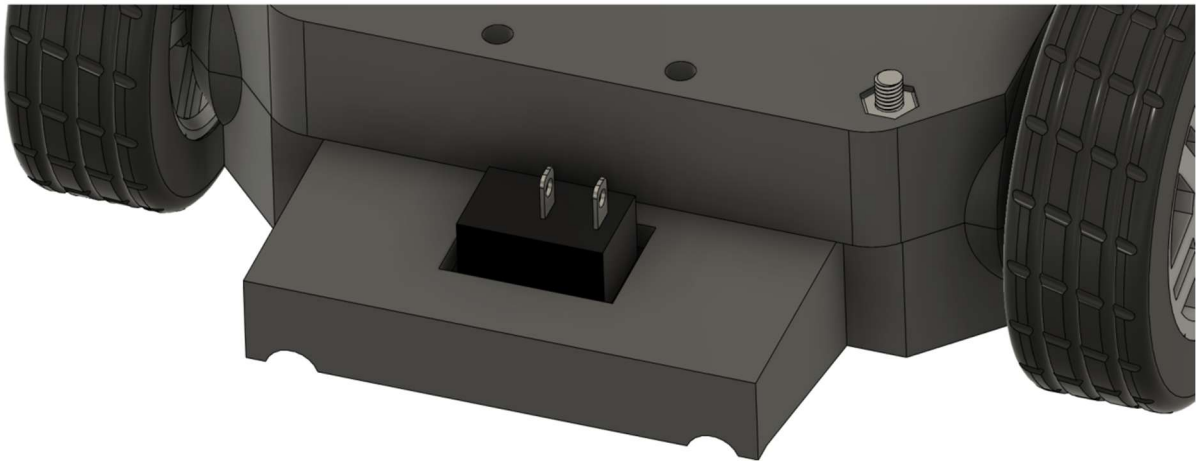
The cut-out for the screws has to face the vehicle's bottom!

3



The on/off switch is pushed into the rear bumper (with gentle pressure, it snaps in place).

4

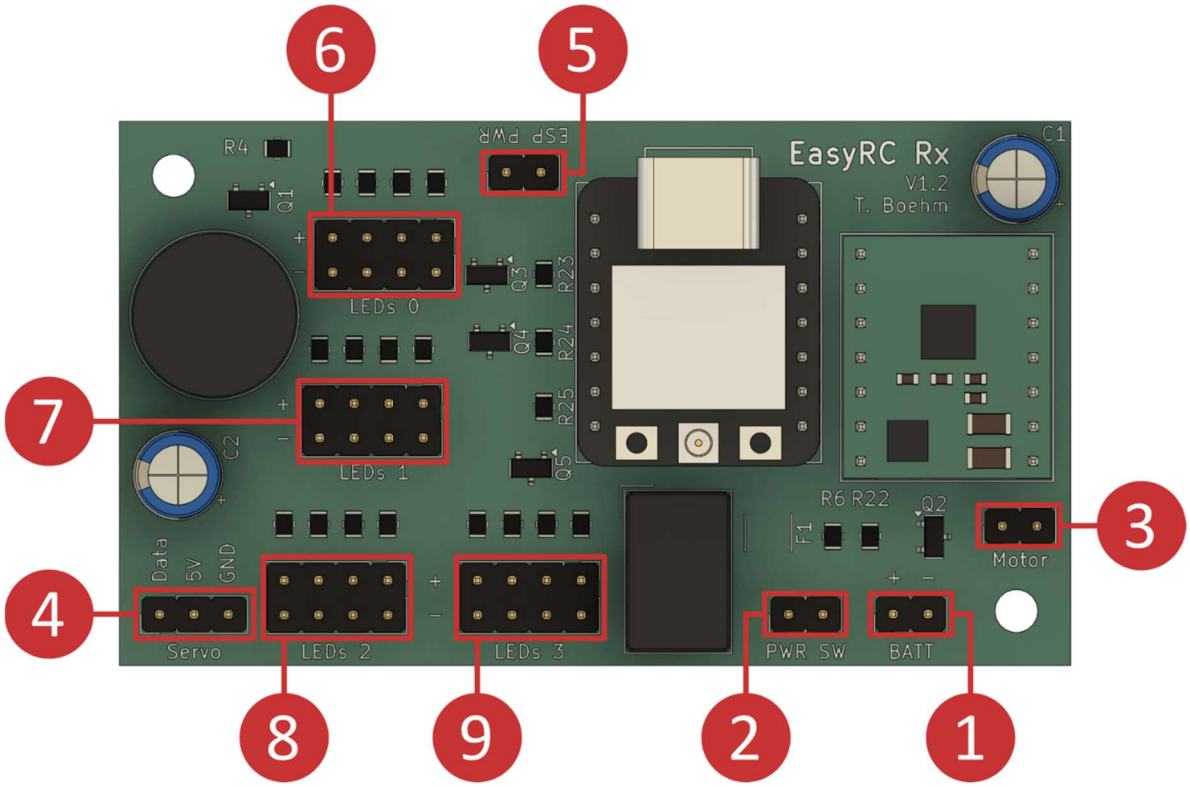


4.4.4 Wiring

The wires that connect to the chassis encompass the numbers 1 to 4: Battery holder, on/off switch, drive motor, and steering servo can already be connected. Ensure the polarity is correct (refer to the table below).

The short-circuit bridge (jumper, mark 5) must be removed before programming the vehicle. If the microcontroller is programmed while the jumper is still installed, excessive current can flow from the USB connector to the circuit board, which may result in damage to the PC.

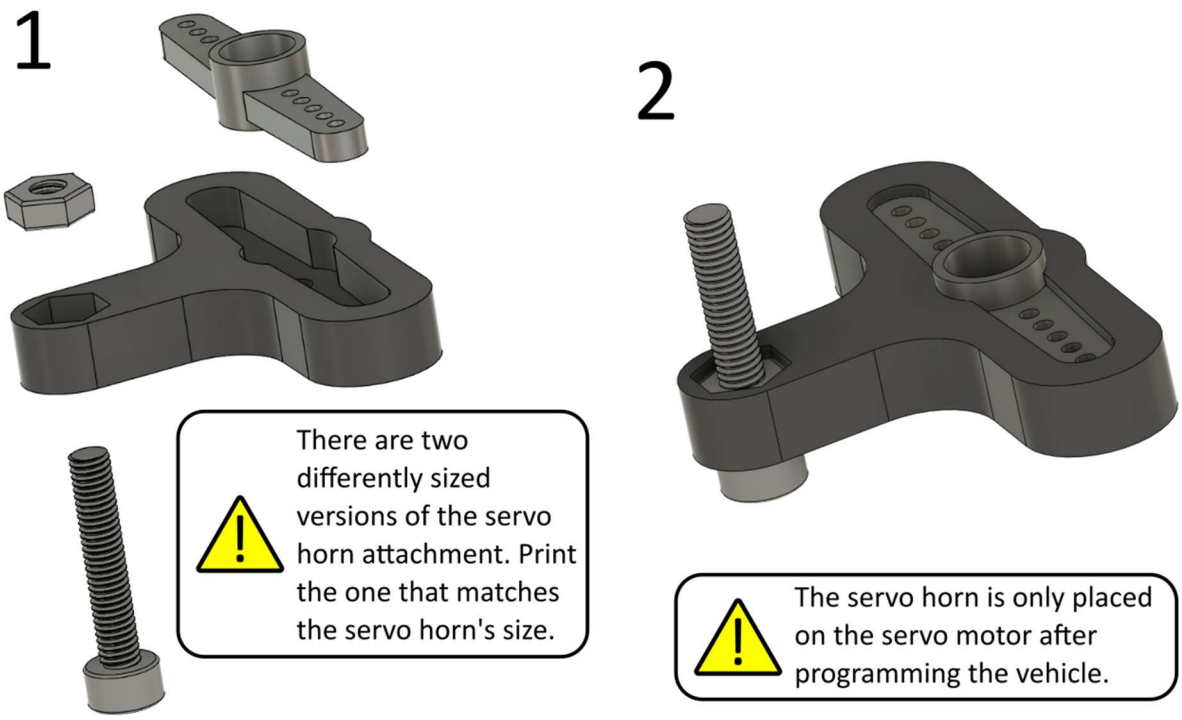
The LEDs (labels 6 to 9) will be connected during the final assembly of the car, as described in Chapter 7.3. Four LEDs can be connected per channel. The standard body kit is designed for two LEDs per channel. Any of the four connectors per channel can be used to connect an LED (without preferential order).



Number	Designation	Polarity
1	BATT (battery connection)	Red to +; black to -
2	PWR SW (on/off switch)	No matter
3	Motor (drive motor)	No matter (rotation direction set by software)
4	Servo (steering servo)	Yellow to data; red to 5V; brown to GND
5	ESP PWR (jumper)	No matter
6	LEDs 0 (headlights)*	Red to +; black to -
7	LEDs 1 (rear lights)*	Red to +; black to -
8	LEDs 2 (left indicator lights)*	Red to +; black to -
9	LEDs 3 (right indicator lights)*	Red to +; black to -
*The LED connector headers consist of four connector pairs: Up to four LEDs can be connected per channel. In this view, the upper contacts are + and the lower contacts are -. The body kit is designed for two LEDs per channel.		

4.4.5 Preparation of the servo horn

Required parts: Servo horn attachment, servo horn, M3x16 mm screw, M3 hex nut



There are two 3D models for the servo horn attachment because the MG90 servos come with servo horns of different sizes. The smaller servo horn has a length of approximately 32 mm, and the larger one has a length of approximately 37 mm. The servo horn attachment with the suitable recess must be printed and employed to enable steering without backlash.

4.5 Body kit

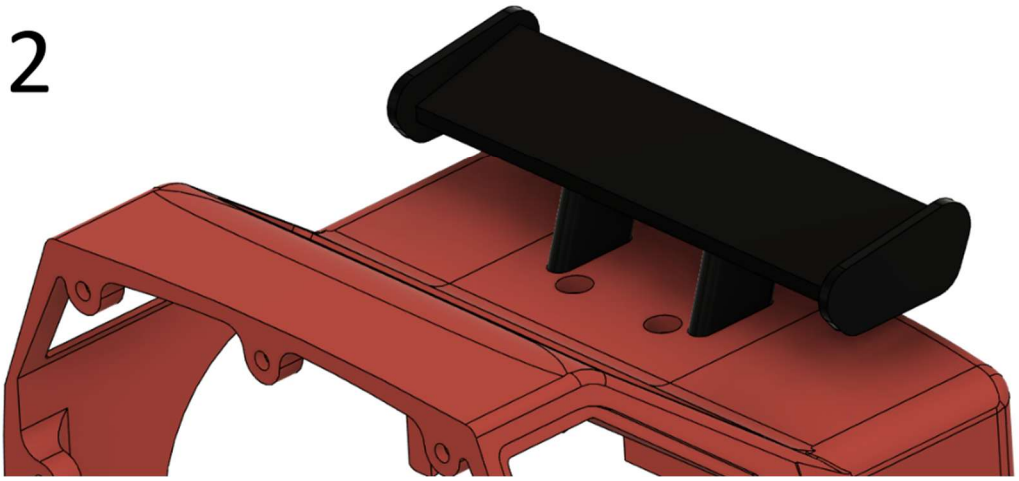
4.5.1 Rear wing

Required parts: Rear body, Rear wing, Rear wing holder, M3x8 mm screw (2x), M3x16 mm screw (2x), M3 hex nut (2x)

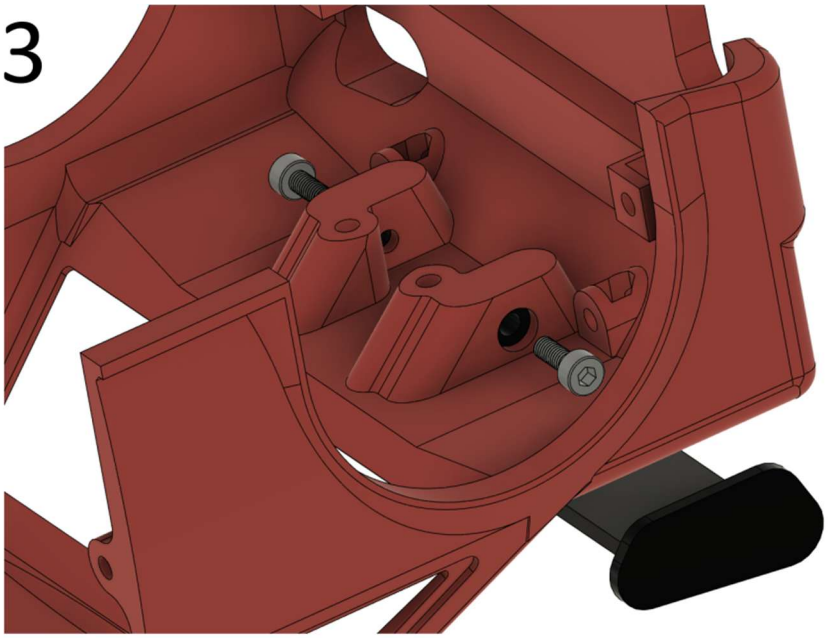
1

If the rear wing gets stuck when trying to insert it, the parts that are pushed into the rear body can be sanded down with sand paper.

2

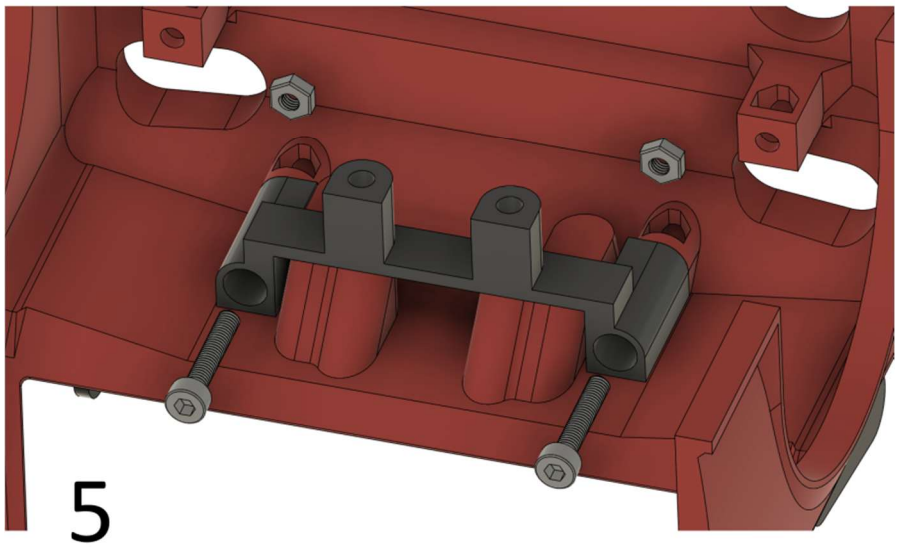


3

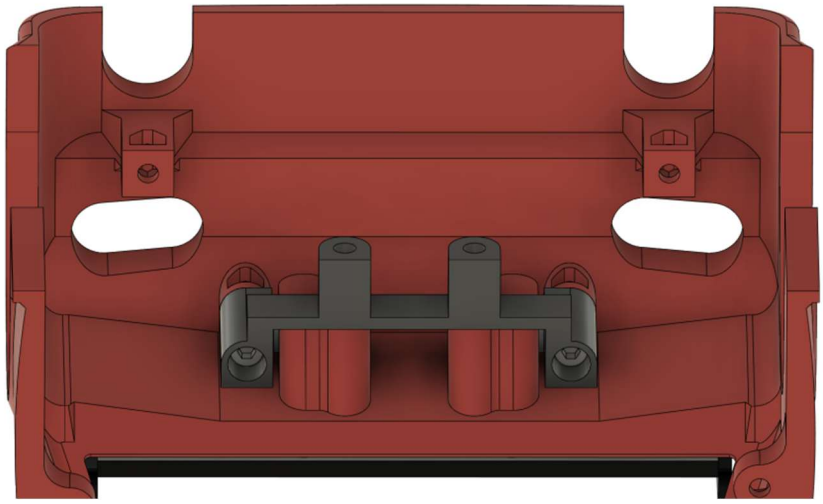


4

The M3x8 mm screws are only pushed in and are kept in place by the rear wing holder.



5

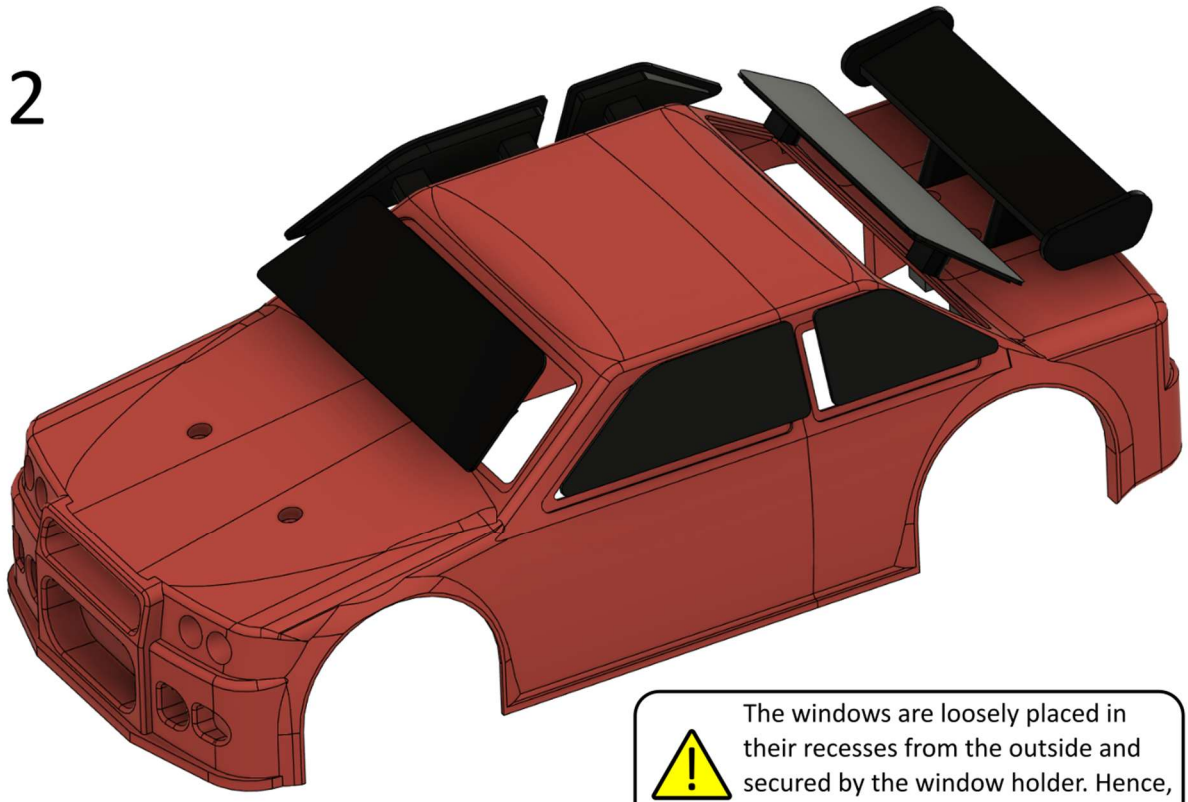


4.5.2 Body connection and windows

Required parts: Rear body (prepared), Front body, Windows (1x front, 1x left front, 1x left rear, 1x right front, 1x right rear, 1x rear), Window holder, M3x8 mm screw (15x), M3 hex nut (15x)

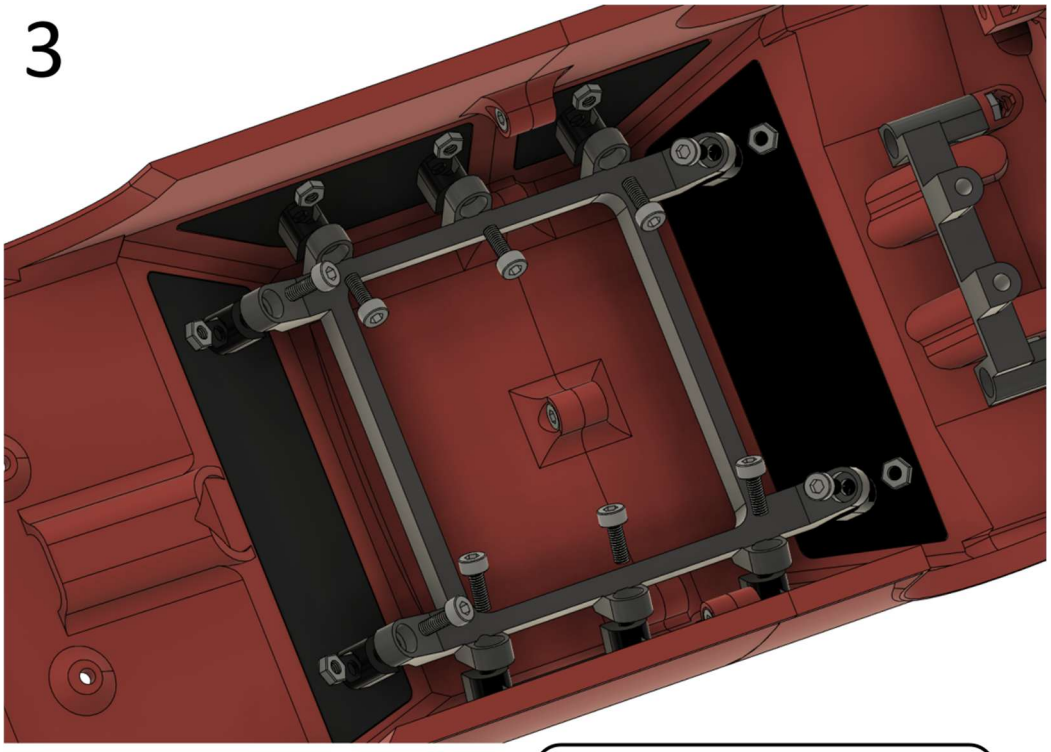
1

2



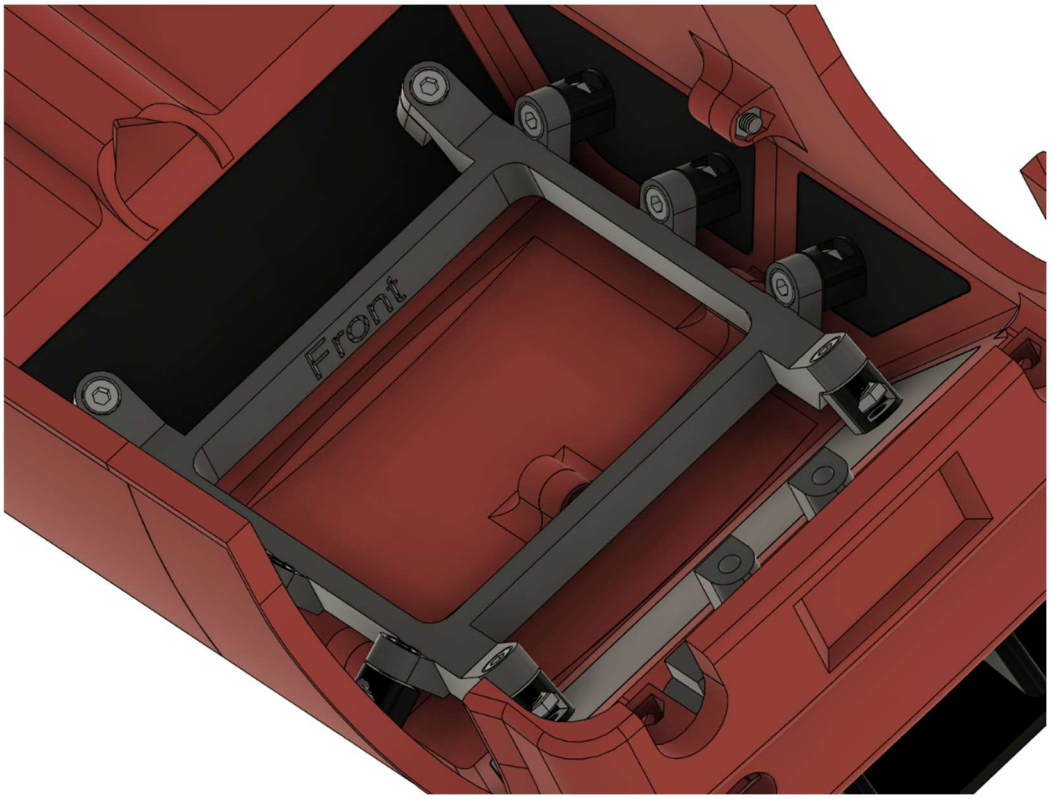
The windows are loosely placed in their recesses from the outside and secured by the window holder. Hence, place and screw on after the other.

3



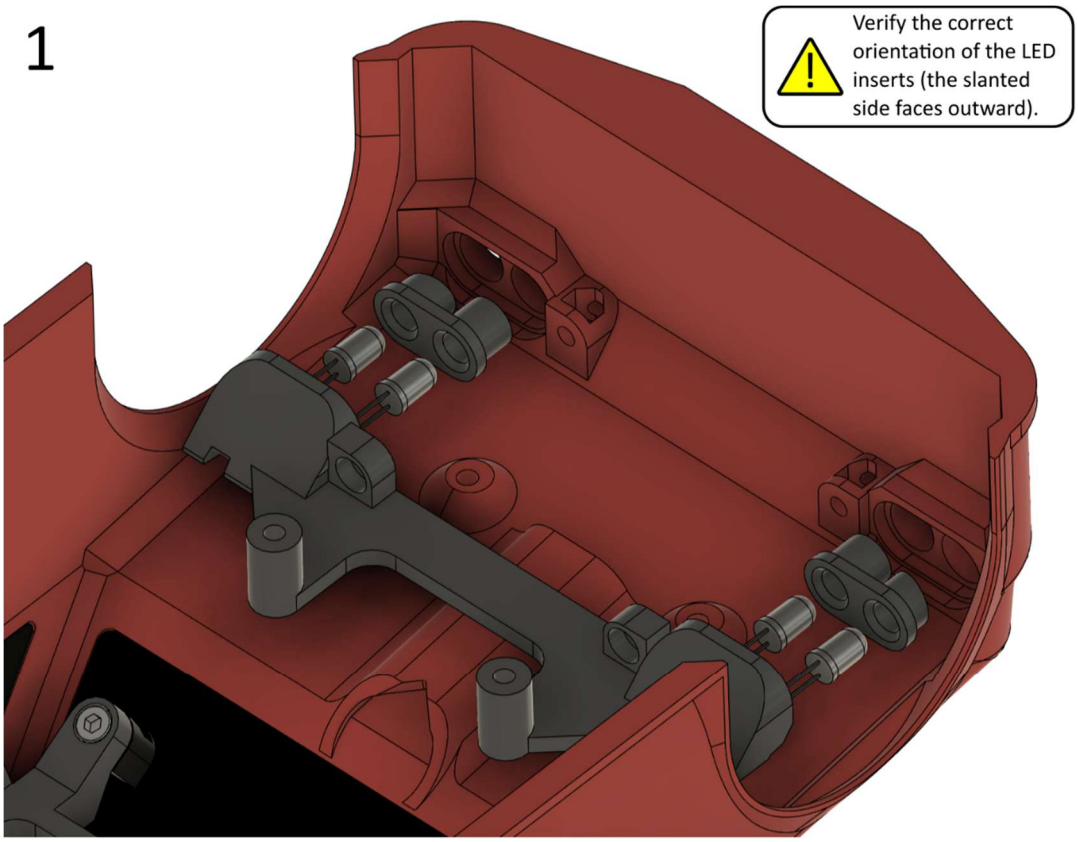
4

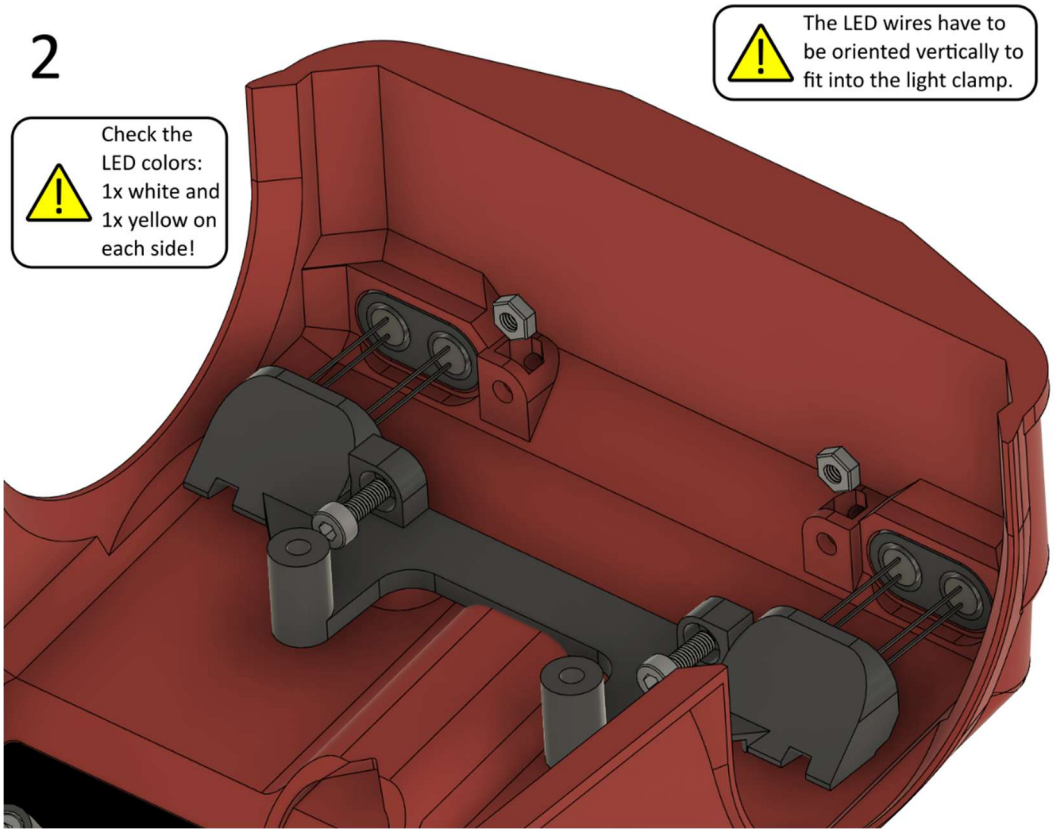
Mind the correct orientation of the window holder.



4.5.3 Headlights

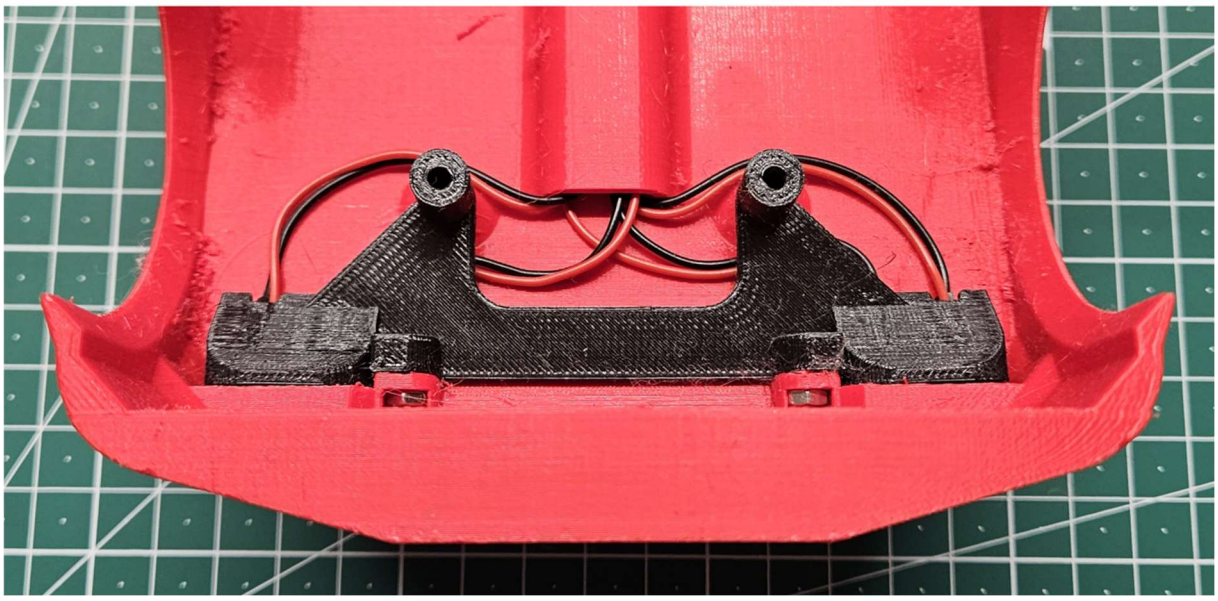
Required parts: Body (prepared), Front light clamp, Front LED insert (2x), LED white (2x), LED yellow (2x), M3x8 mm screw (2x), M3 hex nut (2x)





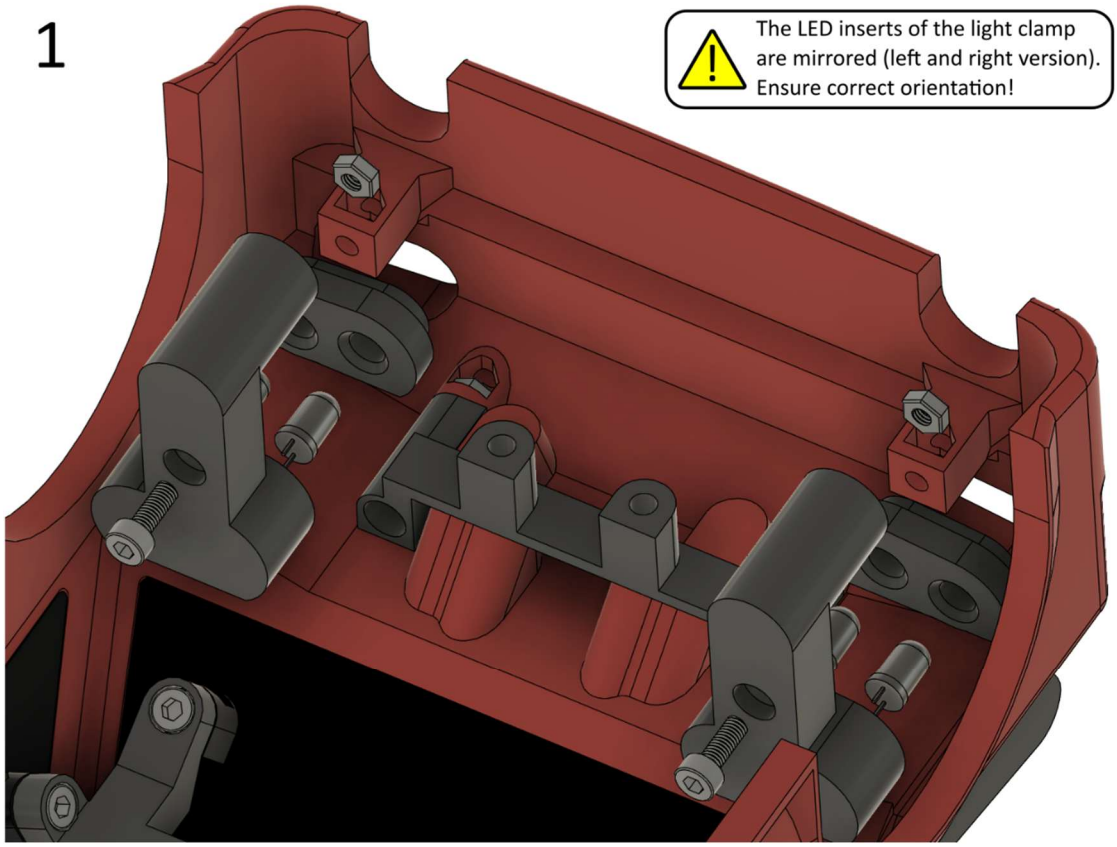
3

The LED wires have to be oriented as shown in the photograph.



4.5.4 Rear lights

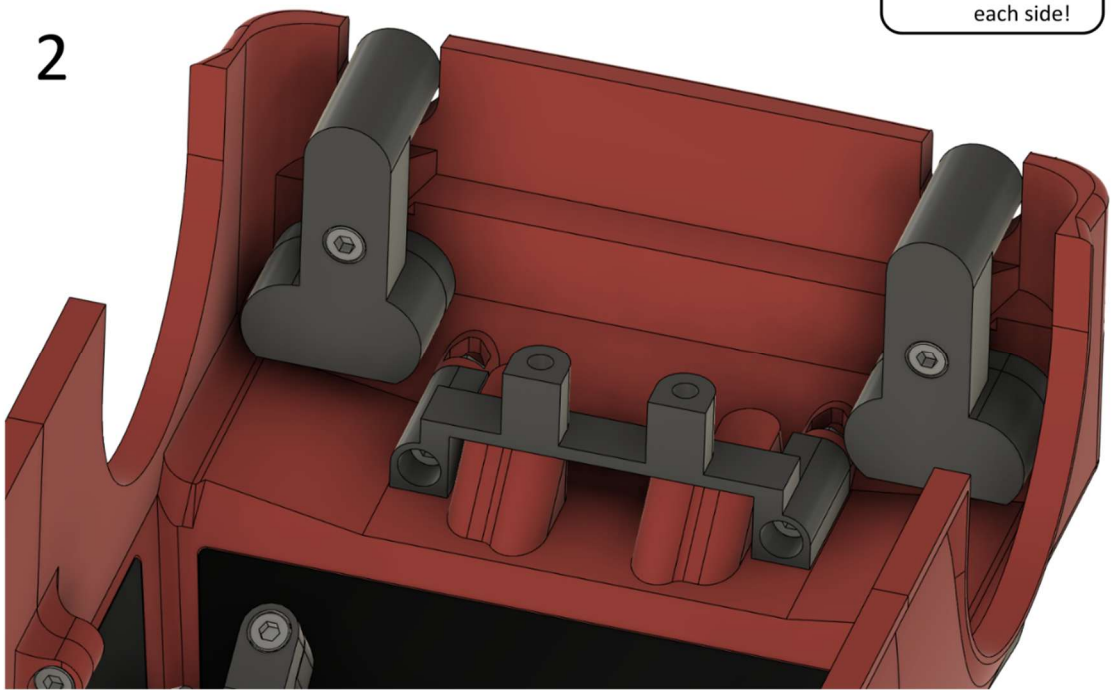
Required parts: Body (prepared), Rear light clamp (2x), Right rear LED insert, Left rear LED insert, LED red (2x), LED yellow (2x), M3x8 mm screw (2x), M3 hex nut (2x)



The hex nuts have to be inserted prior to placing the light clamps.

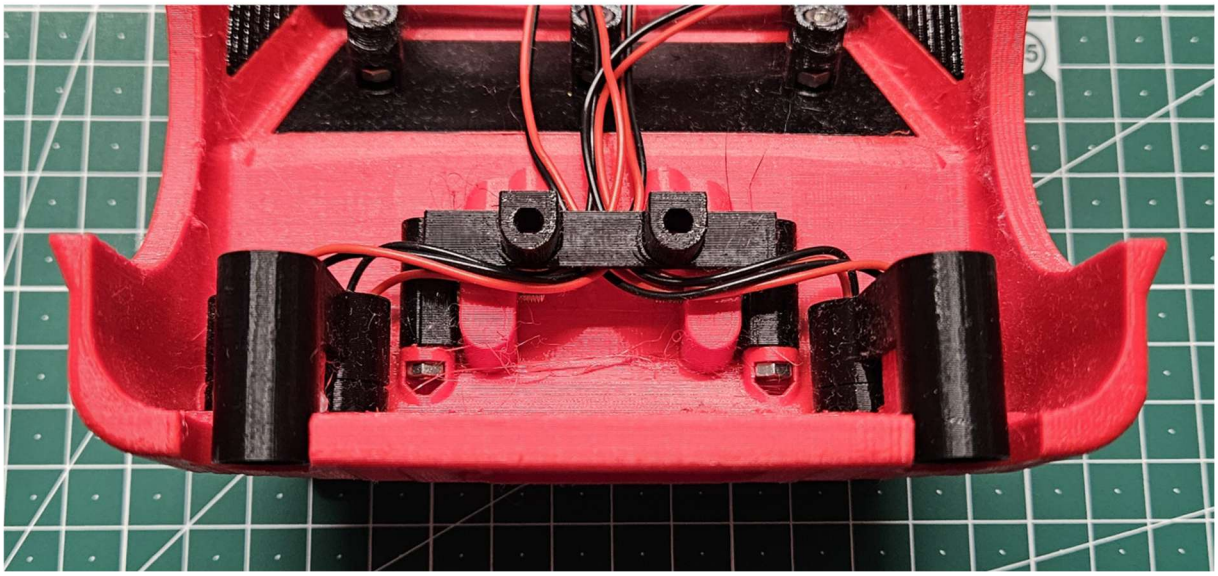
The LED wires have to be oriented vertically to fit the light clamps!

Check the LED colors: 1x red and 1x yellow on each side!



3

The LED wires have to be oriented as shown in the photograph.



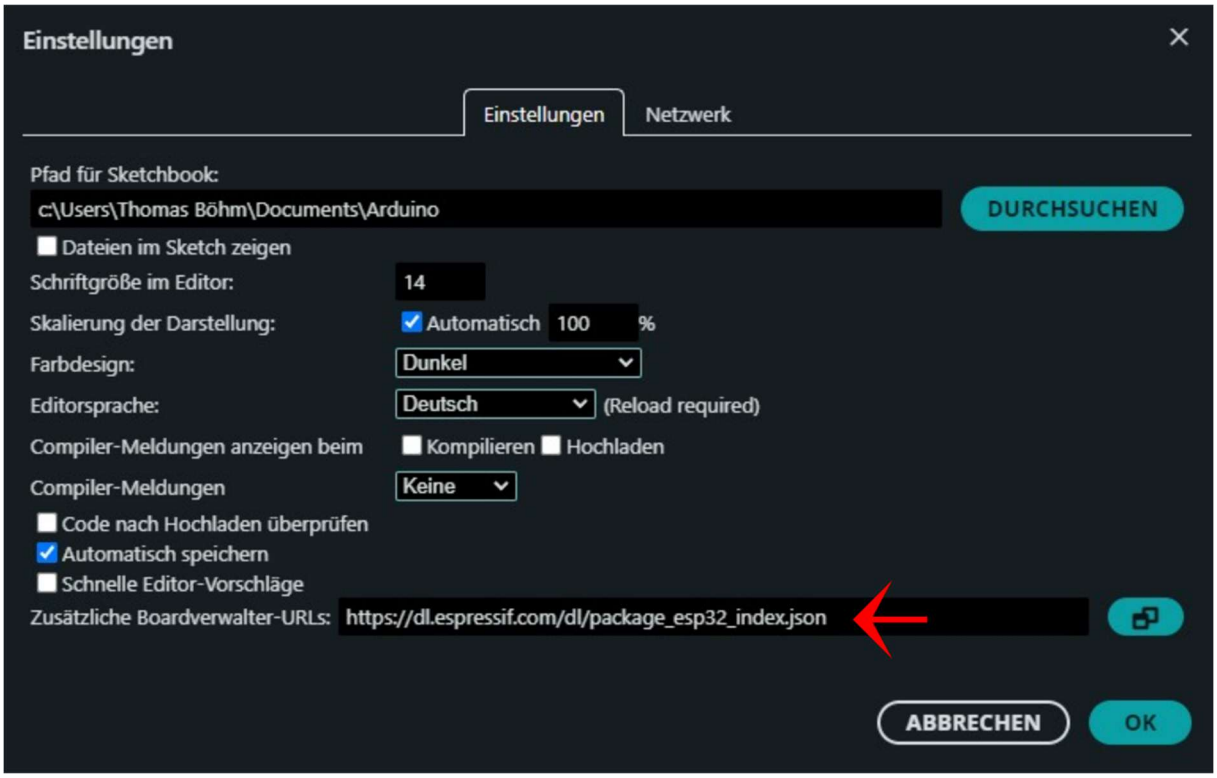
5. Programming

5.1 Installation of Arduino IDE

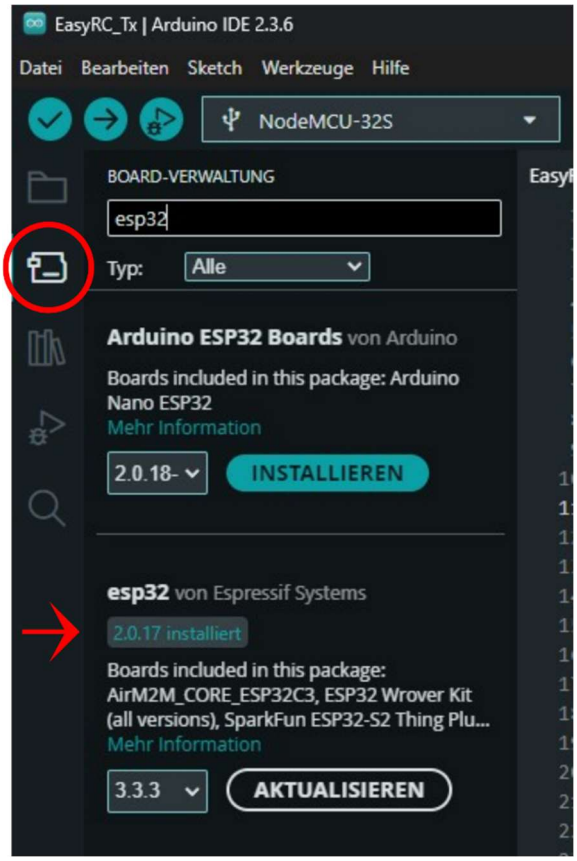
EasyRC utilizes a microcontroller from the ESP32 family, developed by Espressif Systems. This platform can be programmed freely using the Arduino development environment. Therefore, the software Arduino IDE (tested with version 2.3.6) is used to program the microcontrollers:

https://www.arduino.cc/en/software/

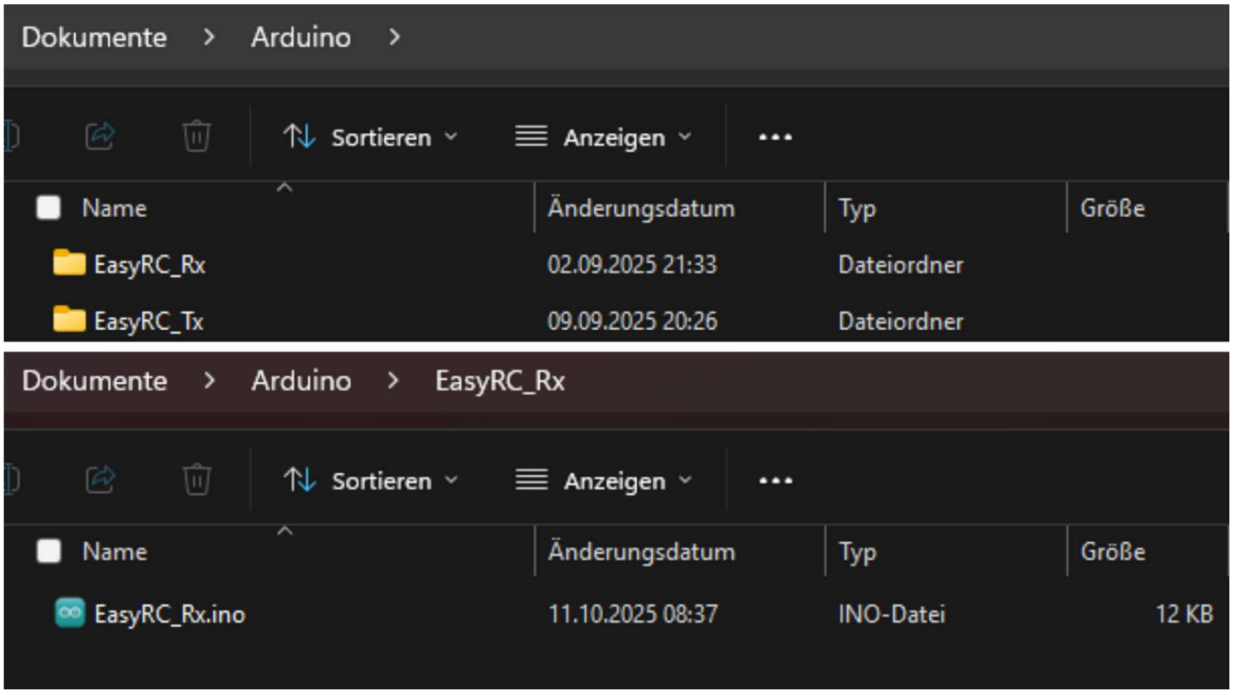
After installing the software, it must be set up for use with ESP32 microcontrollers. Enter the address https://dl.espressif.com/dl/package_esp32_index.json under File -> Preferences in the field "Additional boards manager URLs ".



Next, the ESP32 boards manager must be installed. To this end, click on the second symbol from the top in the left symbol column of the application's main window. Then, the boards manager opens as a sidebar. Here, enter "esp32" and select and install "esp32 by Espressif Systems". Ensure you select version 2.0.17 instead of the latest version. This legacy version is required because some of the libraries used for programming the ESP32 need a different syntax in version 3 compared with version 2. However, version 3 was not as stable as version 2, which is why EasyRC was written for the older version.



You can open a code file in the Arduino IDE when it is saved in a folder with the same name as the file (but without the .ino suffix). The default path for Arduino IDE in Windows is Documents -> Arduino. If no changes are to be applied, you can create the two required folders for the EasyRC remote control and vehicle codes, and then save the code files in these folders. The files are open source and available for download at https://github.com/TRD-B/EasyRC.



5.2 Possible and necessary code adjustments

EasyRC is designed to allow uploading the receiver code (of the vehicle) (EasyRC_Rx.ino) without changes to the microcontroller if the hardware address (MAC address) should not be modified. The hardware address must be changed if more than one EasyRC system is to be used at the same location. The hardware address defines which remote control is in charge of which vehicle (see also Chapter 9.1). The code of the transmitter (the remote control) (EasyRC_TX.ino) includes several variables that must be modified to calibrate the car. They can be found in the section "//calibration data "within the code:

EasyRC_Tx.ino

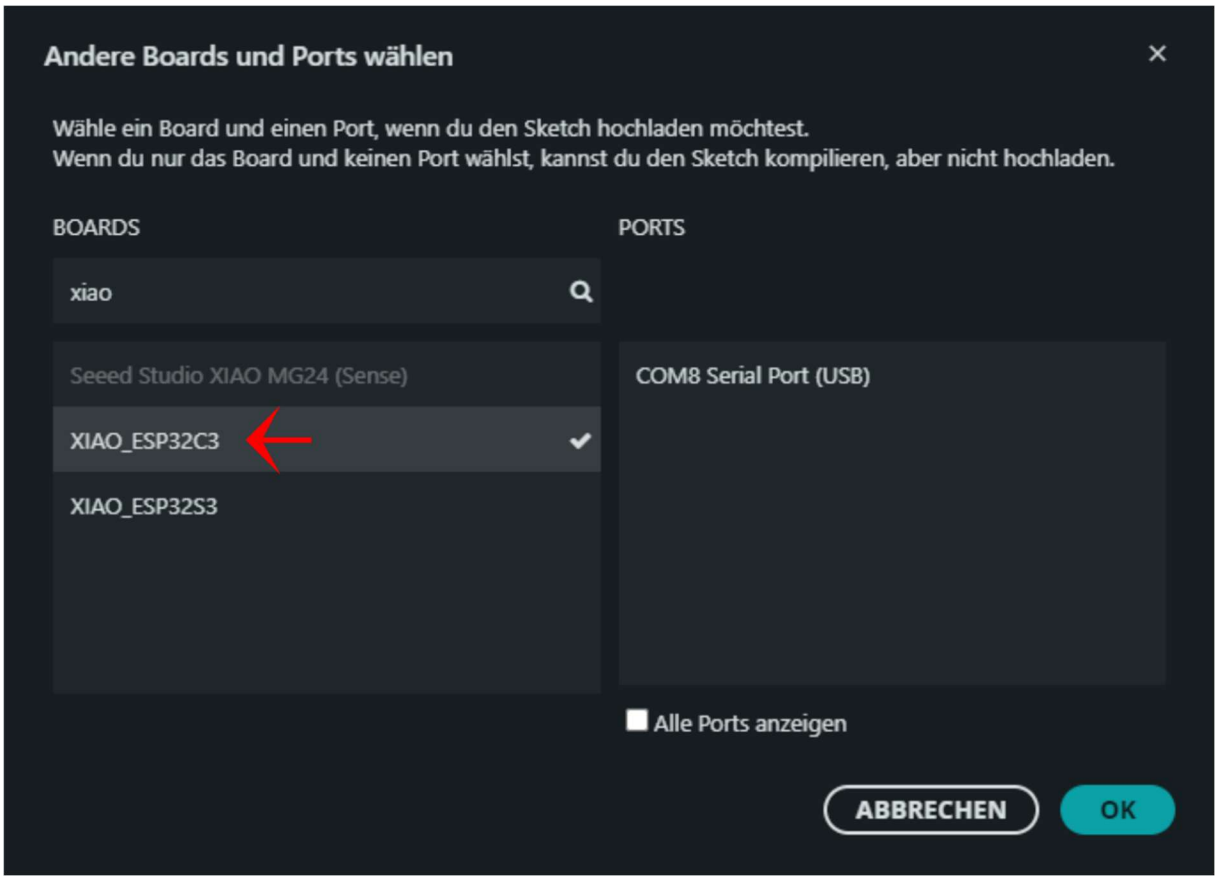
```
1  //Libraries
2  #include <WiFi.h>
3  #include <esp_now.h> //library for bidirectional wireless communication between two ESP32
4  #include <esp_wifi.h> //needed to adjust the MAC address
5  #include <Wire.h>
6
7  //Pin definitions
8  #define vsense 2
9  #define joyx 3
10 #define joyy 4
11 #define button 5
12 #define led 20
13
14 //calibration data
15 bool motordirection = false; //change this variable to false to invert the motor spin direction
16 int motorint = 10; //sets the delay time between motor speed changes (in ms). Higher values result in
17 //better traction but slower responses. Values between 5 and 10 are recommended.
18 int speedlimit = 127; //this variable sets the maximum speed of the car (values between 0 and 255)
19 int leftlimit = 358; //steering limit calculation: leftlimit = x/20ms*2^12 (default: 205 (x = 1ms))
20 int rightlimit = 258; //steering limit calculation: rightlimit = x/20ms*2^12 (default: 410 (x = 2ms))
21 //RC servos run at 50 Hz (20 ms intervals), with the center at 1.5 ms and a range of +-0.5 ms around the center
22 //starting values should be 205 to 410 (left to right or right to left, depends on the servo)
23 //adjust these values to a smaller or larger interval depending on the actual steering angles
24 //individually decrease/increase one of the two values to trim the center of the steering
25
26 //MAC addresses
27 uint8_t CarMAC[] = {0xAA, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
28 uint8_t RemoteMAC[] = {0xB0, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control
29 uint8_t broadMAC[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}; //using this MAC address as a receiver for sending
30 //packages means broadcasting (ignoring if the data has been received)
```

Line	Code	Possible values	Explanation
15	bool motordirection = false;	true, false	This variable inverts the rotation direction of the drive motor. Replace true with false if the car drives backward when pushing the joystick forward.
16	int motorint = 10;	1 - 20	This variable adjusts the rate at which the vehicle responds to changes in speed input from the joystick position. Lower values result in a higher responsiveness. Higher values prevent tire slipping but cause the car to react more slowly to brake and acceleration inputs.
18	int speedlimit = 127;	51 - 255	This variable limits the vehicle's maximum speed. 255 is the maximum possible speed. Values below 51 (20% maximum speed) are possible but are not recommended, as the motor may not provide sufficient torque to actually move the car.

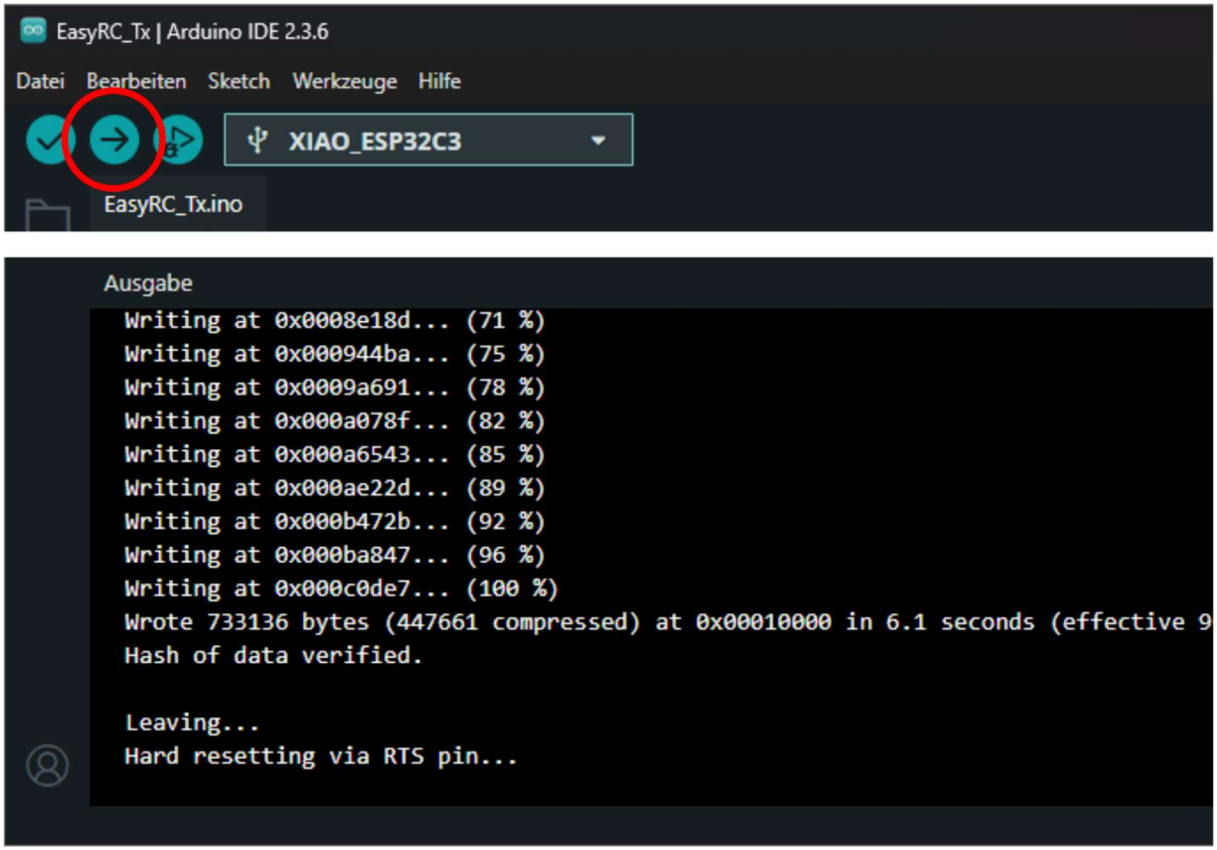
19	int leftlimit = 358;	308 - 410	This variable defines the maximum steering angle of the car to the left. The value is a result of how the microcontroller addresses the steering servo. A value of 308 equals the servo's neutral position (center). For the tested servo motors, a higher value equals turning the wheels to the left. However, this can vary depending on the servo model used. A value of 410 equals the maximum possible angle of the servo motor. Do not set this maximum value mindlessly, but check for the largest possible value in a stepwise manner. The largest possible value corresponds to the largest steering angle of the car at which the wheels do not yet touch the body.
20	int rightlimit = 258;	205 - 308	This variable defines the maximum steering angle of the car to the right (analogously to the steering angle to the left). The neutral position of the car's steering is a result of the mean value of the maximum steering angles on the left and right sides. Hence, if one of the two values is increased (larger than 308), the other value has to be reduced by the same amount (smaller than 308) to avoid altering the vehicle's straight-line stability. The theoretical maximum interval between 205 and 410 for controlling the servo may differ depending on the servo model. For the tested servo motors, a smaller interval was sufficient.

5.3 Programming the microcontrollers

Uploading code to a microcontroller requires it to be connected to the PC via USB. **Remove the "ESP PWR" jumper before programming the vehicle's microcontroller (otherwise, the USB port of the PC can be damaged!).** The remote control can be directly programmed via the externally accessible USB port. After connecting the microcontroller to the PC, the correct model and USB port must be selected in the Arduino IDE. To this end, click on the field with the USB symbol in the top bar. Then, a dropdown menu opens in which you must click on "select other board and port...". Search for "Xiao" in the left column to find the correct microcontroller. The correct model is the "Xiao ESP32C3". The right column shows connected USB devices. The port number (COM8 in the figure) varies depending on the USB port used on the PC. If more than one port is listed here, although only one microcontroller is connected, you may remove any unnecessary USB devices and search again for active ports.



Once the model and port of the microcontroller are selected, the code can be uploaded. To do this, click on the arrow pointing to the right in the top bar of the Arduino IDE. Then, the computer checks the code for syntax errors, compiles it (converts it into machine-readable code), and transfers it via USB to the microcontroller.



The output window shows the displayed text ("Leaving... Hard resetting via RTS pin...") when the upload is successful. However, the USB data transfer is prone to error. If the upload aborts or fails to start with an error message, the microcontroller should be unplugged and then reconnected. Also, double-check that the correct USB port was selected. If the error persists after several attempts, restarting the PC may also resolve the issue.

6. Microcontroller code

This chapter provides a brief explanation of the code for the two microcontrollers, without focusing on the syntax. No snippets of the code are shown to keep this chapter concise. The codes can be downloaded at any time via GitHub: https://github.com/TRD-B/EasyRC

6.1 Remote control

In the remote control, the microcontroller must read the joystick sensors, check the battery voltage, and send the processed joystick data to the vehicle. First, program libraries are included, and the required variables are defined. Furthermore, outside of the functions within the code, the ESP-NOW data packet sent to the vehicle is defined. Then, the setup function follows that is executed once at the start of the microcontroller and performs initializations. For instance, it performs a calibration of the joystick axes: Upon switching the remote control on, the microcontroller reads both axes multiple times to determine their neutral position. Also, pin functions are defined, and ESP-NOW is initialized. The permanently repeated loop checks if the joystick button is pressed and sends a data packet to the vehicle every 50 ms. Also, it calls additional functions that include the remaining required actions (reading joystick axes, measuring battery voltage). It also generates a serial output that shows the sensor status of the remote control if a PC is connected.

A small area around the neutral position is not considered a deviation from the neutral point when reading the joystick axes, thereby preventing jittering of the steering and drive motor due to measurement noise and joystick tolerances. Furthermore, to maintain symmetry in the joystick axes, the microcontroller accounts for the possibility that the neutral position is not necessarily at the midpoint between 0 V and 3.3 V. Consequently, the maximum and minimum values of the joystick are capped at a specified threshold from the neutral position. The code responsible for reading the x-axis of the joystick (drive motor) contains a segment that inverts the output. This segment can be (de)activated by setting the variable (Chapter 5.2). Additionally, the x-axis readout instructs the vehicle to adjust the brightness of the rear lights (brake light versus rear light) based on whether it should drive forward, accelerate, maintain the neutral position, or reverse. Equally, the y-axis (steering) is used to obtain the data for the car to activate the indicator lights according to the steering inputs.

Reading the battery voltage is done by correcting for the employed voltage divider. It lets an internal counter count up if the voltage falls below a threshold (3.4 V). This threshold is comparably high for a Li-ion cell that can be discharged to around 3.0 V without deep discharge damage. However, since the battery must power an internal DC-DC converter on the Xiao ESP32-C3 developer board to provide a 3.3 V logic voltage to the microcontroller, a higher input voltage is required. As soon as the counter exceeds a defined value, the battery warning (the LED on the remote control blinks) is triggered. This counter is used to avoid triggering the battery warning already with a single event (voltage falling below the threshold once) to prevent false alerts due to measurement noise or abrupt movements of the remote control that result in voltage fluctuations (due to contact resistances between the battery and the battery holder).

The serial output shows the current sensor values of the microcontroller's inputs. This code segment is not necessary for the remote control's function, but it does not interfere with its operation and can be used for debugging purposes.

6.2 Vehicle

The microcontroller of the vehicle must receive data from the remote control and control the steering servo and the drive motor accordingly. Additionally, it controls a piezo buzzer, the vehicle's LEDs, and reads its battery voltage. The first segment of its code is used to call program libraries, define variables, and set the usage of ESP-NOW, similarly to the code of the remote control. The setup function initializes the usage of its pins. It programs the PWM timer of the microcontroller for the servo motor (setting to 50 Hz, which is the usual control frequency for RC servos). Then, ESP-NOW is activated, and LED light sequences and beep sequences of the piezo buzzer are called, indicating when the car is powered on, when it waits for a connection to the remote control, and when this connection has been established. The loop function calls additional functions that control the motors and the piezo buzzer. A larger code segment within the loop function checks the different alert levels and performs warnings (LEDs blink and the piezo buzzer beeps) when these levels are reached. The alert levels are connection losses to the remote control, no user input for extended periods, and the battery voltage falling below a specified threshold.

A connection loss to the remote control is detected by resetting a timer every time a new data packet is received (rctimer). If the current time exceeds the time when the last data packet was received by 1 s, the microcontroller considers this event as a connection loss. A lack of user input is measured using a separate function that resets a timer whenever the car is supposed to drive or should activate its horn. If neither of them was requested for 60 s, the threshold for no user input is exceeded. The battery voltage is measured using a separate function, similar to the way it is measured in the remote control; however, the voltage divider and low-battery threshold are adjusted to account for the higher battery voltage of the vehicle.

Controlling the indicator lights and the brake light is directly executed from the loop function of the vehicle's microcontroller, based on the input from the remote control. The rear light is less bright than the brake light, which is not achieved by PWM dimming of the LEDs, but by intermittently activating and turning off the LEDs every third iteration of the loop code. Still, no flickering of the LEDs can be observed because of how fast the ESP32 executes its code (<= 1 ms, resulting in a frequency of several hundred Hz). The indicator lights blink every second (they are switched on for half a second every second). Here, the timer used for this operation is reset with every direction change, so the indicator always starts with "lights on" whenever the wheels turn left or right.

Additionally, the loop code of the microcontroller checks if the vehicle's horn should be activated, and if yes, the required function to reprogram a PWM timer and channel (setPWMchannel) is called. Theoretically, the ESP32-C3 should have six separately programmable PWM channels; however, this feature does not function properly when programming it with the Arduino IDE. Whenever more than two channels are called, malfunctions occur in the PWM channels. Therefore, the code designates one channel and timer for the steering servo, and the other is used to control either the drive motor or the piezo buzzer, as required. This configuration results in the limitation that the car cannot drive and use the piezo buzzer simultaneously, but it allows for error-free operation of all components. The tone frequency of the piezo buzzer is set by its PWM frequency, and a tone is generated by setting the duty cycle of the PWM channel to 50% (in the code function buzzer). The drive motor is controlled with a PWM frequency of 20 kHz. This high frequency (inaudible region) avoids undesired whining and chirping noises of the motor under partial load.

The servo motor is directly controlled using the incoming steering commands (function ServoControl) because these inputs have already been processed in the remote control's code to allow controlling an RC servo. The servo motor requires a PWM signal with a frequency of 50 Hz, with the duty cycle specifying the desired angle of the motor. A signal duration of 1.5 ms equals the mean value, and the typically usable interval spans from 1 to 2 ms. The PWM channel of the servo motor is addressed with a depth of 12 bit, which means 1.5 ms corresponds to a value of 308 (308/(2^12) times 20 ms (50 Hz interval)), and the limits are 205 and 410.

The drive motor is controlled using the function setMotorSpeed, which adjusts the requested speed change depending on the current speed and the desired speed. If large changes are requested, this function executes an interpolation that results in a delayed actual change in motor speed. Hence, overly abrupt inputs that would cause a lot of strain on the drive train are prevented. The drive direction is only changed when the speed is close to zero, which is also meant to avoid excessive stress on the drive train.

7. First steps and operation of the vehicle

Once the microcontrollers of the remote control and the vehicle are programmed, the car is almost ready to run! Before starting to drive, the steering of the car must be trimmed, and the body must be attached to the chassis.

7.1 Inserting batteries and switching on the devices

First, insert the batteries of the vehicle and the remote control. The EasyRC circuit boards are protected from reverse polarity, but you should still never rely solely on safety features. Always double-check the orientation of the batteries when inserting them. Then, the remote control can be switched on. The status LED should be active permanently when the remote control works as intended. If it blinks, the battery voltage is low. Then, it must be charged before proceeding with the vehicle. Avoiding deep discharging the batteries is important: Li-ion cells are damaged if their voltage drops too low.

The joystick of the remote control is automatically calibrated when the remote control is switched on. The position that the joystick has when it is switched on is saved as its neutral position. Hence, the joystick must not be touched while switching on the remote control. This calibration is executed every time the remote control is switched on. Therefore, if the joystick was accidentally moved and not in its resting position, the remote control can simply be switched off and on again to obtain a correct calibration. Then, the vehicle can be switched on. The ESP PWR jumper must be installed before proceeding, as the microcontroller will not receive current otherwise. As the body is not yet installed, the LEDs are not yet connected to the PCB. Therefore, only the piezo buzzer provides feedback on the vehicle's status. The buzzer beeps three times after the car is switched on, with the last tone being higher-pitched and longer than the first two. Then, the piezo buzzer beeps every two seconds while the vehicle is not yet connected to its remote control. As soon as the car is connected to "its" remote control, a signal sequence with a longer and a shorter beep follows.

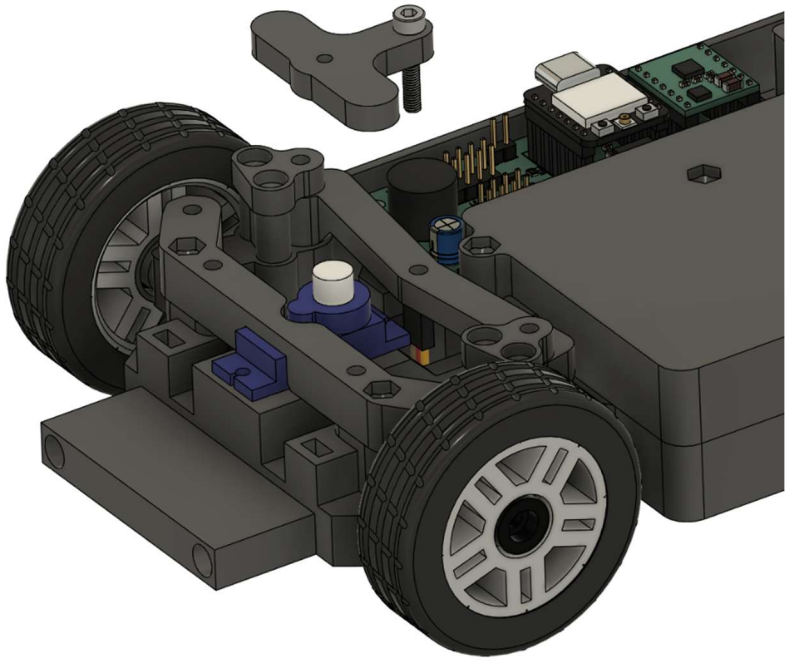
7.2 Finalizing the steering assembly

First, the steering system is completed. As described in the section on programming (5.2), the code contains two variables that define the maximum steering angles to the left and right. They must be adjusted so that the car moves in a straight line when the joystick is in its neutral position (relative to the axis from left to right). Further, the front wheels must not touch the body (the maximum steering must not be too large).

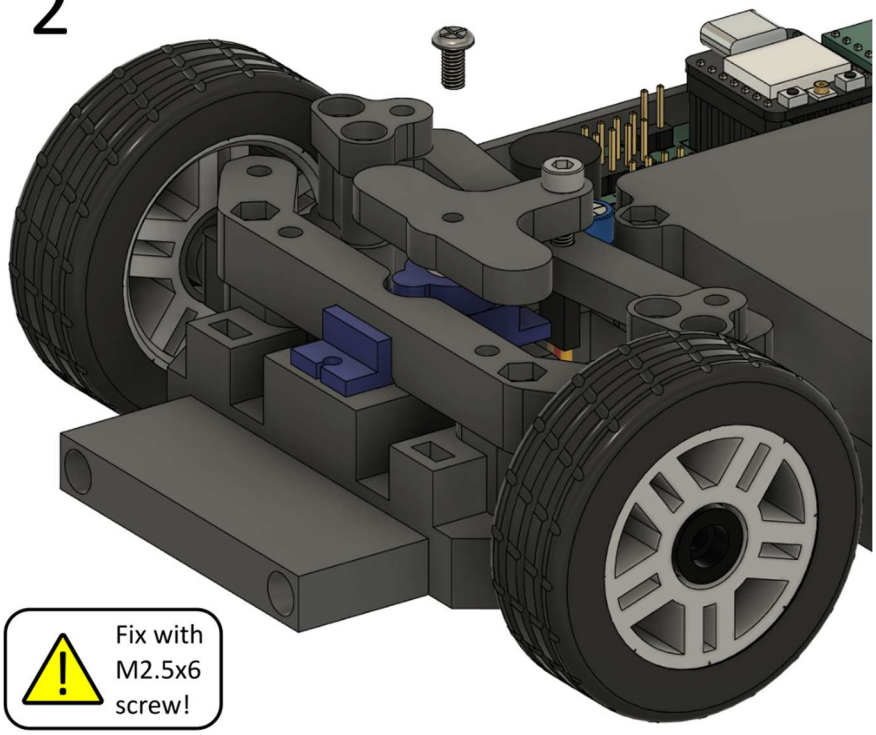
The servo horn is now placed on the servo motor and connected to the steering linkage. This step is only performed at this point because the servo motor must first reach its neutral position. The servo motor automatically rotates to its neutral position when the joystick of the remote control is not moved, and the mean value between the left and right maximum steering angles equals 308. This is the default setting when using the code from GitHub without modifications.

The servo horn must be placed on the motor at an angle that results in the line between the servo output shaft and the joint that connects the servo horn with the steering linkage being as parallel as possible with the longitudinal axis of the vehicle (see the red line in image 3 of the following figure). The output shaft of the servo features a small gear, and the servo horn must mesh with these teeth. Therefore, it may happen that the servo horn cannot be placed on the servo along this ideal line. However, this is not an issue if it is oriented as close as possible to this ideal line. The preinstalled M3x16 mm screw of the servo horn attachment must be pushed through the matching hole in the steering linkage. Finally, the servo horn must be secured to the servo motor using a machine screw with a length of 6 mm. The instructions specify this screw as an M2.5 screw; however, the diameter may vary depending on the servo type. Then, the basic settings for the steering are completed. The exact trimming is performed solely using the software of the remote control after finishing the assembly of the vehicle and its body.

1

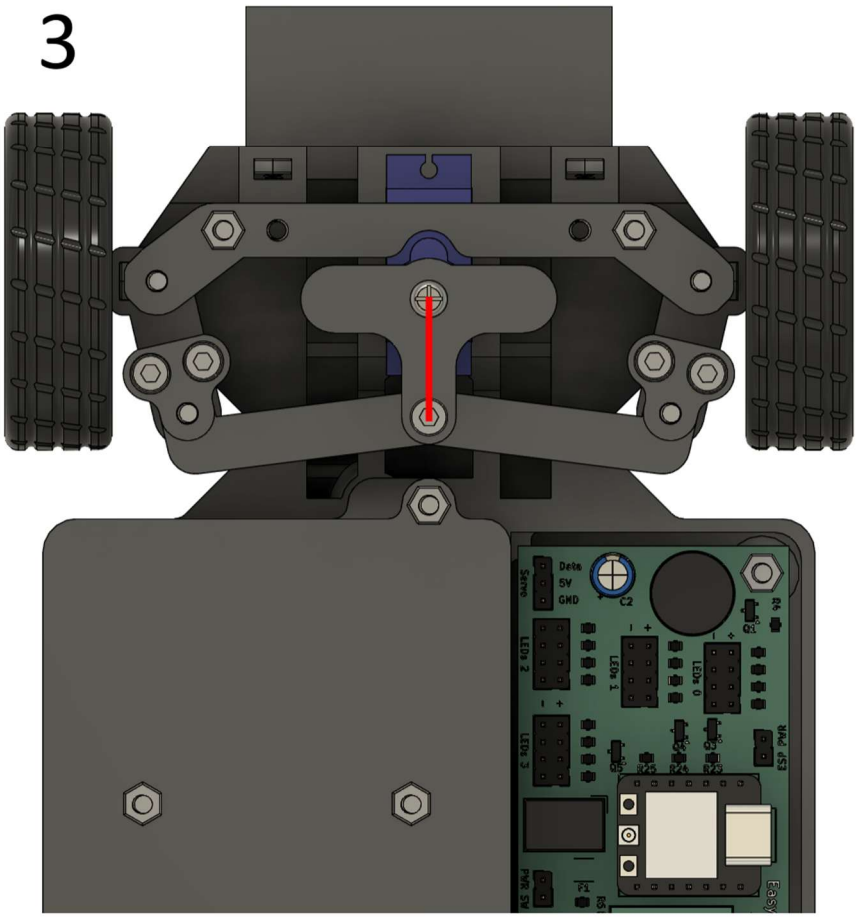


2



Fix with M2.5x6 screw!

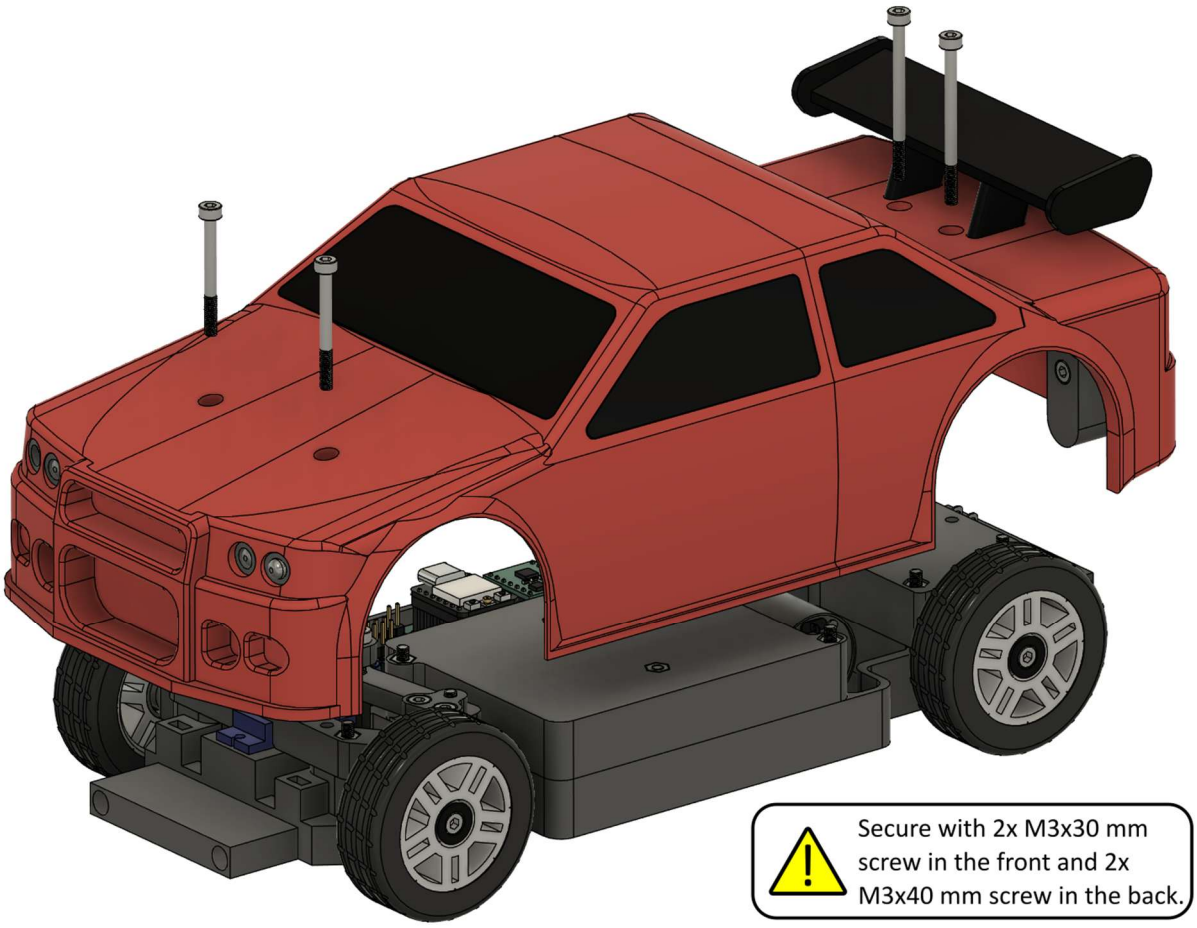
3



After setting the neutral position of the servo, the servo horn has to be pushed onto the shaft oriented as parallel as possible to the red line.

7.3 Finishing the assembly of the vehicle

The LEDs of the chassis are connected to the PCB, followed by mounting the body on top of the chassis. The LEDs are connected as described in Chapter 4.4.4. Important: Do not pinch any wires when securing the body with its screws. Furthermore, the wires must be routed with sufficient distance from the wheels to prevent them from touching. Do not forget to account for the steering angle of the front wheels.



7.4 Control principle and functions of the vehicle

The vehicle is controlled using the single joystick of the remote control. Moving the joystick forward lets the car accelerate forward. Moving the joystick backward lets the car drive in reverse. If this assignment is inverted, the variable "motordirection" in the remote control's code must be modified (see chapter 5.2).

When the joystick is moved to the left, the vehicle steers to the left; similarly, the vehicle steers to the right when the joystick is moved to the right. If the vehicle steers in an inverted manner, the values of the variables "leftlimit" and "rightlimit" in the remote control's code must be exchanged (see Chapter 5.2). The neutral steering position may not be correctly aligned before trimming the steering, which is performed in the following chapter.

The white headlight LEDs are permanently lit if the vehicle is switched on. The red rear lights shine with full intensity when the vehicle is standing still or in reverse. Their intensity is reduced when the vehicle drives forward. The indicator lights blink automatically when the joystick is moved to the left or to the right. If the assignment of the lights does not match this description, the wiring of the LEDs should be checked to ensure that all of them are connected to the correct LED header.

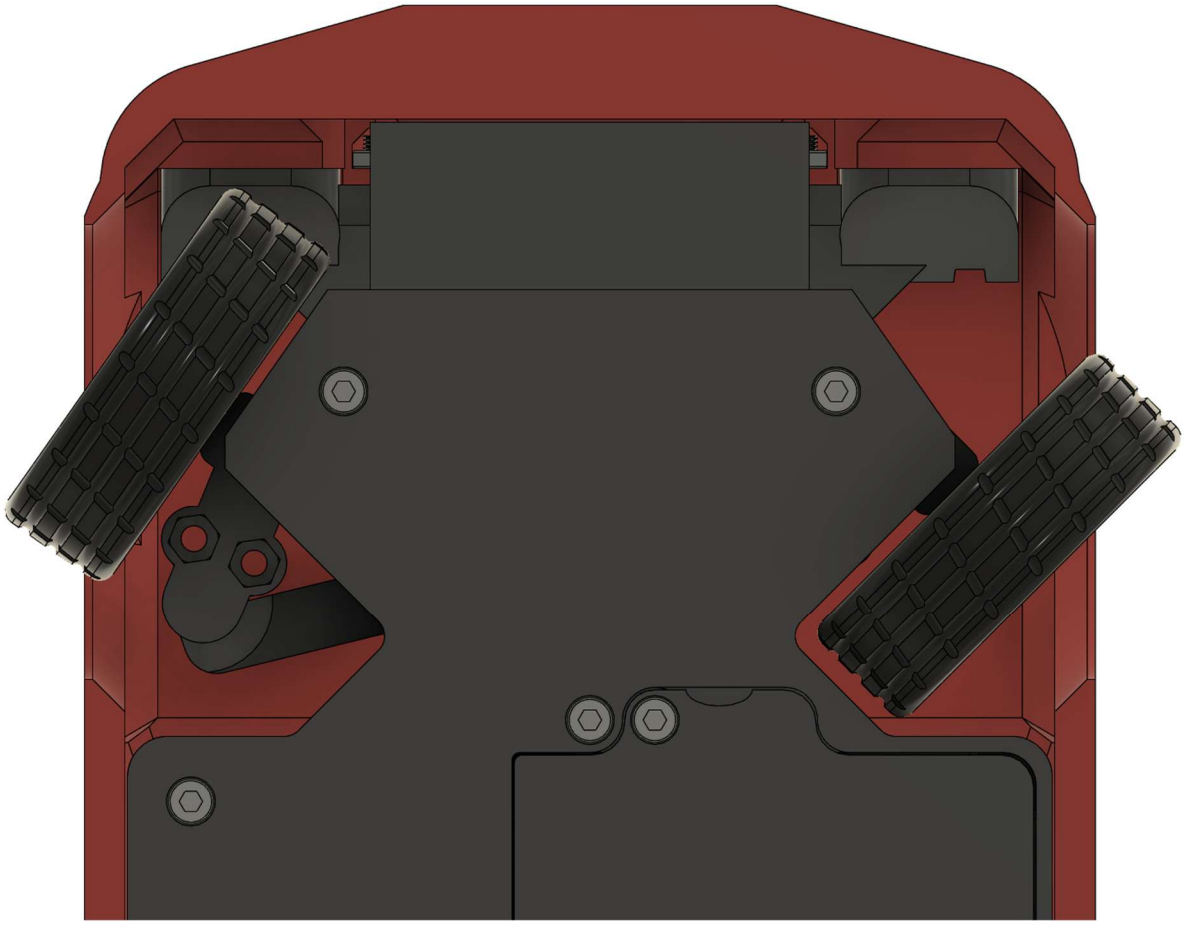
The piezo buzzer serves as both an acoustic warning for the system and a horn. Pressing the joystick produces a tone as long as the joystick remains pressed. However, the vehicle only honks when it is standing still. This function is disabled when the car is driving.

The horn and indicator lights also serve as a status indicator for the vehicle. Besides the initial sound and light sequence when switching on the car and connecting the remote control (see chapter 7.1), a sound sequence combined with activating the indicator lights is executed by the vehicle if the battery voltage is low, the connection to the remote control is lost, or when no user input has been recognized for a longer period of time. In all three cases, the drive motor is deactivated, and no further data signal is transmitted to the steering servo. The latter two cases are reversible, meaning the alert status can be deactivated by reconnecting the remote control or providing user inputs. The low battery warning can only be reset by restarting the vehicle.

Cause	**Beep and LED frequency**
Battery voltage drops below 10.2 V	Indicators blink, and the horn sounds every 3 s for 0.5 s
No user input for at least 60 s	Indicators blink, and the horn sounds every 3 s for 1.0 s
Signal from the remote control lost for at least 1 s	Indicators blink, and the horn sounds every 3 s for 1.5 s

7.5 Trimming the steering

The steering is adjusted by driving the car in a straight line by pushing the joystick forward. If the car pulls to one side, the variables "leftlimit" and "rightlimit" must be altered. The maximum steering angle must not exceed 45° for the inner wheel of the turn (see the figure below). As long as the maximum steering angle is not yet reached, pulling to the left can be corrected by increasing the difference between the value of "rightlimit" and the mean value of the servo control (308) (analogously for the car pulling to the right by adjusting "leftlimit"). If the maximum steering angle is already reached, pulling to the left is corrected by setting "leftlimit" closer to the servo control's mean value (308) (analogously for the car pulling to the right by adjusting "rightlimit"). This modification of the steering can be executed directly within the code of the remote control, while the vehicle is switched on (see also Chapter 5.2). Hence, a rapid adjustment is possible without the need to disassemble anything.



7.6 Fine-tuning the driving characteristics

The vehicle is basically ready to run as soon as the steering is calibrated. However, there are additional variables that influence the driving performance. Firstly, the vehicle's maximum speed can be adjusted in the remote control code using the variable "speedlimit" (see Chapter 5.2). The maximum value is 255, which results in a speed of approximately 10 km/h. This value can theoretically be lowered to 0, but the speed does not scale linearly with the value of this variable because it sets the maximum voltage for the motor (set by pulse width modulation). A value of around 120 to 150 is a reasonable compromise between high maximum speed and good controllability: Very high speeds make cornering and braking difficult to master.

Another modification option is the sensitivity with which the motor reacts to joystick inputs. The variable "motorint" (see Chapter 5.2) in the remote control code defines the number of milliseconds that pass between refreshing the desired speed input. A lower value means that the vehicle reacts quickly to abrupt joystick inputs. This responsive behavior harbors the disadvantage that the wheels tend to spin, which deteriorates controllability, particularly when braking. Increasing this value allows the motor to react more slowly to input changes, which improves traction but also means that the vehicle accelerates more slowly and requires more time to brake. A value of 10 is set as the default for this variable. This value can be increased or decreased in steps until the preferred compromise between responsiveness and controllability is achieved.

These adjustments are performed in the code of the remote control, similarly to trimming the steering, which means that the vehicle does not have to be switched off or disassembled when conducting these changes.

8. Possible errors and their solutions

Error description	Cause	Solutions
Status LED of the remote control does not light up after switching on.	Battery inserted incorrectly.	Confirm correct polarity. Also, check the connection between the circuit board and the battery holder!
Status LED of the remote control blinks twice per second.	Battery voltage is low (<3.4 V).	Charge battery.
Status LED of the remote control flickers erratically or with reduced brightness.	Battery voltage too low for correct operation of the remote control (<3.0 V).	Check battery voltage. If it is below 2.5 V, the cell was deeply discharged and needs to be replaced. Otherwise, charge the battery.
There is no beep, and no LED lights up after switching on the car.	At least one of the three batteries is inserted incorrectly.	Confirm correct polarity. Also, check the connection between the circuit board and the battery holder!
There is no beep, and no LED lights up after switching on the car.	The batteries are dead (the three together provide <7 V, meaning less than 2.3 V per cell).	Check batteries individually and replace broken/deeply discharged cells.
The sound sequence for the connection with the remote control does not sound (the vehicle beeps every 2 s although a switched-on remote control is nearby).	The hardware addresses between the remote control and the vehicle are not defined correctly (i.e., not identical).	Lines 27 and 28 in the remote control code and lines 23 and 24 in the vehicle code must be identical. See Chapter 9.1.
The sound sequence for the connection with the remote control does not sound (the vehicle beeps every 2 s although a switched-on remote control is nearby).	The antennas are not correctly installed in the connectors of the microcontrollers.	Check the antenna connectors for a secure fit. The radio range without the external antennas is extremely low.
The sound sequence for the connection with the remote control does not sound (the vehicle beeps every 2 s although a switched-on remote control is nearby).	Strong interference due to WLAN.	Change the radio channel of the EasyRC platform. See Chapter 9.2.
The vehicle drives without the joystick of the remote control being moved.	Joystick not correctly calibrated.	Switch the remote control off and on. Do not move the joystick from its neutral position while doing so.
The vehicle turns, although the joystick is only moved to the front or rear.	Steering is not correctly trimmed.	Follow the guideline in Chapter 7.5 to trim the steering.
The vehicle turns, although the joystick is only moved to the front or rear, and/or does not turn despite receiving joystick inputs.	The servo motor is broken (e.g., due to a previous collision with the front wheels).	Replace the servo motor, then follow the steps to mount and trim the steering (see Chapters 7.2 and 7.5).

9. Appendix

9.1 Changing the hardware addresses of the EasyRC platform

The hardware addresses of the EasyRC platform can be changed to allow for the simultaneous operation of multiple EasyRC systems. Importantly, there must always be a pair of a transmitter and a receiver that use the same hardware addresses. The vehicle stores the hardware address of the paired remote control and checks if an incoming data packet has been sent from a device with the correct hardware address. Therefore, the variables CarMAC and RemoteMAC must be changed in both the microcontrollers of the remote control and the vehicle when a second EasyRC system is to be operated.

The hardware addresses are stored in the remote control in lines 27 and 28 of the code. In the vehicle, they can be found in lines 23 and 24. A hardware address consists of six pairs of characters that can be numbers and letters. The figure shows the hardware addresses of the code in the remote control. It is sufficient to change a single character of a character pair to change a hardware address.

```
26  //MAC addresses
27  uint8_t CarMAC[] = {0xAA, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
28  uint8_t RemoteMAC[] = {0xB0, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control
29  uint8_t broadMAC[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}; //using this MAC address as a receiver for sending
30  //packages means broadcasting (ignoring if the data has been received)
31
```

Below, there is a small selection of possible hardware addresses that can replace the code lines directly. The uppermost set is the default hardware addresses. The selected pair has to be stored both in the remote control and in the vehicle.

uint8_t CarMAC[] = {0xAA, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xB0, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control

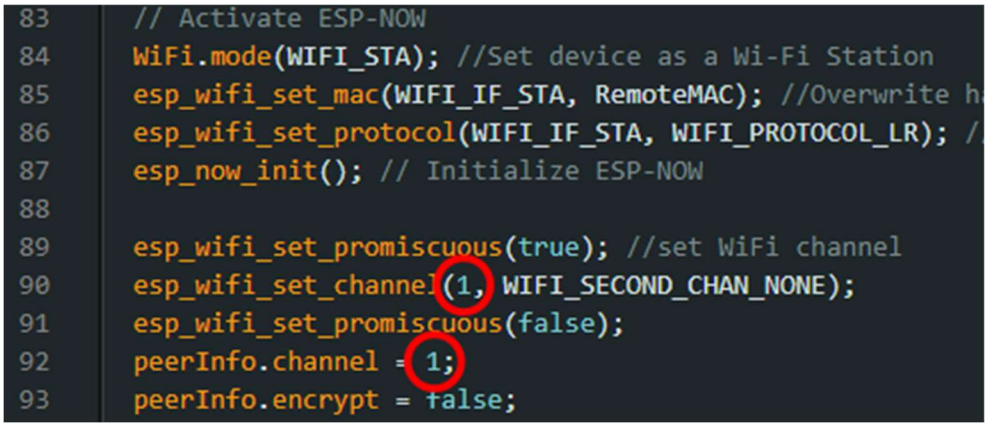
uint8_t CarMAC[] = {0xA0, 0xA0, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xBA, 0xAD, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control

uint8_t CarMAC[] = {0xAA, 0xAA, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xB0, 0xAA, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control

uint8_t CarMAC[] = {0xA0, 0xAA, 0xBE, 0xEF, 0xFE, 0xED}; //we will set this MAC address for the car
uint8_t RemoteMAC[] = {0xBA, 0xAA, 0xBE, 0xEF, 0xFE, 0xEF}; //we will set this MAC address for the remote control

9.2 Changing the radio channel of the EasyRC platform

The radio channel is set in lines 90 and 92 (remote control, depicted in the figure) and in lines 124 and 126 (vehicle) by altering the numbers that are marked in red. The platform's default setting is channel 1. Values between 1 and 13 can be selected. Important: A pair of vehicle and remote control must use the same channel. Otherwise, no stable connection is possible.



10. Improvements to be implemented

- Connector for any external motor driver: The vehicle circuit board shall be equipped with a connector that provides GND, as well as the two pins that control drive direction and motor speed. Then, an external motor driver with higher power can be used. Alternatively, an output with 5 V, one PWM pin, and GND could be integrated to use RC motor drivers. However, they come with the limitation of an internal logic that converts control inputs into motor speed and rotation direction, which can only be adjusted to a limited extent (if at all).
- A diode instead of the jumper to prevent reverse currents from the USB port to the 5 V line of the circuit board of the vehicle. This replacement should improve safety and user friendliness.
- Redo the annotation of the printed circuit boards.
- Develop alternative revisions of the vehicle and remote control that are larger and offer enhanced performance.

11. Imprint

EasyRC is an open-source platform developed by Thomas Böhm. All relevant information and updates on the project are available in the corresponding GitHub repository:
https://github.com/TRD-B/EasyRC

The author accepts no liability for damage or injury that may arise from the use of the hardware and software provided!